Chapter 3
Transport Layer

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Chapter 3: Transport Layer

our goals:

- understand principles behind transport layer services:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control
- learn about Internet transport layer protocols:
  - UDP: connectionless transport
  - TCP: connection-oriented reliable transport
  - TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

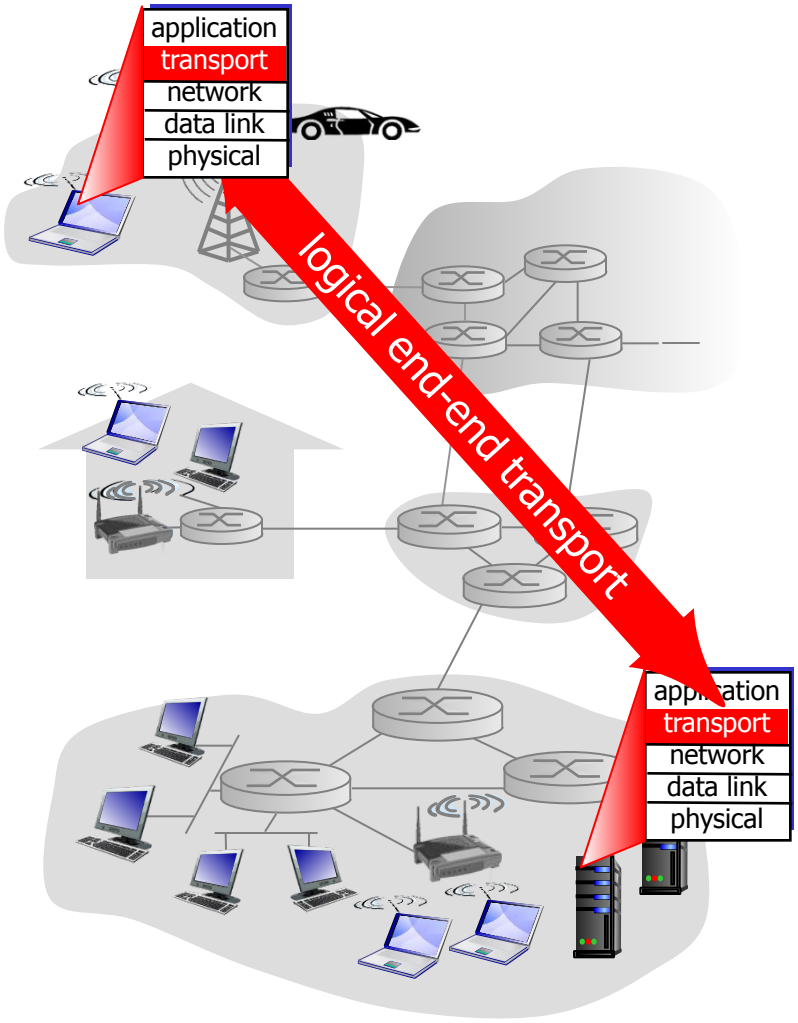
3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

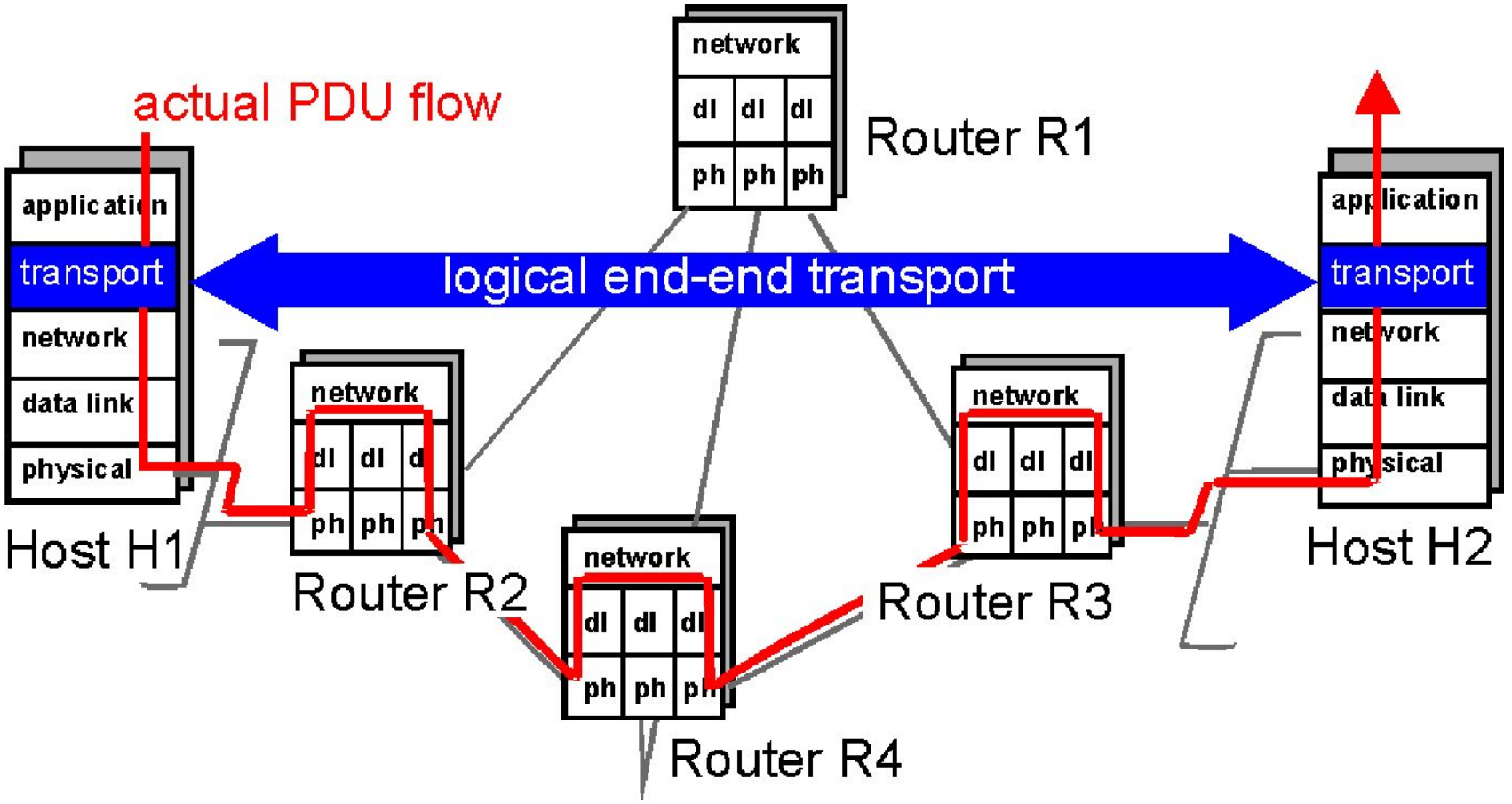
Transport services and protocols

- provide *logical communication* between app processes running on different hosts
- transport protocols *run in end systems*
  - send side: breaks app messages into *segments*, passes to network layer
  - rcv side: reassembles segments into messages, passes to app layer
- *more than one transport protocol* available to apps
  - Internet: TCP and UDP



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Transport services and protocols



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Transport vs. network layer

- *network layer:* logical communication between hosts
- *transport layer:* logical communication between processes
  - relies on, enhances, network layer services

household analogy:

12 kids in Ann's house sending letters to 12 kids in Bill's house (cousins):

- app messages = letters in envelopes
- processes = kids
- hosts = houses
- transport protocol = Ann and Bill who mux/demux to in-house siblings
- network-layer protocol = postal service

© J.F. Kurose, K.W. Ross. All Rights Reserved

Transport vs. network layer

household analogy (contd.):

- Ann and Bill – *work in their respective homes only*
- Ann and Bill replaced by Susan and Harvey
- Susan and Harvey – younger kids *unreliable*
- Ann and Bill services *constrained by postal services* does not provide max bound on delay

household analogy (contd.):

- Certain services can be offered by a transport protocol even when the underlying network protocol does not offer the corresponding service at the network layer.
  - Reliable data transfer service
  - Use encryption to provide security

© J.F. Kurose, K.W. Ross. All Rights Reserved

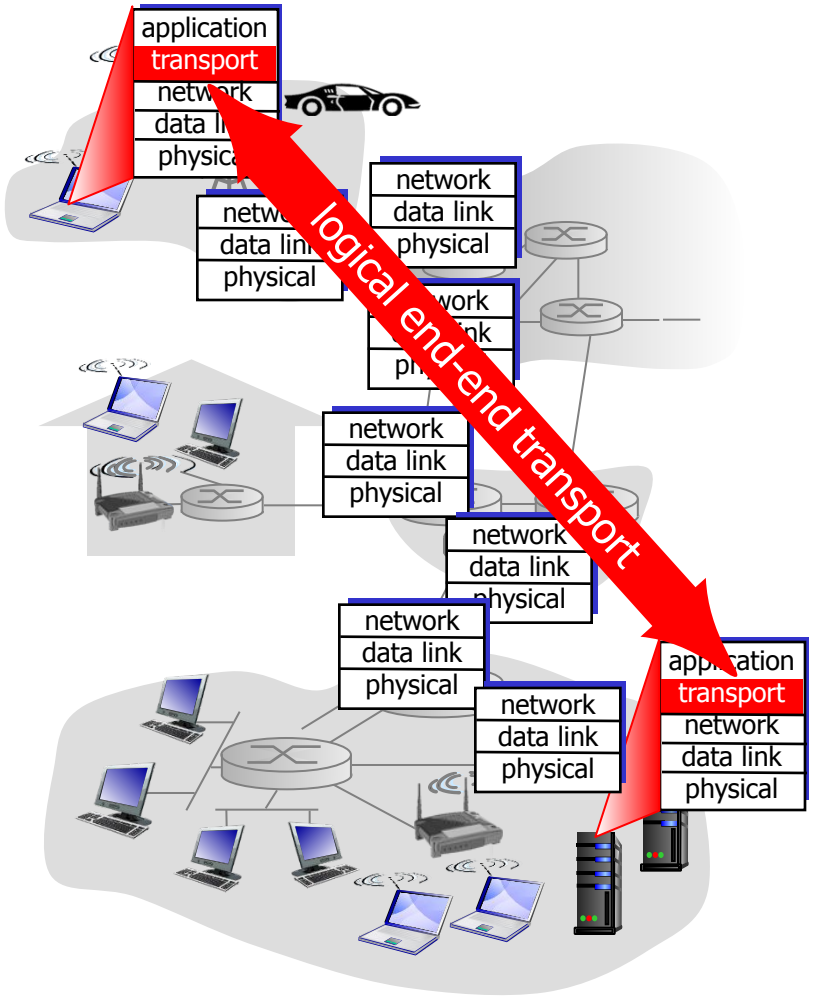
Internet network-layer protocol

- Internet Protocol (IP)
- Provide logical communication between hosts
- Best effort delivery service
- No guarantees
  - No guarantees of segment delivery
  - No guarantees of orderly delivery of segments
  - No guarantee of integrity of data in the segment
- Unreliable service
- Each host has an IP address

© J.F. Kurose, K.W. Ross. All Rights Reserved

Internet transport-layer protocols

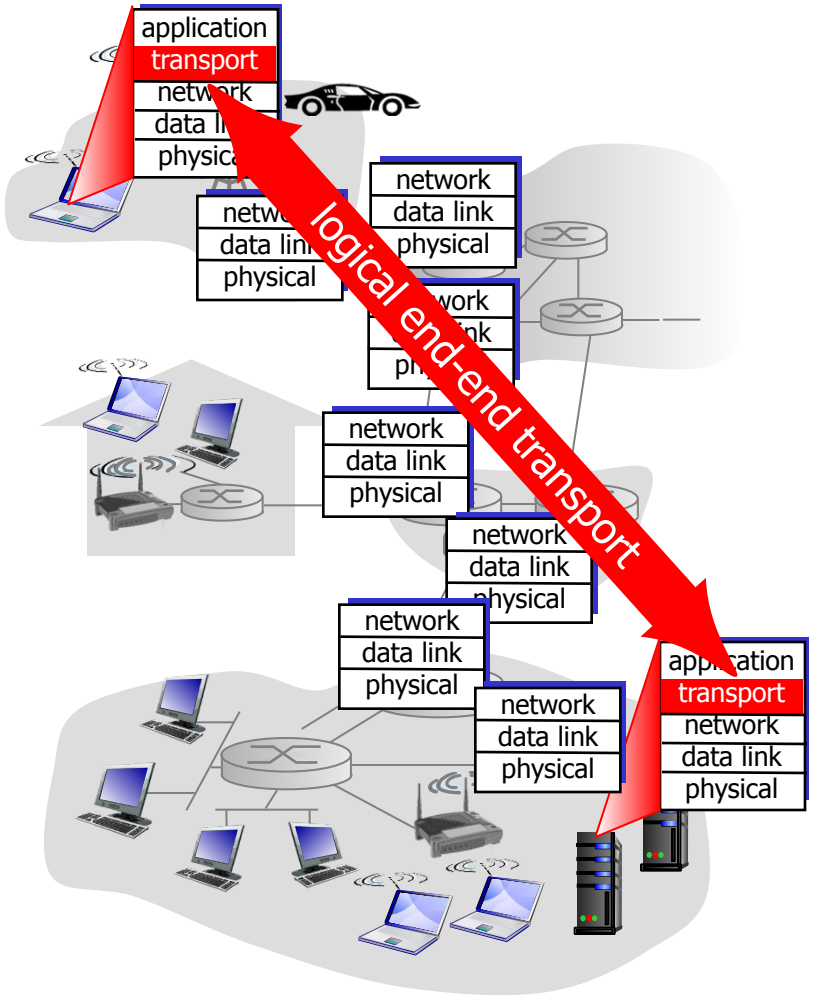
- Responsibilities of both UDP and TCP
  - Process-to-Process Delivery
    - To extend IP's delivery service between two end systems to a delivery service between two processes running on the end systems - Transport layer multiplexing and demultiplexing
  - Error Checking
    - By including error detection fields in their segment headers



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Internet transport-layer protocols

- Additional Services provided by TCP
  - reliable, in-order delivery
    - sequence numbers, acknowledgements and timers
  - congestion control
  - flow control
  - connection setup
  - complex
- unreliable, unordered delivery: UDP
  - no-frills extension of "best-effort" IP
  - Error checking
- services not available:
  - delay guarantees
  - bandwidth guarantees



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

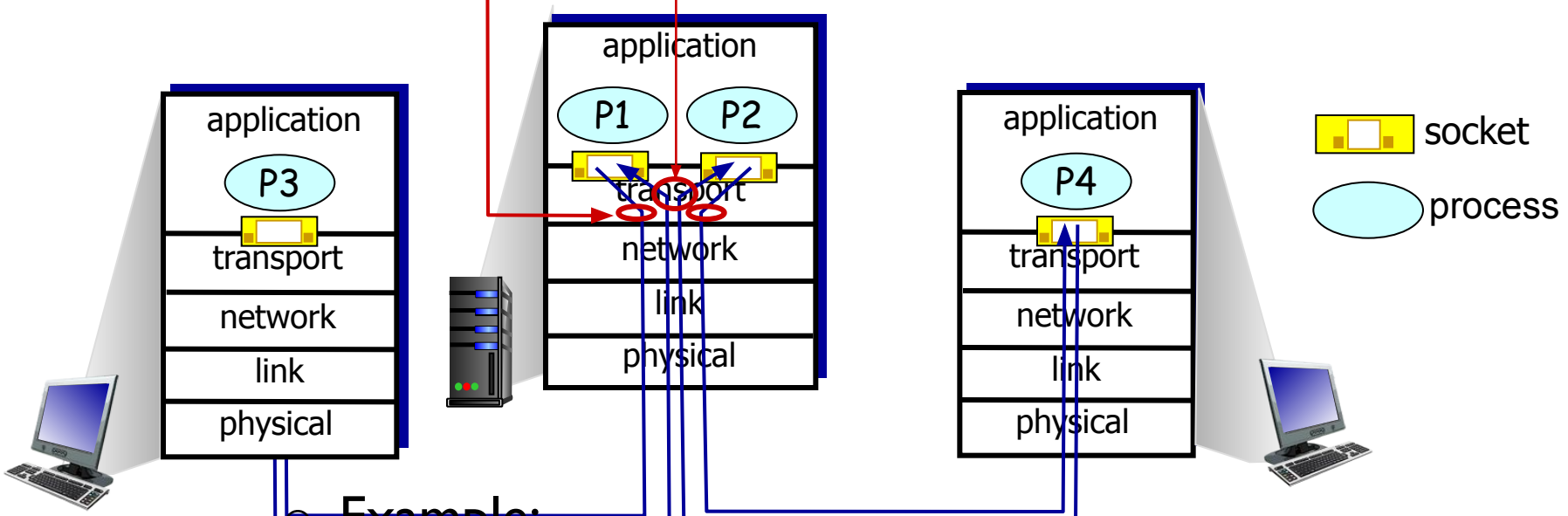
3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

Multiplexing/demultiplexing

multiplexing at sender:
handle data from multiple sockets, add transport header (later used for demultiplexing)

demultiplexing at receiver:
use header info to deliver received segments to correct socket

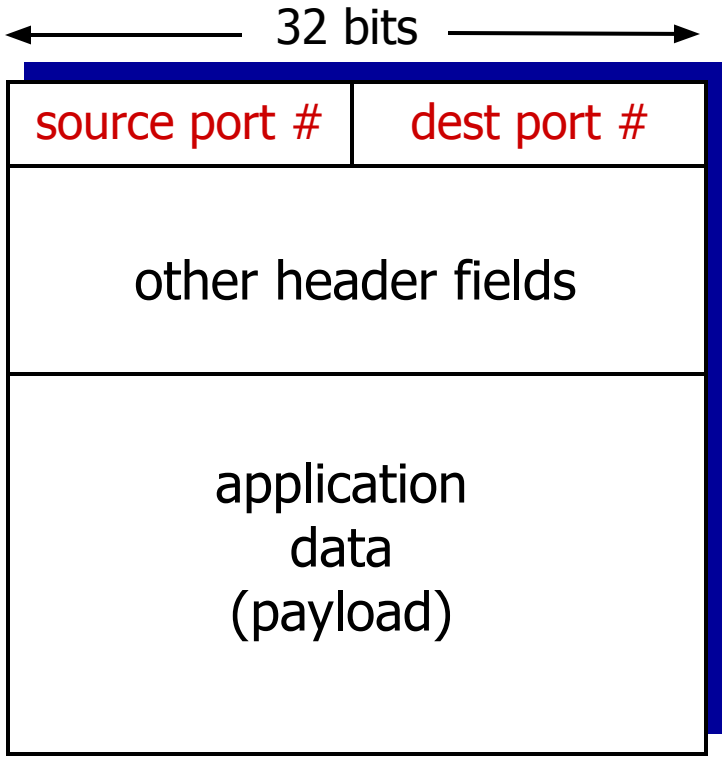


- Example: http,ftp,telnet
- Household example

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

How demultiplexing works

- Transport layer multiplexing requires
  - Sockets have unique ID
  - Each segment has special fields
- host receives IP datagrams
  - each datagram has source IP address, destination IP address
  - each datagram carries one transport-layer segment
  - each segment has source, destination port number
- host uses *IP addresses & port numbers* to direct segment to appropriate socket



TCP/UDP segment format

© J.F. Kurose, K.W. Ross. All Rights Reserved

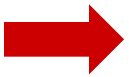
Connectionless multiplexing and demultiplexing

- *recall:* created socket has host-local port #:

```
clientSocket=socket(socket.AF_I
  NET, socket.SOCK_DGRAM)
clientSocket.bind((' ', 19157))
```

- *recall:* when creating datagram to send into UDP socket, must specify *(two tuples)*
  - destination IP address
  - destination port #

- when host receives UDP segment:
  - checks destination port # in segment
  - directs UDP segment to socket with that port #



Two UDP segments with *different source IP addresses and/or source port#,* but with *same dest. IP address and same dest. port #* will be directed to *same dest process via same dest socket*

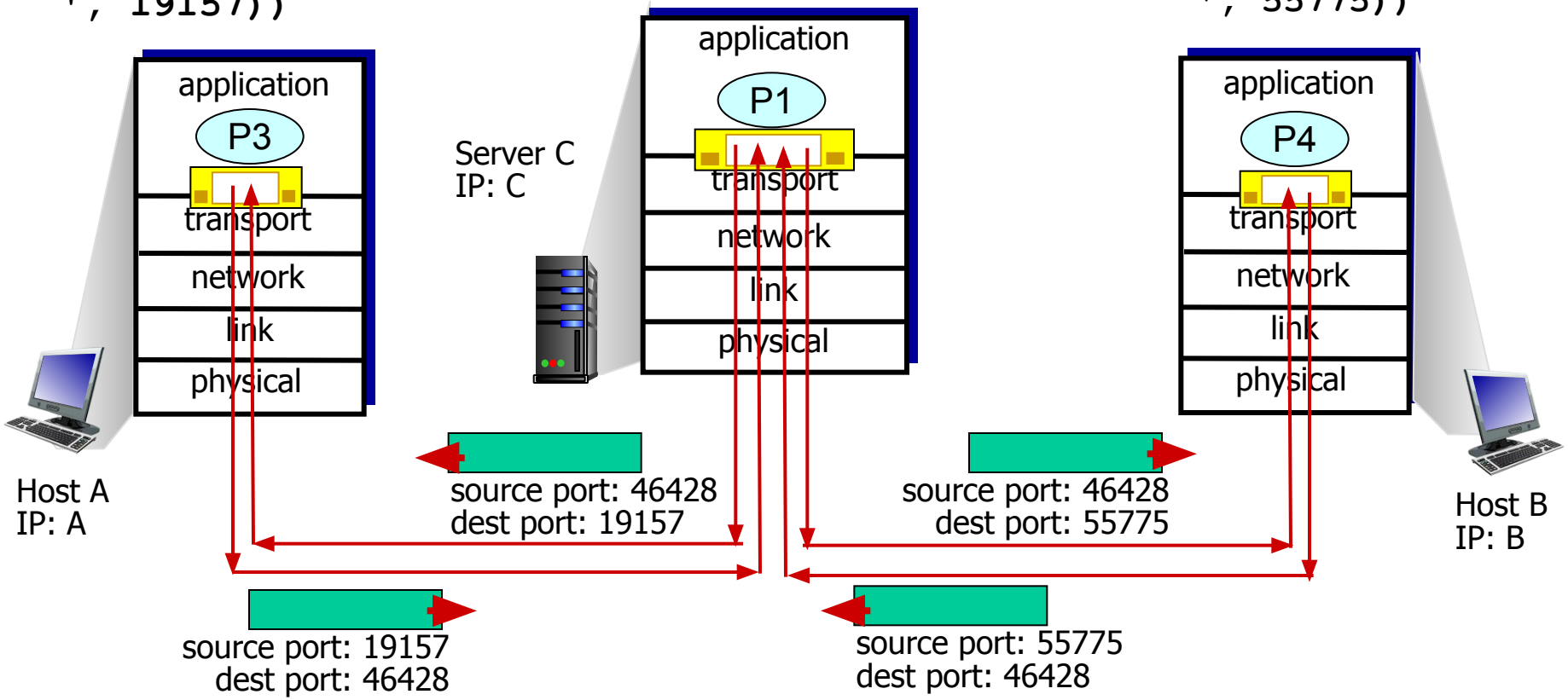
© J.F. Kurose, K.W. Ross. All Rights Reserved

Connectionless mux and demux: example

```
clientSocket=socket(so
   cket.AF_INET,
   socket.SOCK_DGRAM)
clientSocket.bind(('
   ', 19157))
```

```
serverSocket=socket(A
   F_INET,SOCK_DGRAM)
serverSocket.bind(('
   ', 46428))
```

```
clientSocket=socket(
   socket.AF_INET,
   socket.SOCK_DGRAM)
clientSocket.bind(('
   ', 55775))
```



Source Port provides "return address"

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Connection-oriented mux and demux

- TCP socket identified by 4-tuple:
  - source IP address
  - source port number
  - dest IP address
  - dest port number
- demux: receiver uses all four values to direct segment to appropriate socket
- server host may support many simultaneous TCP sockets:
  - each socket identified by its own 4-tuple
- web servers have different sockets for each connecting client
  - non-persistent HTTP will have different socket for each request

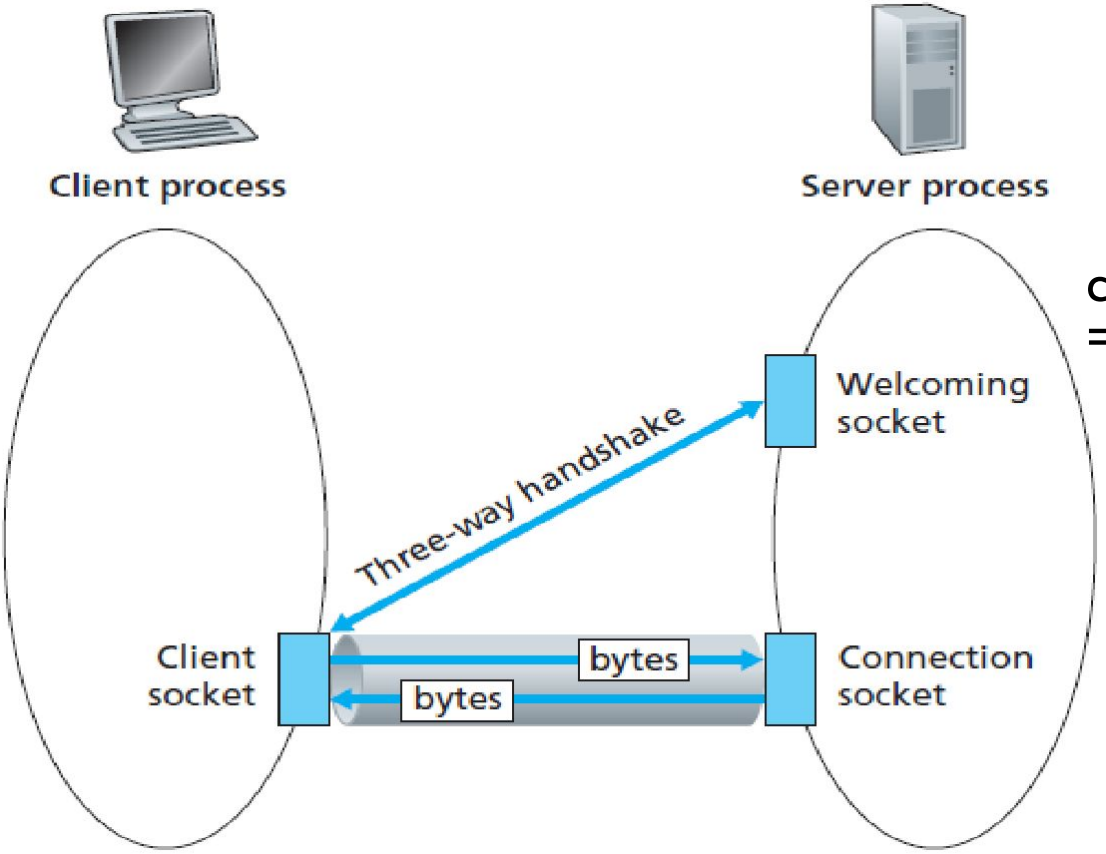
© J.F. Kurose, K.W. Ross. All Rights Reserved

Connection-oriented mux and demux

- Two arriving TCP segments with different source IP addresses or source port numbers will (with the exception of a TCP carrying the original connection establishment request) will be directed to two different sockets.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Connection-oriented mux and demux



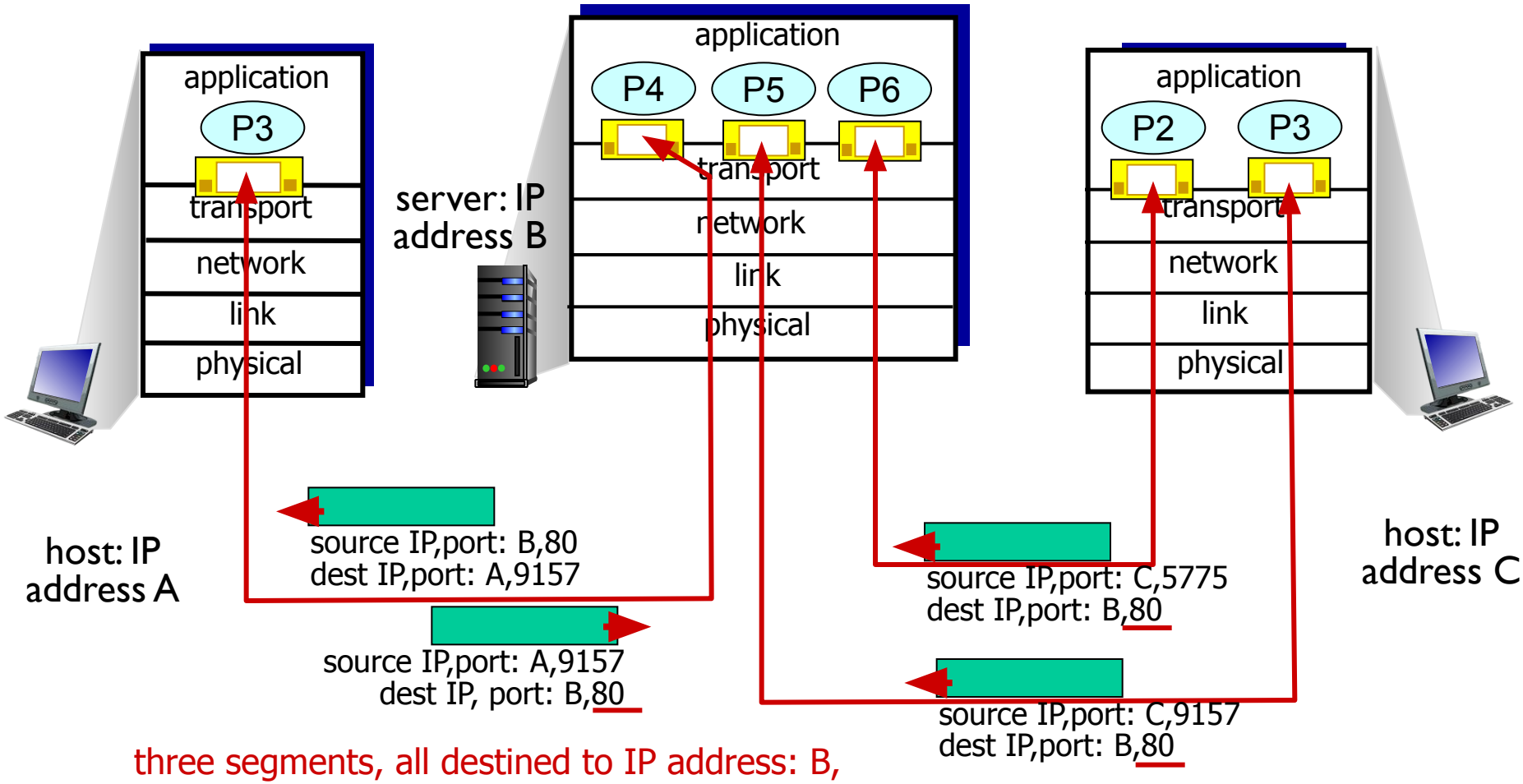
Server running at port no. 12000

connectionSocket, addr = serverSocket.accept()

clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName,12000))

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

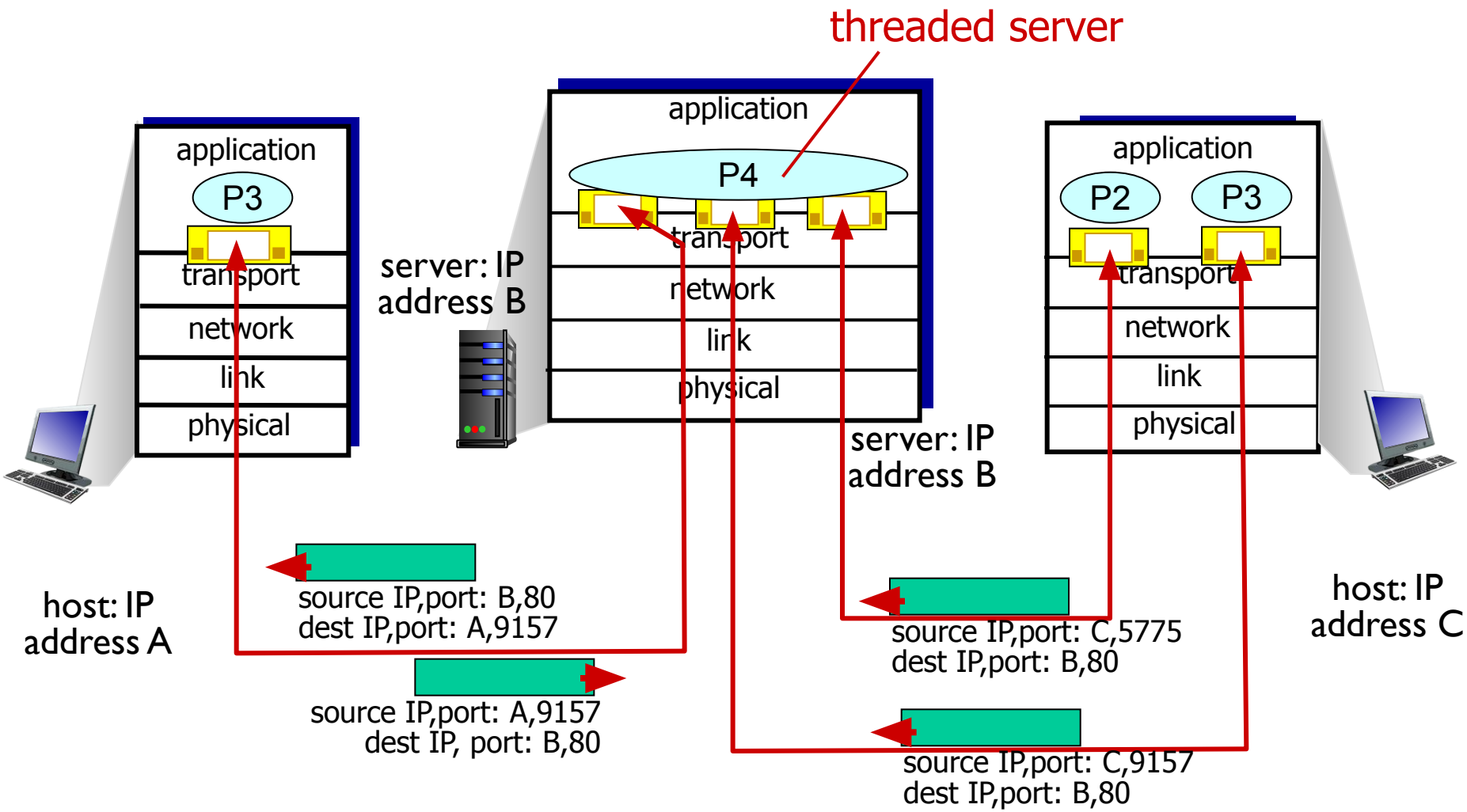
Connection-oriented mux and demux: example



three segments, all destined to IP address: B, dest port: 80 are demultiplexed to *different* sockets

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Connection-oriented mux and demux: example



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

UDP: User Datagram Protocol [RFC 768]

- "no frills," "bare bones" Internet transport protocol
- "best effort" service, UDP segments may be:
  - lost
  - delivered out-of-order to app
- *connectionless:*
  - no handshaking between UDP sender, receiver
  - each UDP segment handled independently of others

- UDP use:
  - streaming multimedia apps (loss tolerant, rate sensitive)
  - DNS
  - SNMP
- reliable transfer over UDP:
  - add reliability at application layer
  - application-specific error recovery!

© J.F. Kurose, K.W. Ross. All Rights Reserved

UDP: User Datagram Protocol [RFC 768]

why is there a UDP?

- finer application-level control over what data is sent and when
- no connection establishment (which can add delay)
- simple: no connection state at sender, receiver
  - receive and send buffers, congestion control parameters, sequence and acknowledgement number parameters
- small header size
- no congestion control: UDP can blast away as fast as desired

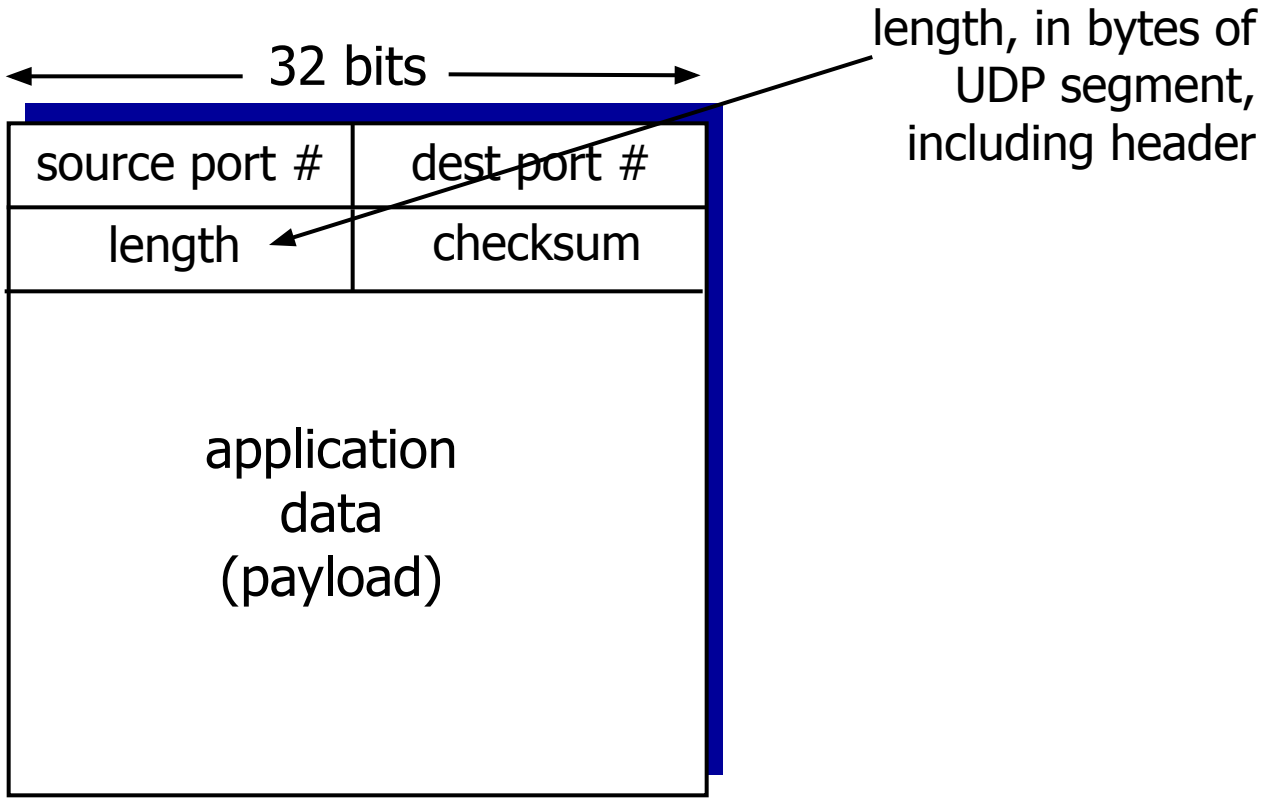
© J.F. Kurose, K.W. Ross. All Rights Reserved

Internet apps: application, transport protocols

Application	Application Layer Protocol	Underlying Transport Protocol
E-mail	SMTP	TCP
Remote terminal access	Telnet	TCP
Web	HTTP	TCP
File Transfer	FTP	TCP
Remote file server	NFS	Typically UDP
Streaming multimedia	Typically proprietary	UDP or TCP
Internet telephony	Typically proprietary	UDP or TCP
Network management	SNMP	Typically UDP
Routing protocol	RIP	Typically UDP
Name Resolution	DNS	Typically UDP

© J.F. Kurose, K.W. Ross. All Rights Reserved

UDP: segment header



UDP segment format

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

UDP checksum

Goal: detect "errors" (e.g., flipped bits) in transmitted segment

sender:

- treat segment contents, including header fields, as sequence of 16-bit integers
- checksum: addition (one's complement sum) of segment contents and then 1's complement of the sum
- sender puts checksum value into UDP checksum field

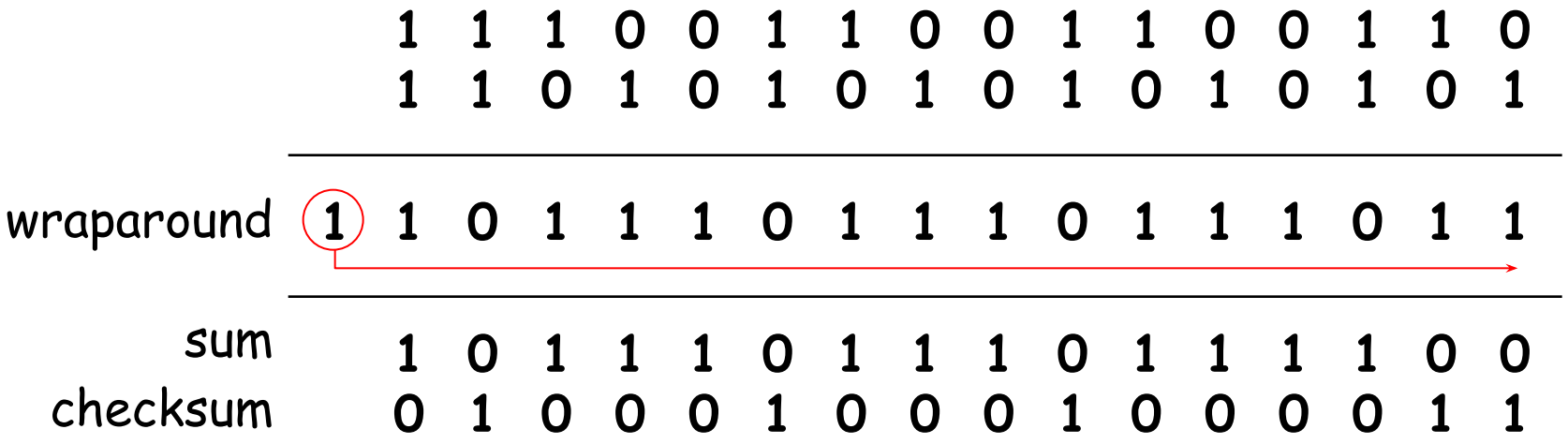
receiver:

- compute checksum of received segment
- check if computed checksum equals checksum field value:
  - NO - error detected
  - YES - no error detected. *But maybe errors nonetheless?* More later ….

© J.F. Kurose, K.W. Ross. All Rights Reserved

Internet checksum: example

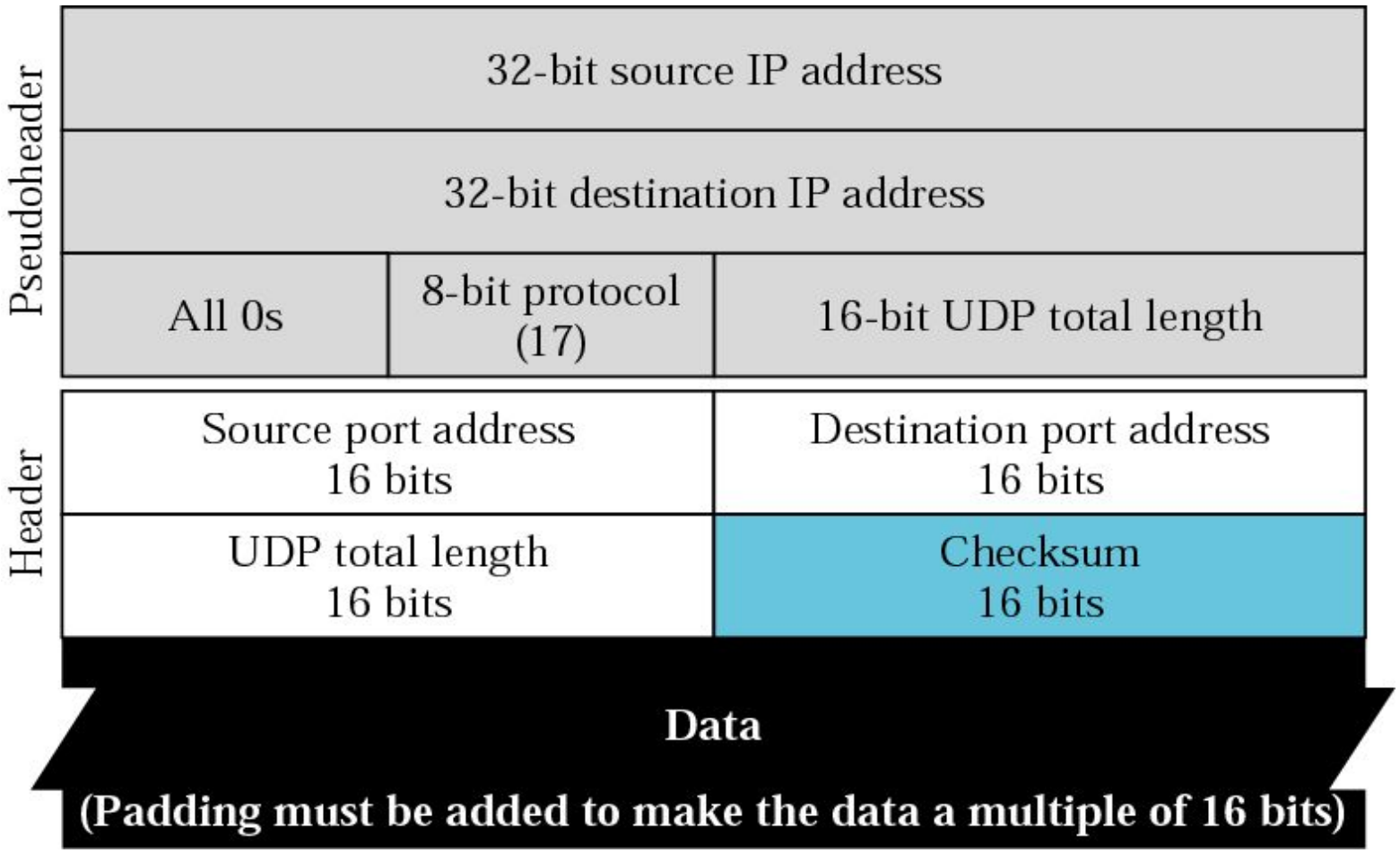
example: add two 16-bit integers



Note: when adding numbers, a carryout from the most significant bit needs to be added to the result

© J.F. Kurose, K.W. Ross. All Rights Reserved

Fields for Checksum Calculation



© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

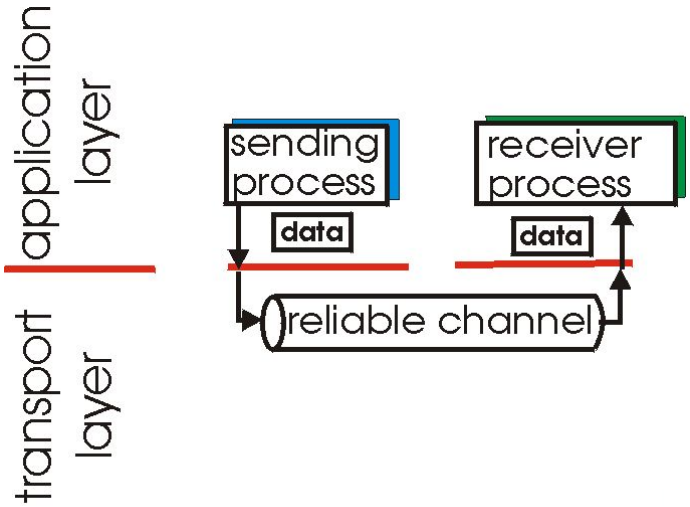
3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!
- Framework



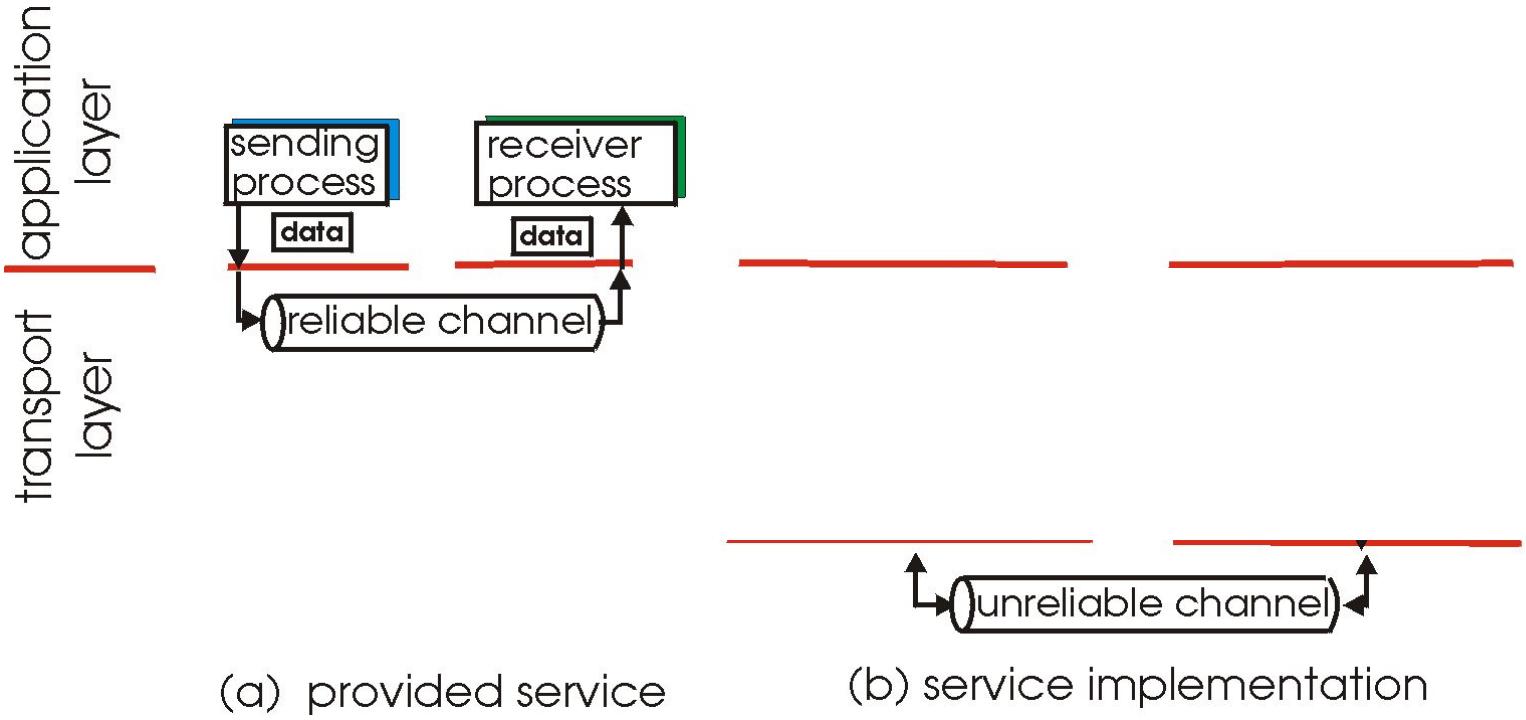
(a) provided service

- characteristics of unreliable channel will determine complexity of *reliable data transfer* protocol (rdt)

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!

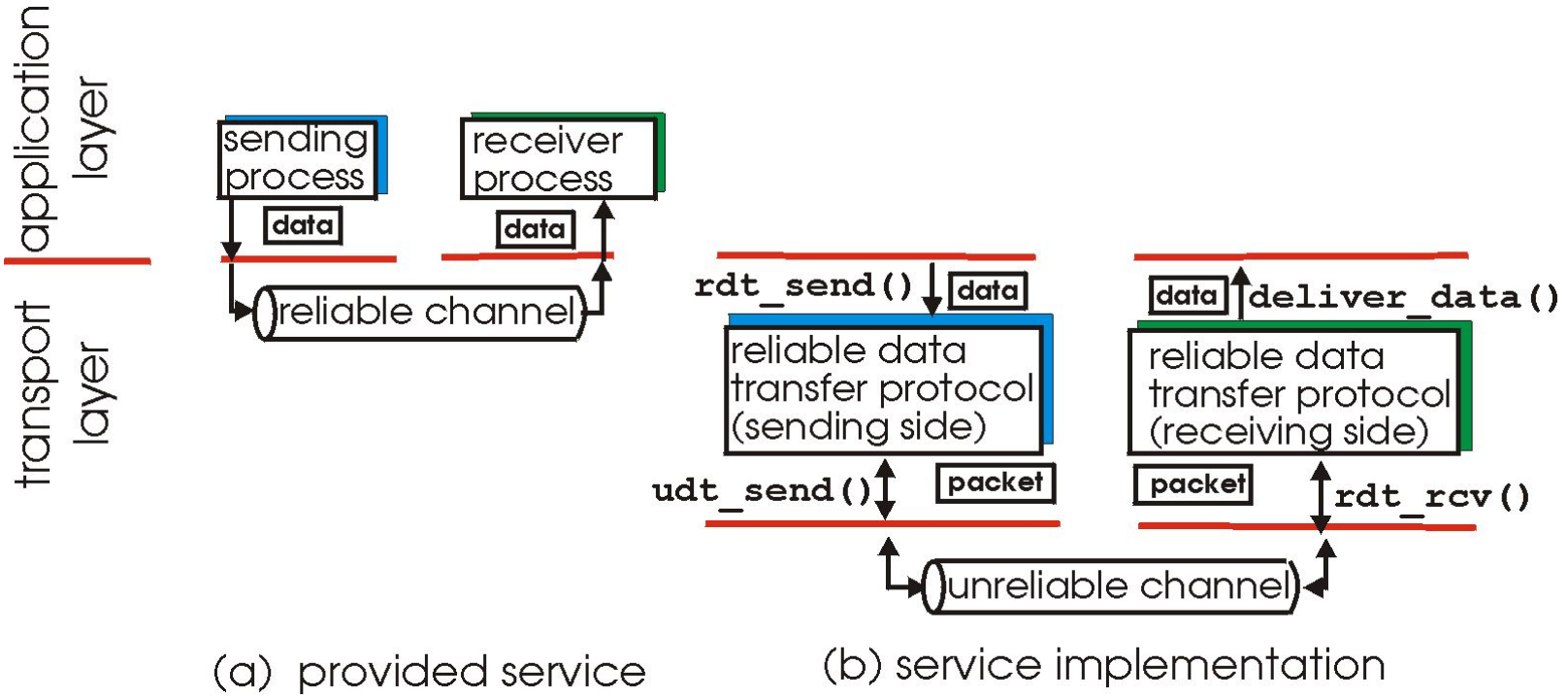


- characteristics of unreliable channel will determine complexity of *reliable data transfer* protocol (rdt)

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Principles of reliable data transfer

- important in application, transport, link layers
  - top-10 list of important networking topics!



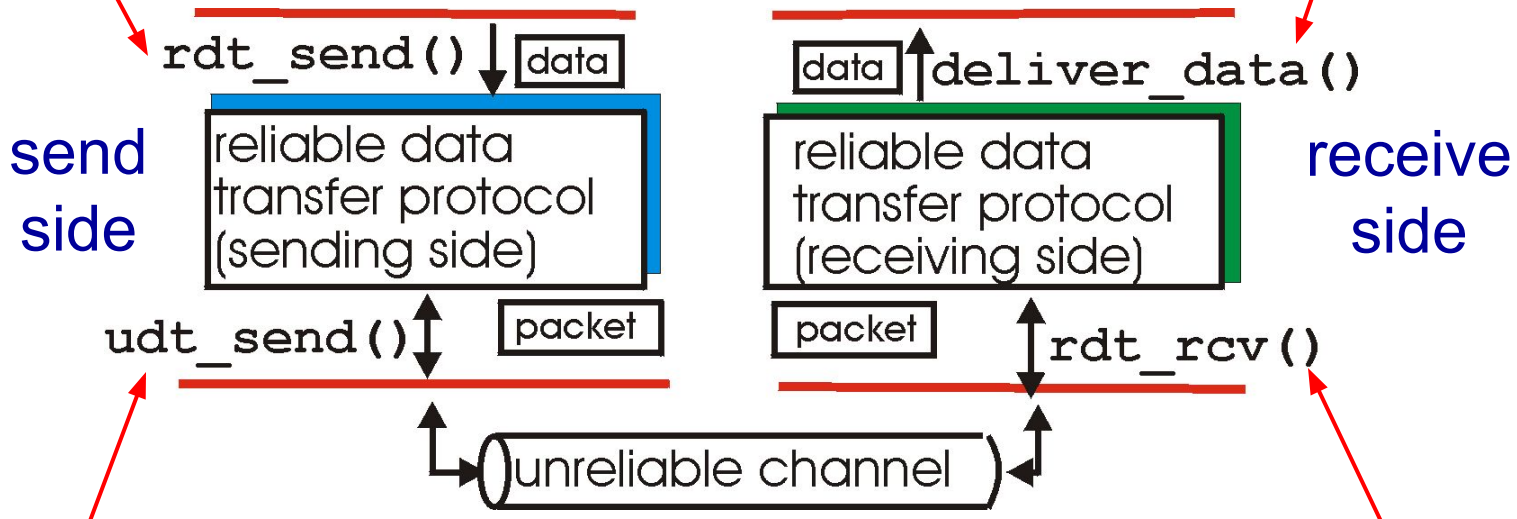
- characteristics of unreliable channel will determine complexity of *reliable data transfer* protocol (rdt)

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Reliable data transfer: getting started

`rdt_send()`: called from above, (e.g., by app.). Passed data to deliver to receiver upper layer

`deliver_data()`: called by **rdt** to deliver data to upper



`udt_send()`: called by rdt, to transfer packet over unreliable channel to receiver

`rdt_rcv()`: called when packet arrives on rcv-side of channel

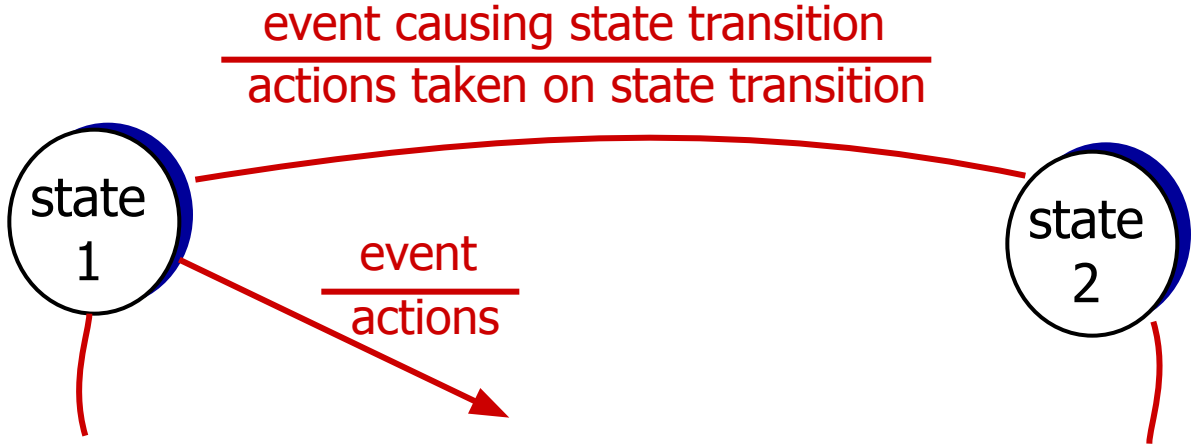
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Reliable data transfer: getting started

we'll:

- incrementally develop sender, receiver sides of reliable data transfer protocol (rdt)
- consider only unidirectional data transfer
  - but control info will flow on both directions!
- use finite state machines (FSM) to specify sender, receiver

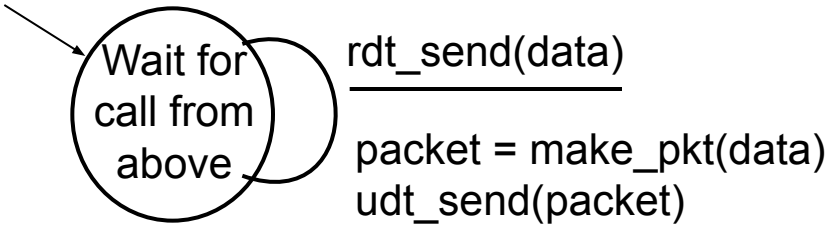
state: when in this "state" next state uniquely determined by next event



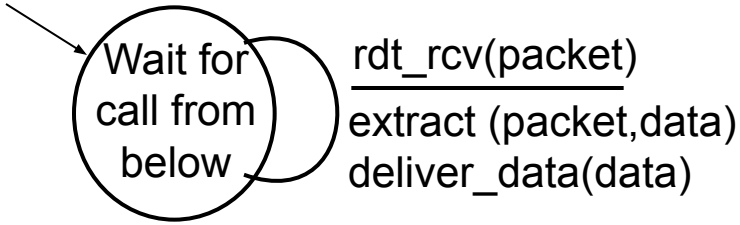
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt1.0: reliable transfer over a reliable channel

- underlying channel perfectly reliable
  - no bit errors
  - no loss of packets
- separate FSMs for sender, receiver:
  - sender sends data into underlying channel
  - receiver reads data from underlying channel



sender



receiver

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors:

How do humans recover from "errors" during conversation?

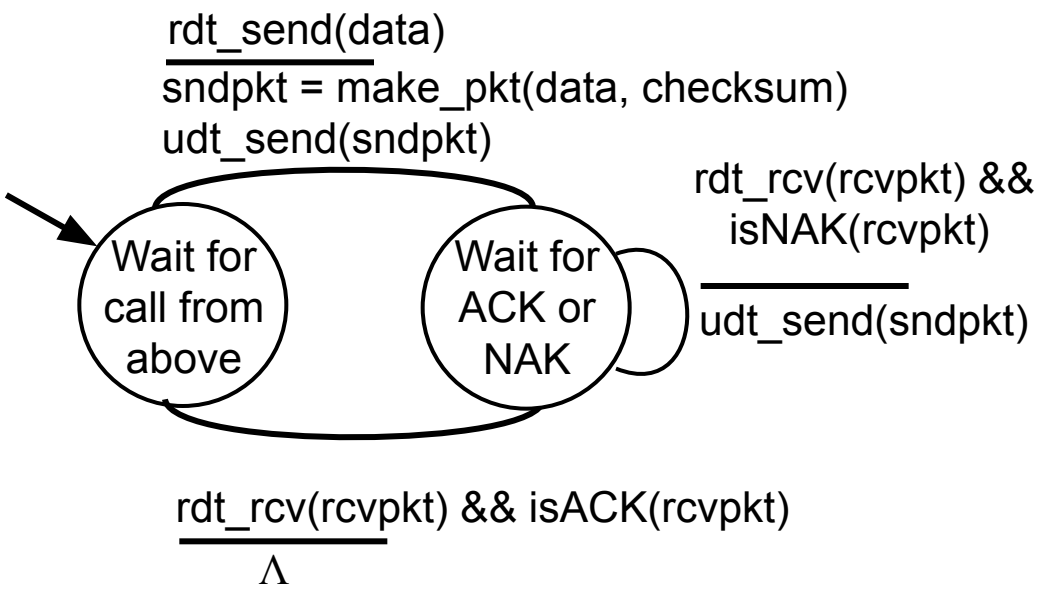
© J.F. Kurose, K.W. Ross. All Rights Reserved

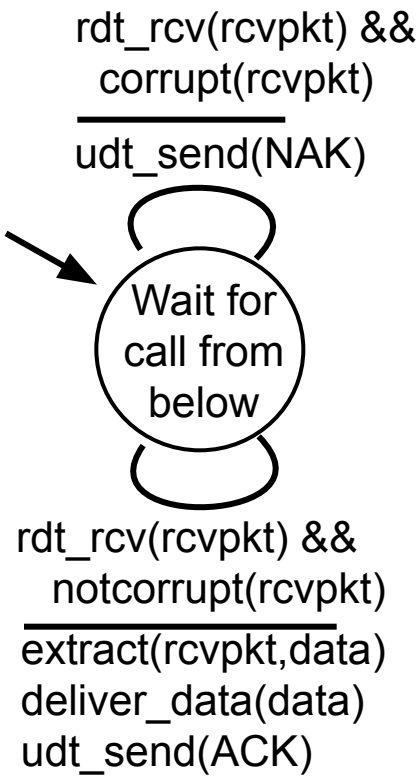
rdt2.0: channel with bit errors

- underlying channel may flip bits in packet
  - checksum to detect bit errors
- *the* question: how to recover from errors: ARQ
  - *acknowledgements (ACKs):* receiver explicitly tells sender that pkt received OK
  - *negative acknowledgements (NAKs):* receiver explicitly tells sender that pkt had errors
  - sender retransmits pkt on receipt of NAK
- new mechanisms in `rdt2.0` (beyond `rdt1.0`):
  - error detection -UDP checksum, error detection and correction techniques
  - Receiver feedback: control msgs (ACK,NAK) from receiver to sender
  - retransmission

© J.F. Kurose, K.W. Ross. All Rights Reserved

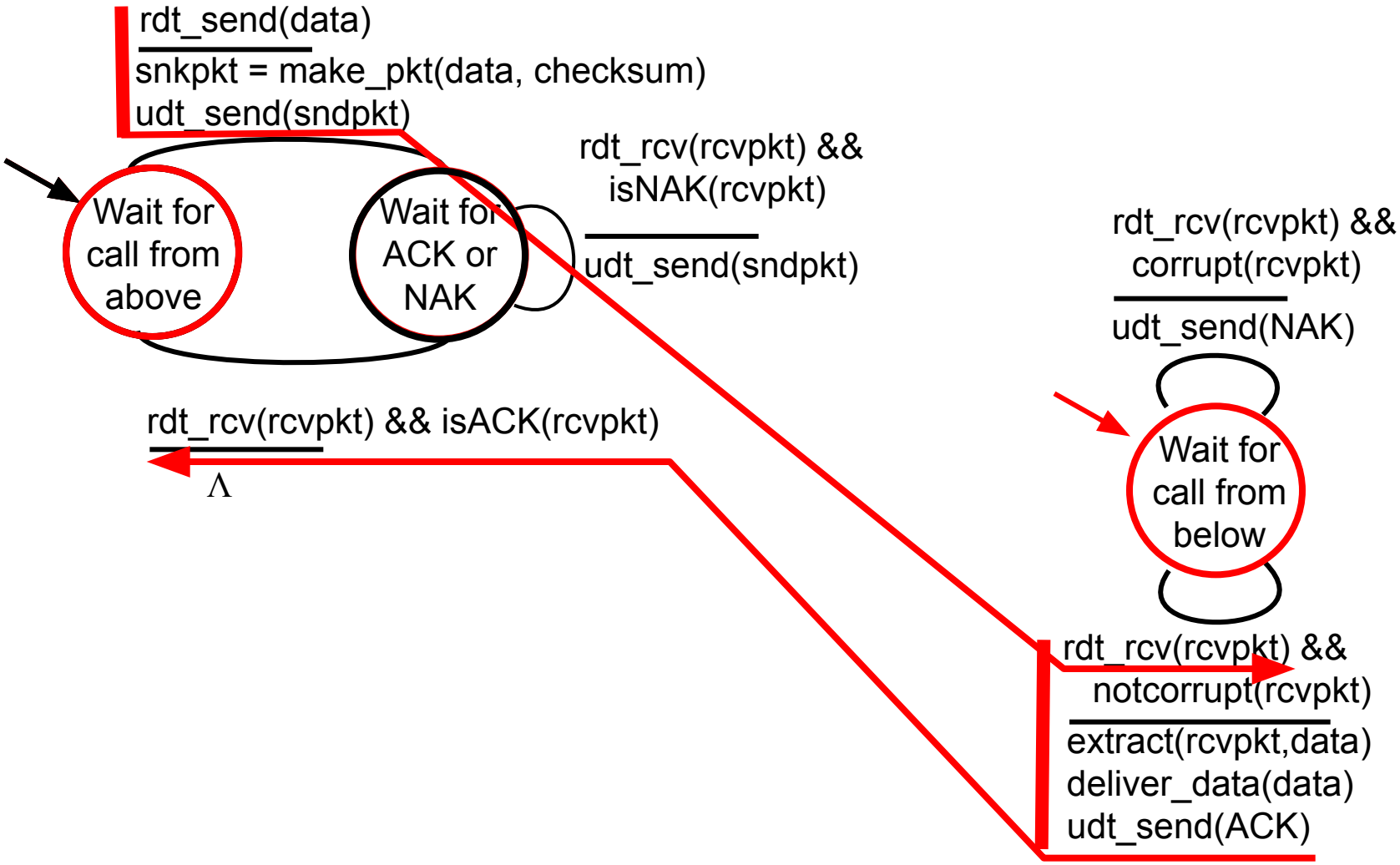
rdt2.0: FSM specification





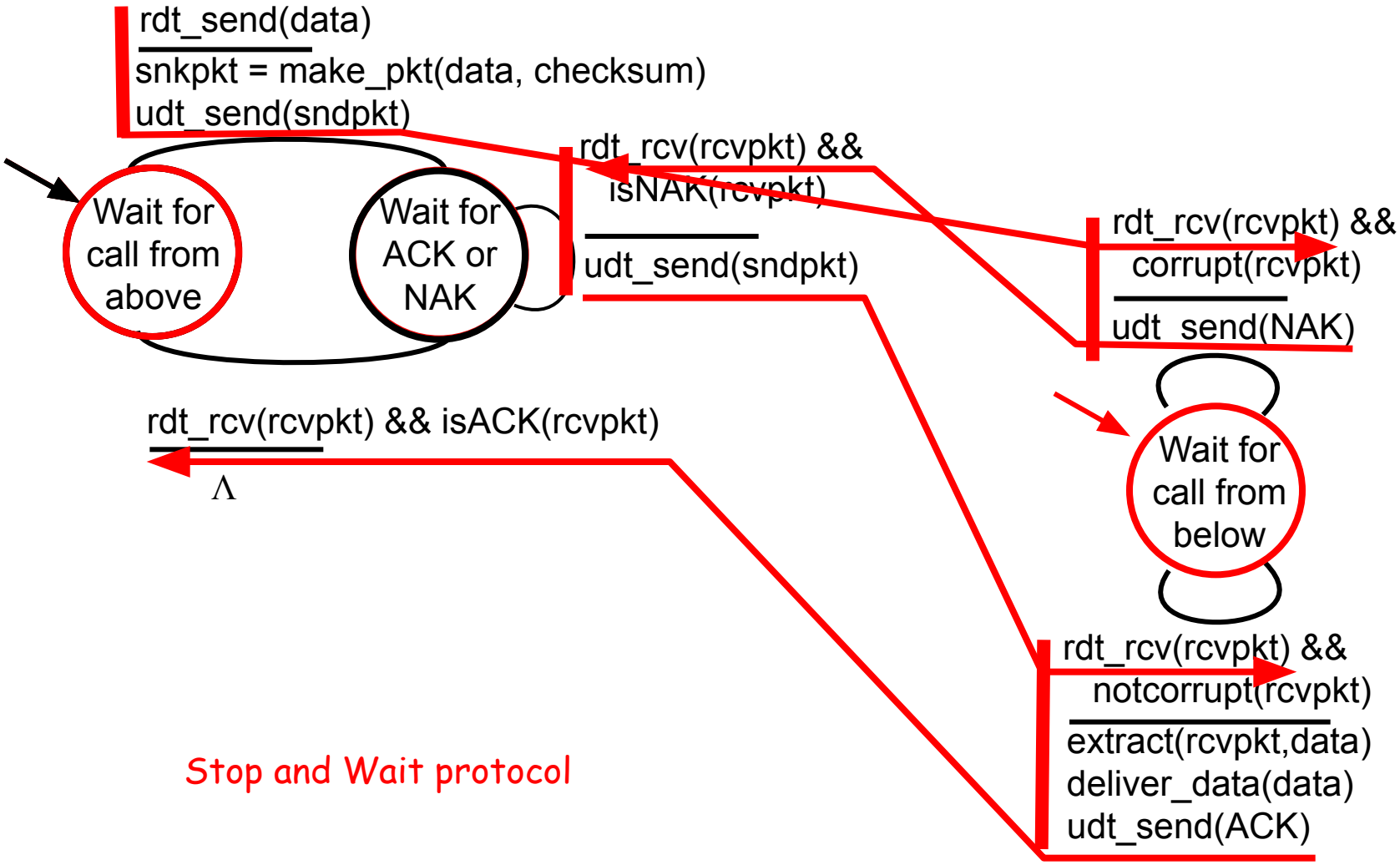
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.0: operation with no errors



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.0: error scenario



Stop and Wait protocol

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.0 has a fatal flaw!

what happens if ACK/NAK corrupted?

- sender doesn't know what happened at receiver!
- Three possibilities for handling corrupted ACKs or NAKs
  - Introduce a new type of sender-to-receiver packet
  - Recover from bit errors
  - Retransmission
- can't just retransmit: possible duplicate

handling duplicates:

- sender adds *sequence number* to each pkt
- sender retransmits current pkt if ACK/NAK corrupted
- receiver discards (doesn't deliver up) duplicate pkt

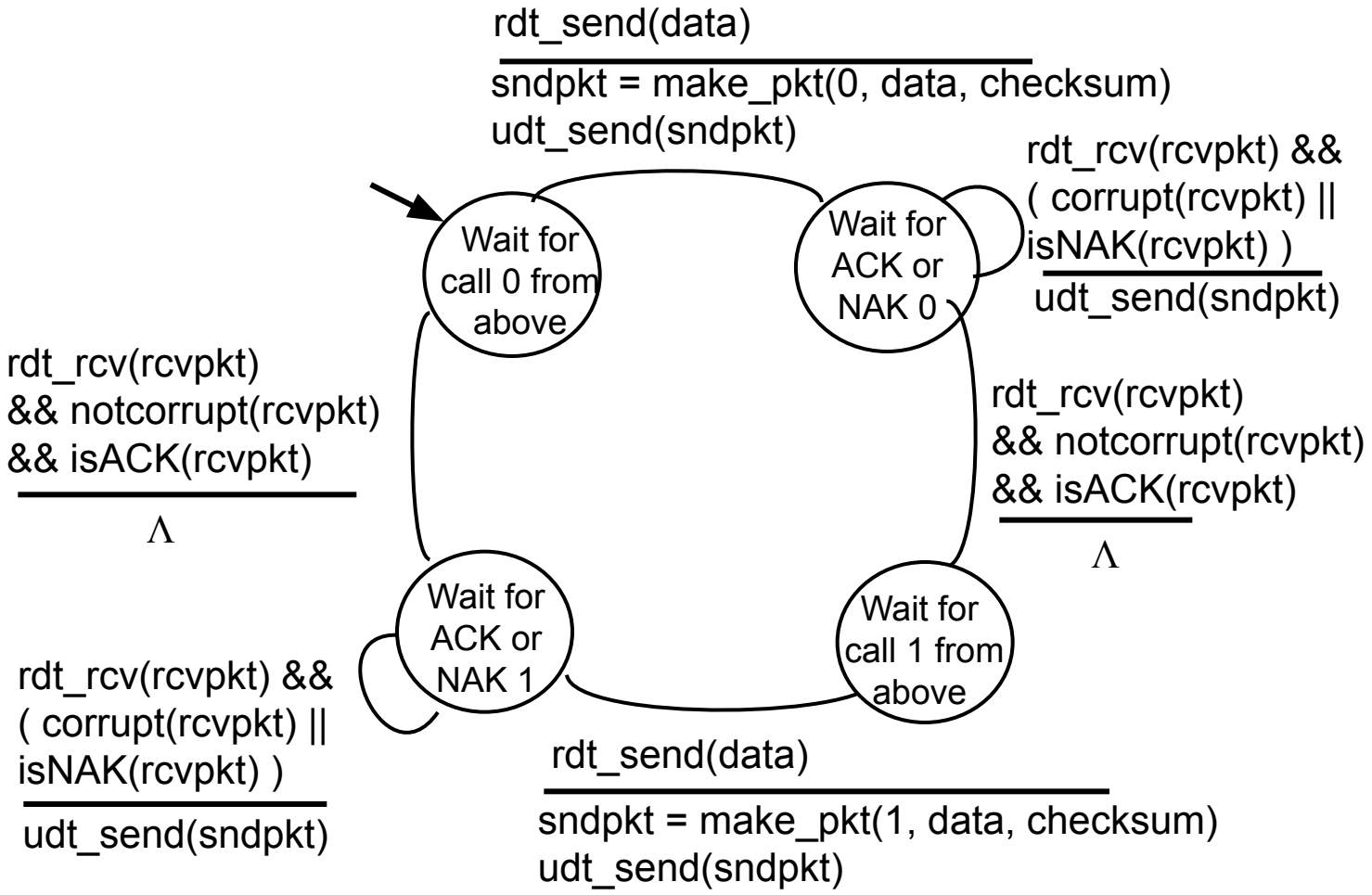
stop and wait

sender sends one packet, then waits for receiver

Response - *1-bit* sequence number, modulo-2

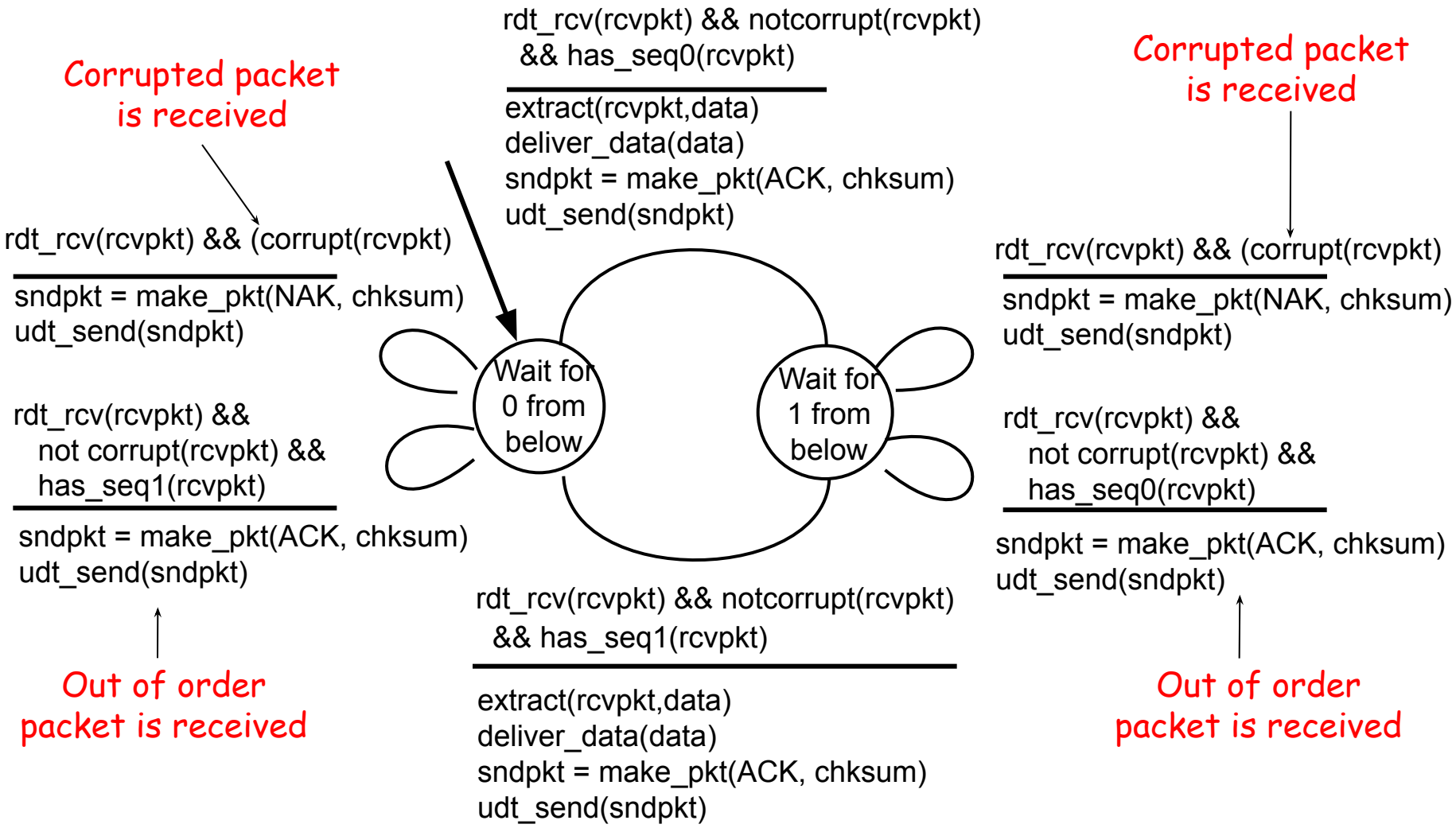
© J.F. Kurose, K.W. Ross. All Rights Reserved

rdt2.1: sender, handles garbled ACK/NAKs



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.1: receiver, handles garbled ACK/NAKs



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.1: discussion

sender:

- seq # added to pkt
- two seq. #'s (0,1) will suffice. Why?
- must check if received ACK/NAK corrupted
- twice as many states
  - state must "remember" whether "expected" pkt should have seq # of 0 or 1

receiver:

- must check if received packet is duplicate
  - state indicates whether 0 or 1 is expected pkt seq #
- note: receiver can *not* know if its last ACK/NAK received OK at sender

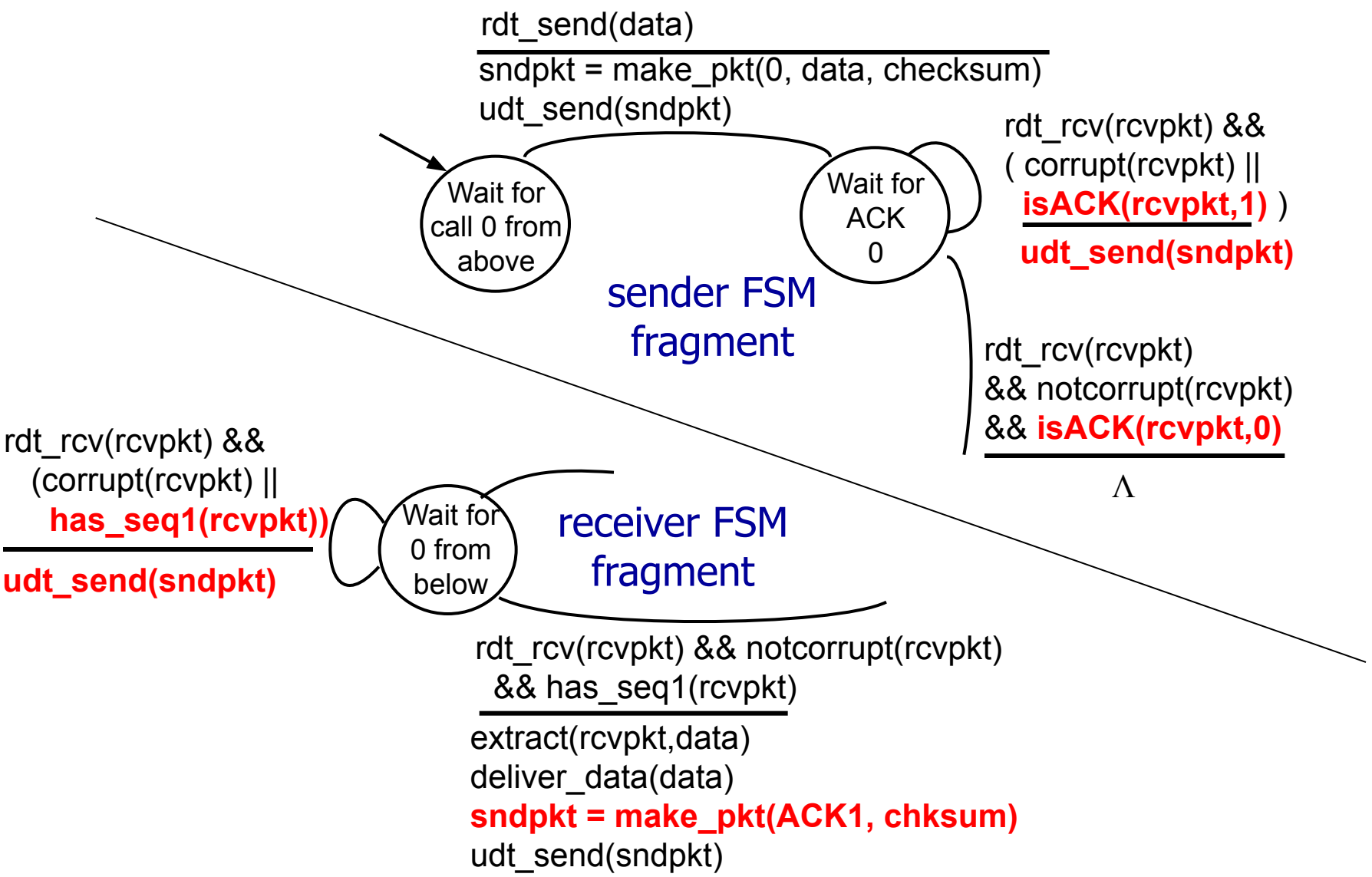
© J.F. Kurose, K.W. Ross. All Rights Reserved

rdt2.2: a NAK-free protocol

- same functionality as rdt2.1, using ACKs only
- instead of NAK, receiver sends ACK for last pkt received OK
  - receiver must *explicitly* include seq # of pkt being ACKed
- duplicate ACK at sender results in same action as NAK: *retransmit current pkt*

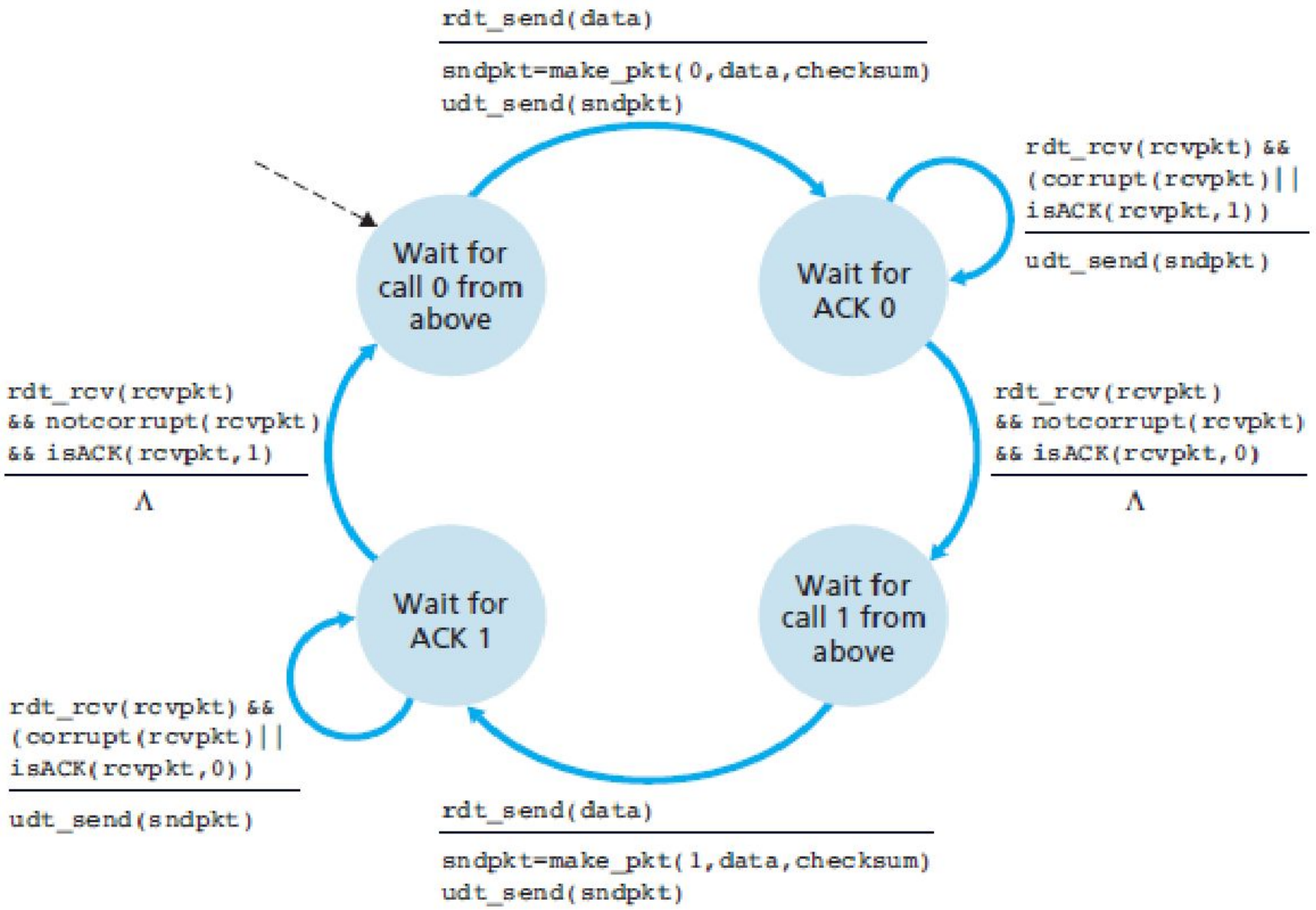
© J.F. Kurose, K.W. Ross. All Rights Reserved

rdt2.2: sender, receiver fragments



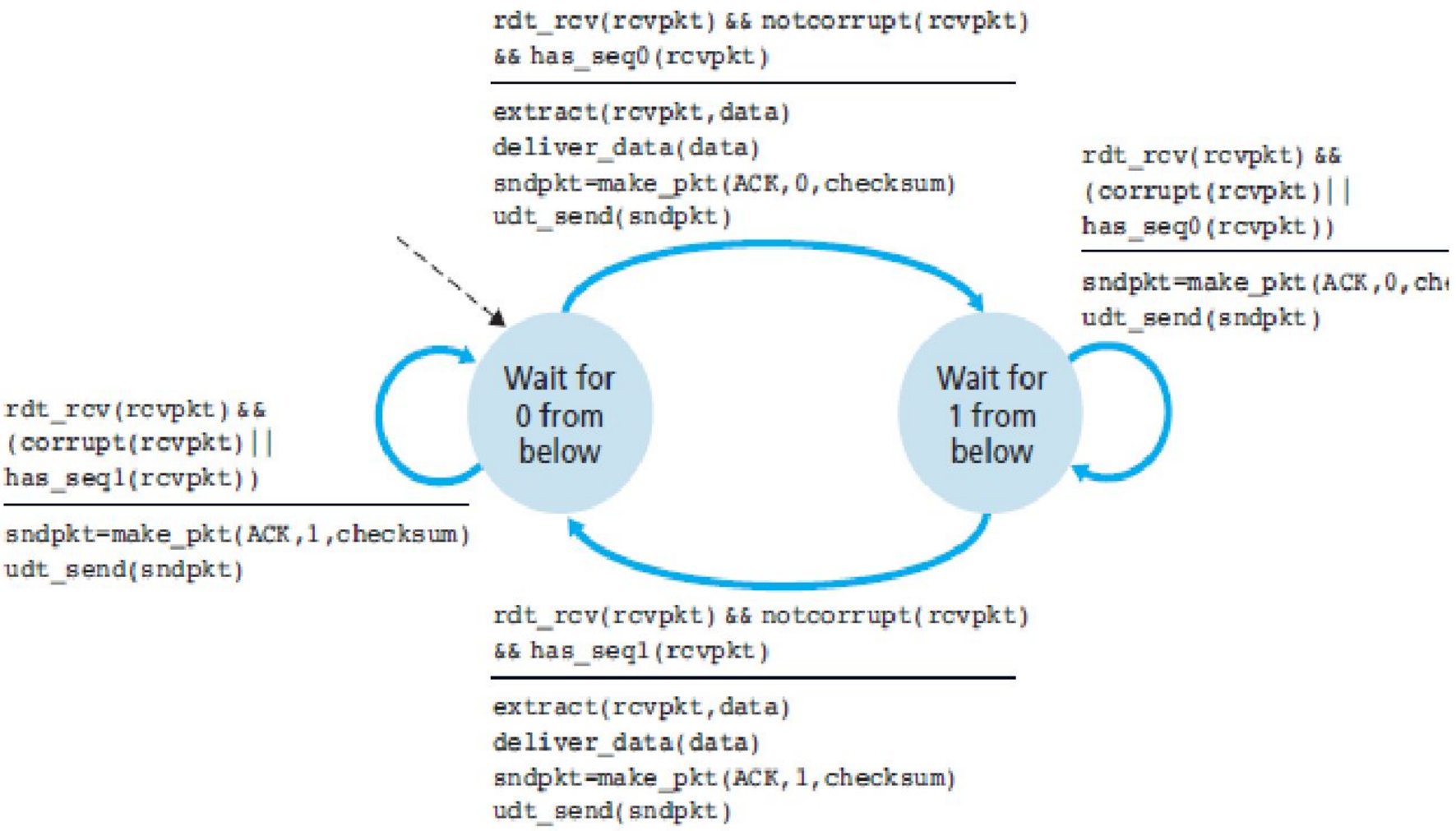
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.2: sender



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt2.2: Receiver



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt3.0: channels with errors *and* loss

new assumption:

underlying channel can also lose packets (data, ACKs)

- Two additional concerns
  - How to detect packet loss?
  - What to do when packet loss occurs?

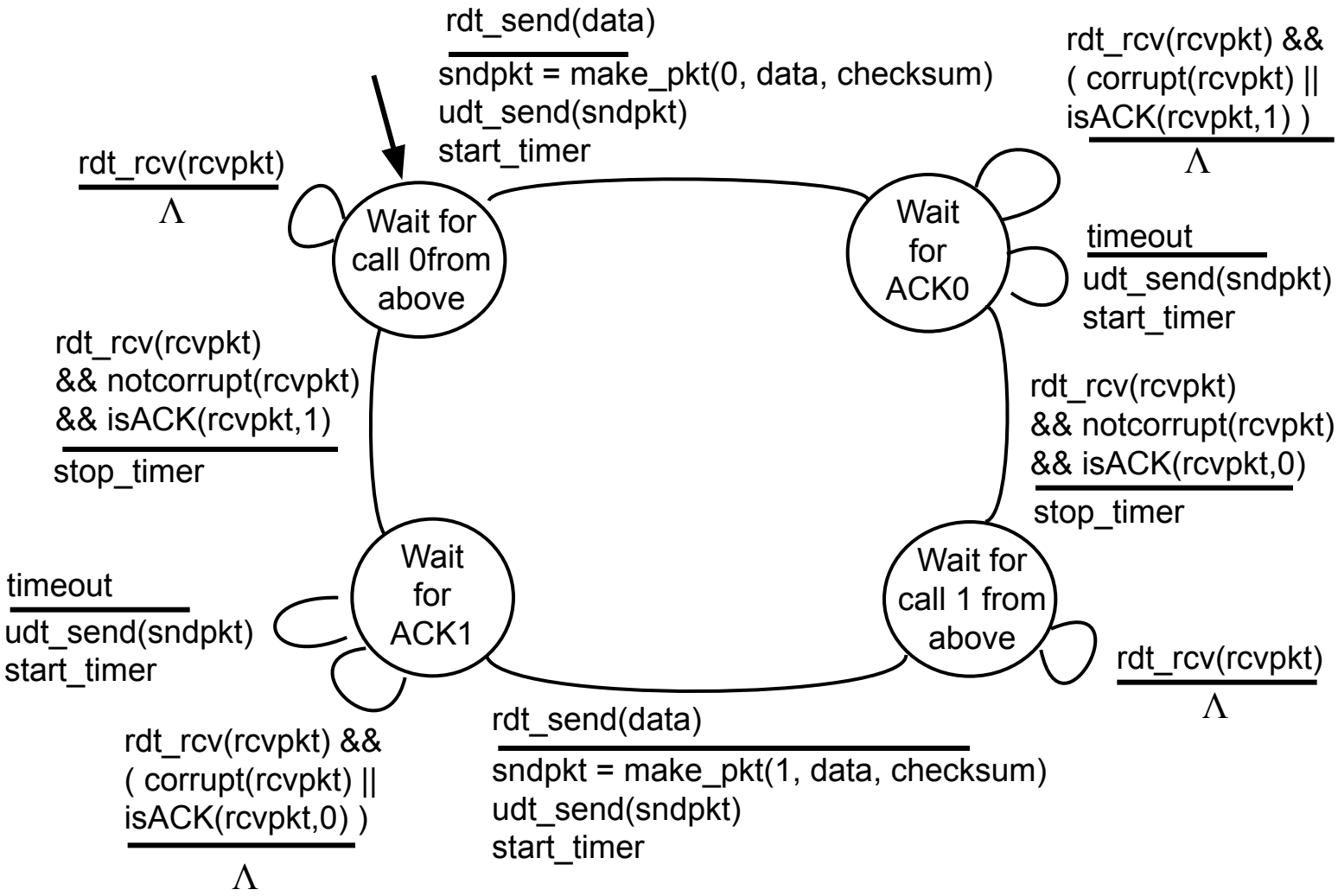
    - checksum, seq. #, ACKs, retransmissions will be of help … but not enough

approach: sender waits "reasonable" amount of time for ACK - RTD

- retransmits if no ACK received in this time
- if pkt (or ACK) just delayed (not lost):
  - retransmission will be duplicate, but seq. #'s already handles this
  - receiver must specify seq # of pkt being ACKed
- requires countdown timer

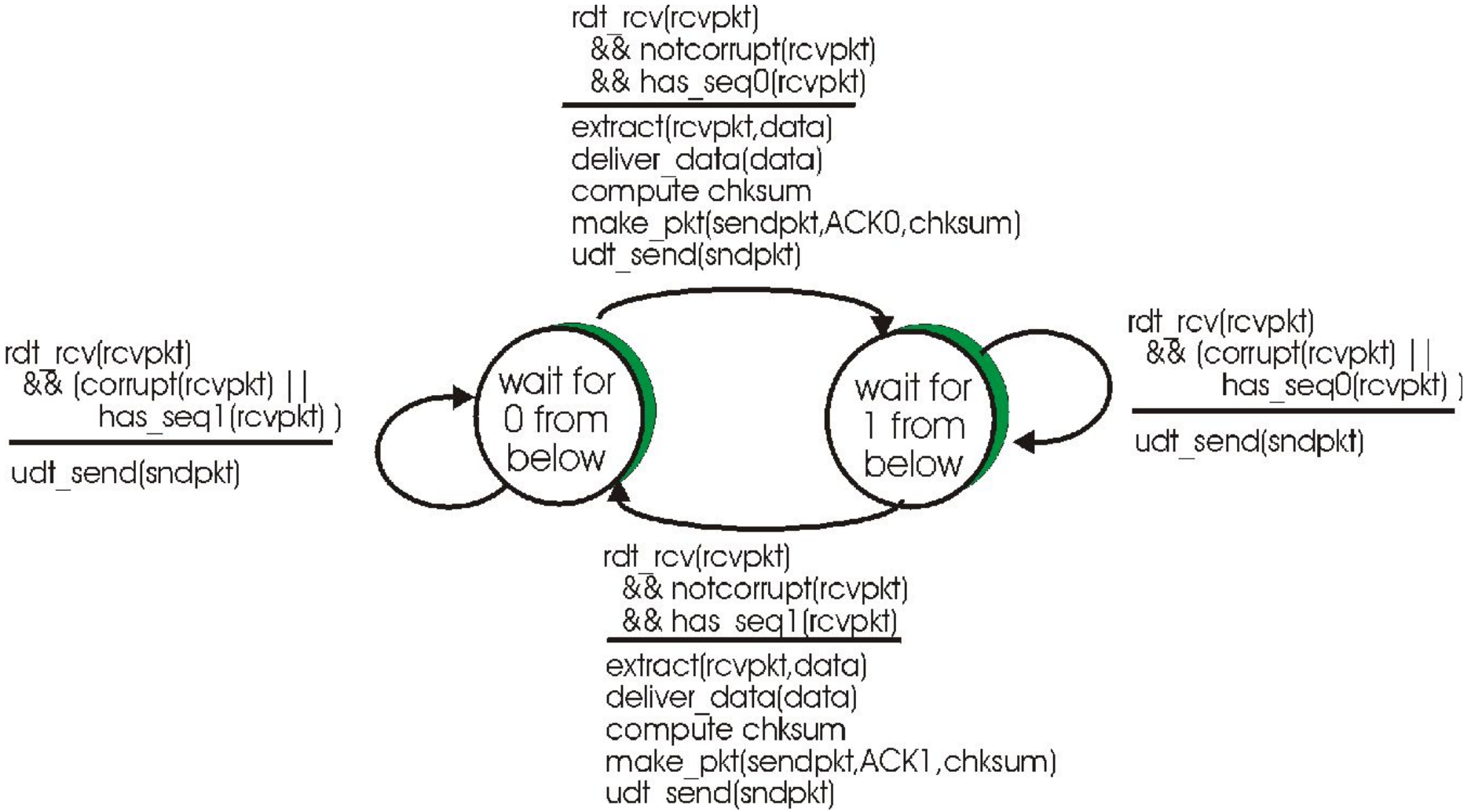
© J.F. Kurose, K.W. Ross. All Rights Reserved

rdt3.0 sender



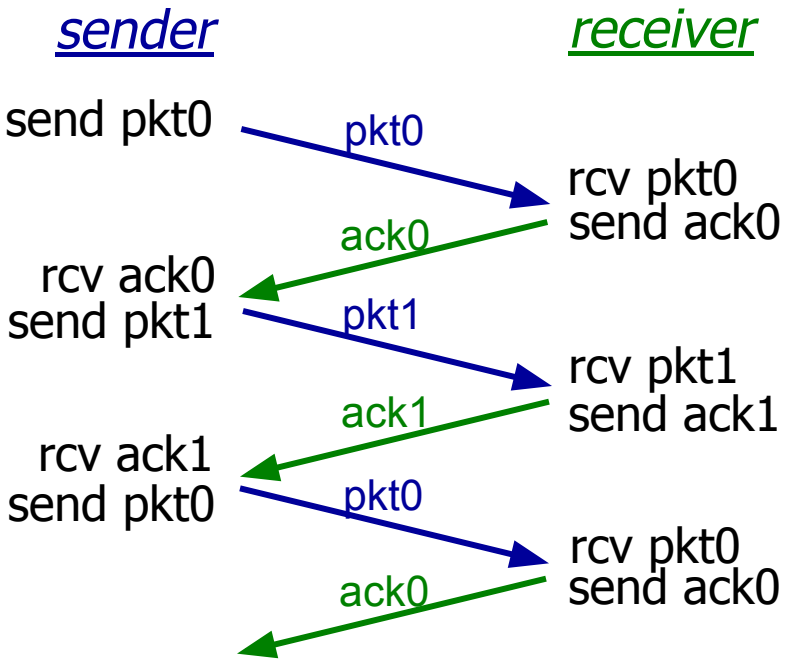
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt3.0 Receiver

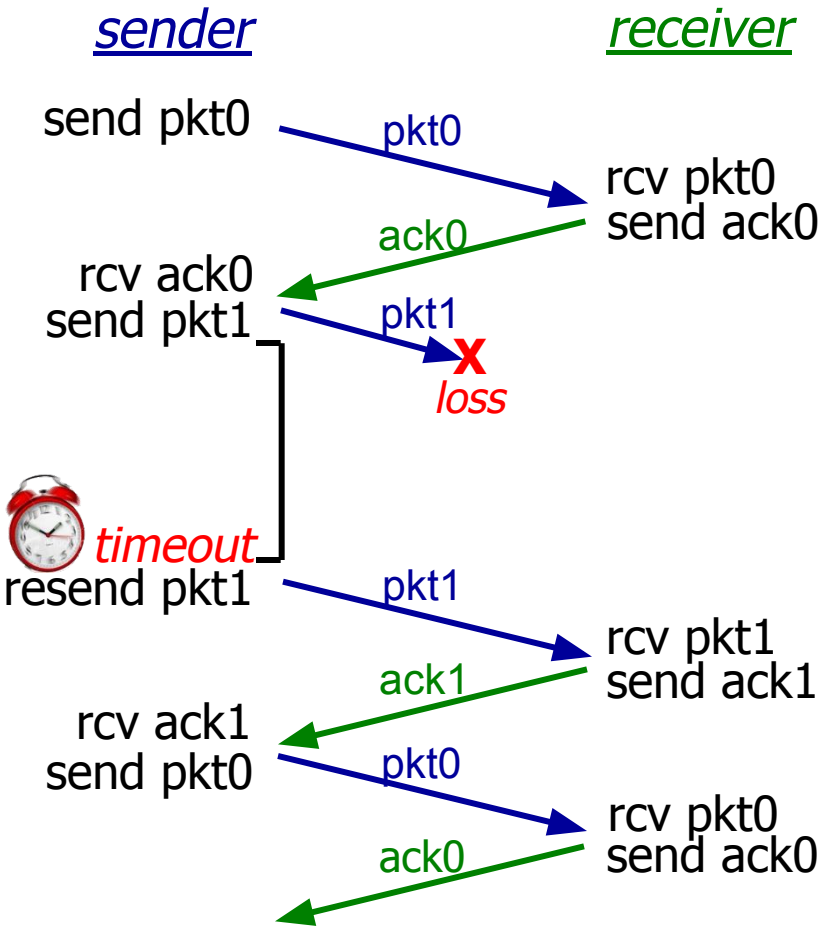


© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt3.0 in action



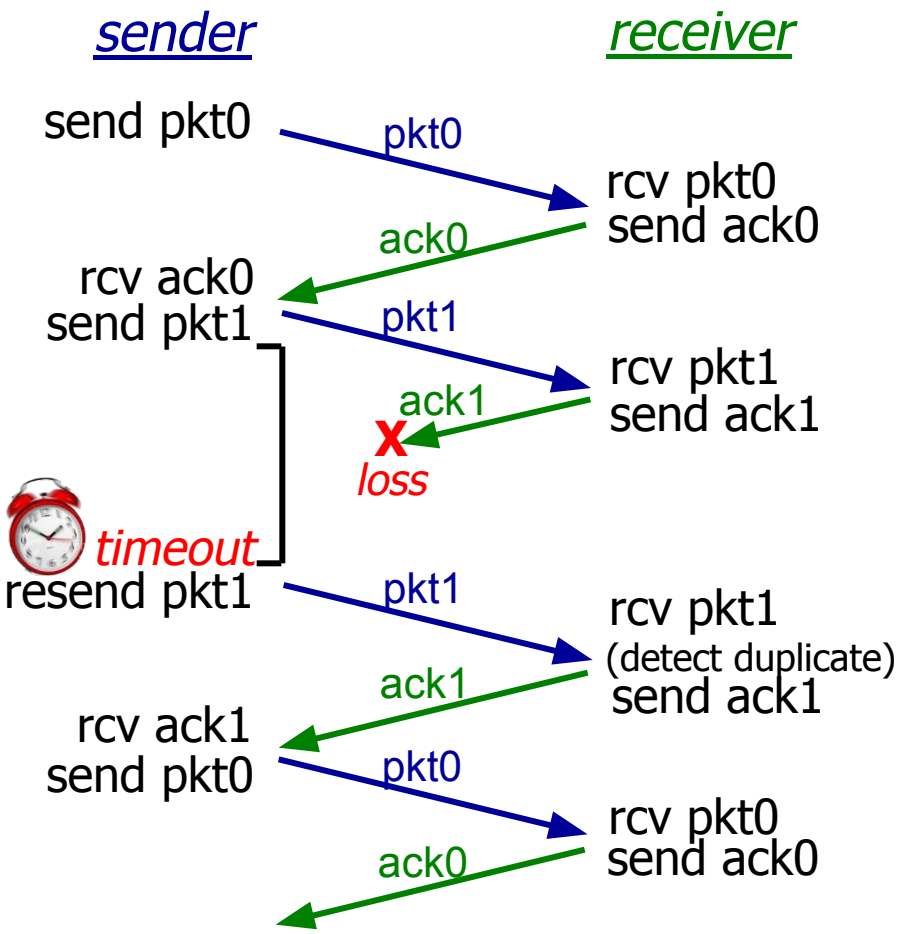
(a) no loss



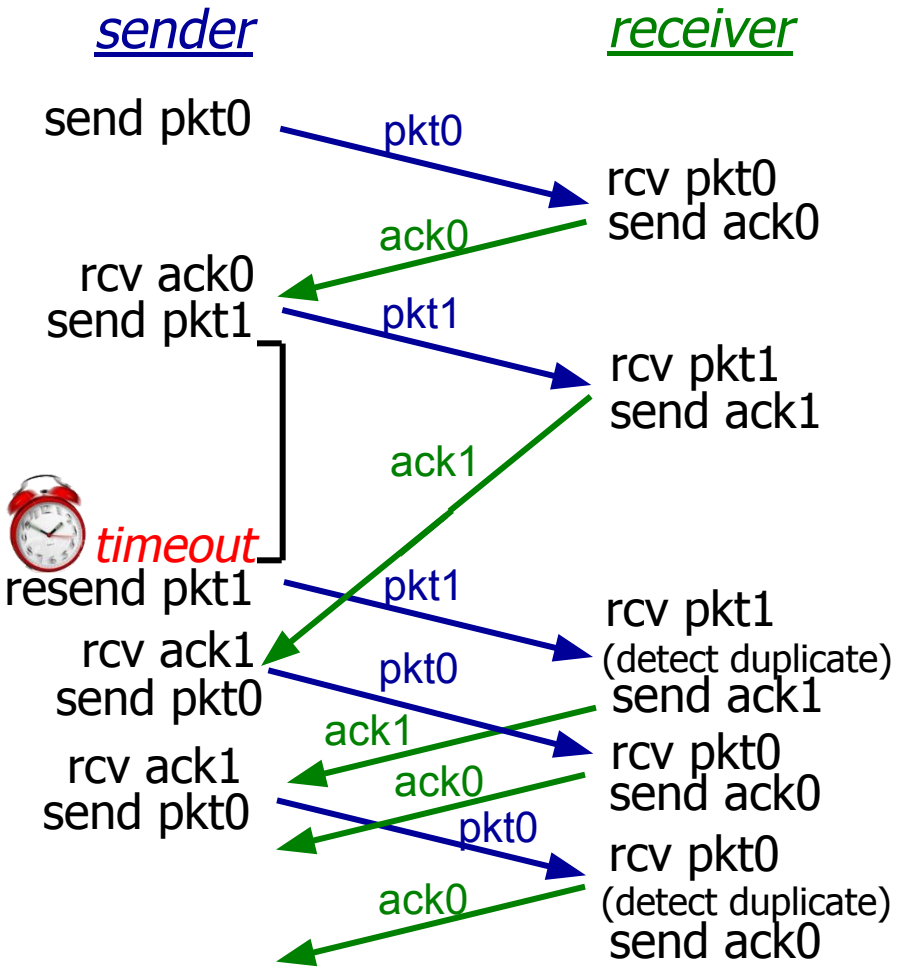
(b) packet loss

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt3.0 in action



(c) ACK loss

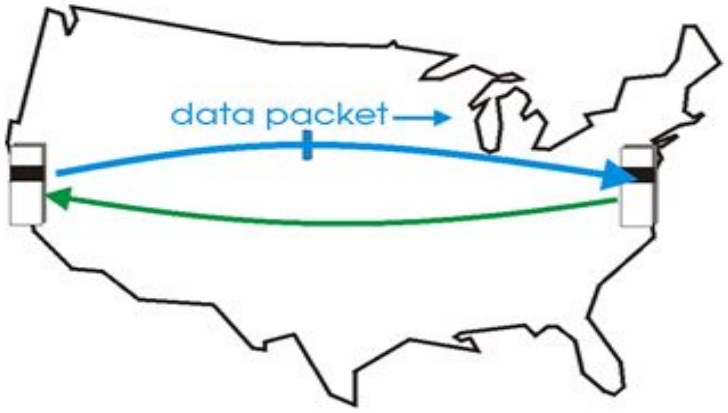


(d) premature timeout/ delayed ACK

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Performance of rdt3.0

- rdt3.0 works, but performance stinks for high speed networks, stop-and-wait protocol
- example: R=1 Gbps link, 15 ms e-e prop. delay, RTT 30ms, L=1KByte=1000 bytes=8000 bits per packet (including header and data):

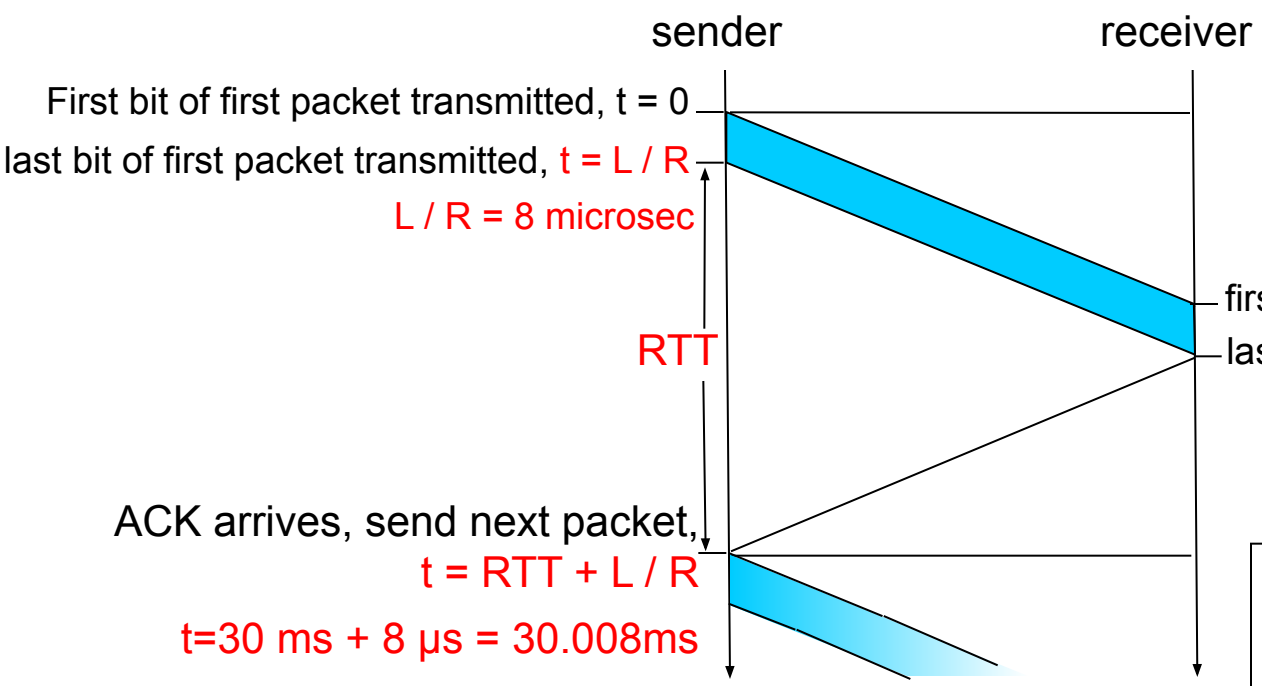


(a) a stop-and-wait protocol in operation

$$d_{trans} = \frac{\text{L (packet length in bits)}}{\text{R (transmission rate, bps)}} = \frac{8000\text{bits}}{10^9\ \text{bits/sec}} = 8\ \text{microsec}$$

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

rdt3.0: stop-and-wait operation



Assume: ACK packets are extremely small
Ignore their Transmission Time
Receiver send ACK as soon as last bit of packet is received

Ignoring: lower layer protocol processing times at sender and receiver, Processing and Queuing delays at routers

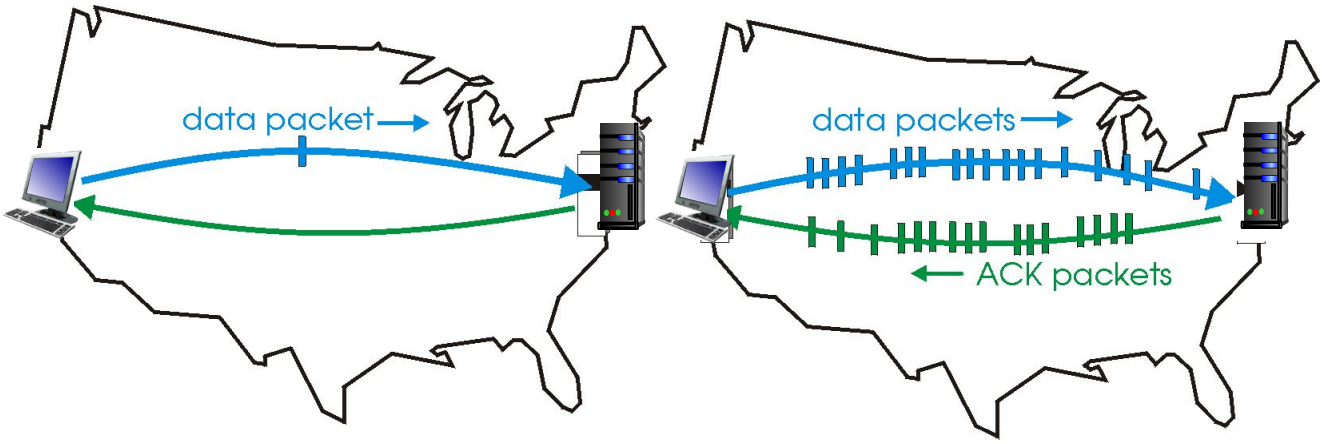
$$U_{sender} = \frac{L / R}{RTT + L / R} = \frac{.008}{30.008} = 0.00027$$

- U_{sender}: utilization of sender or channel – fraction of time sender busy sending bits into the channel
- 1000 bytes every 30.008 msec -> 267 kbps throughput over 1 Gbps link
- Network protocol limits use of physical resources!

© J.F. Kurose, K.W. Ross. All Rights Reserved

Pipelined protocols

pipelining: sender allows multiple, "in-flight", yet-to-be-acknowledged pkts

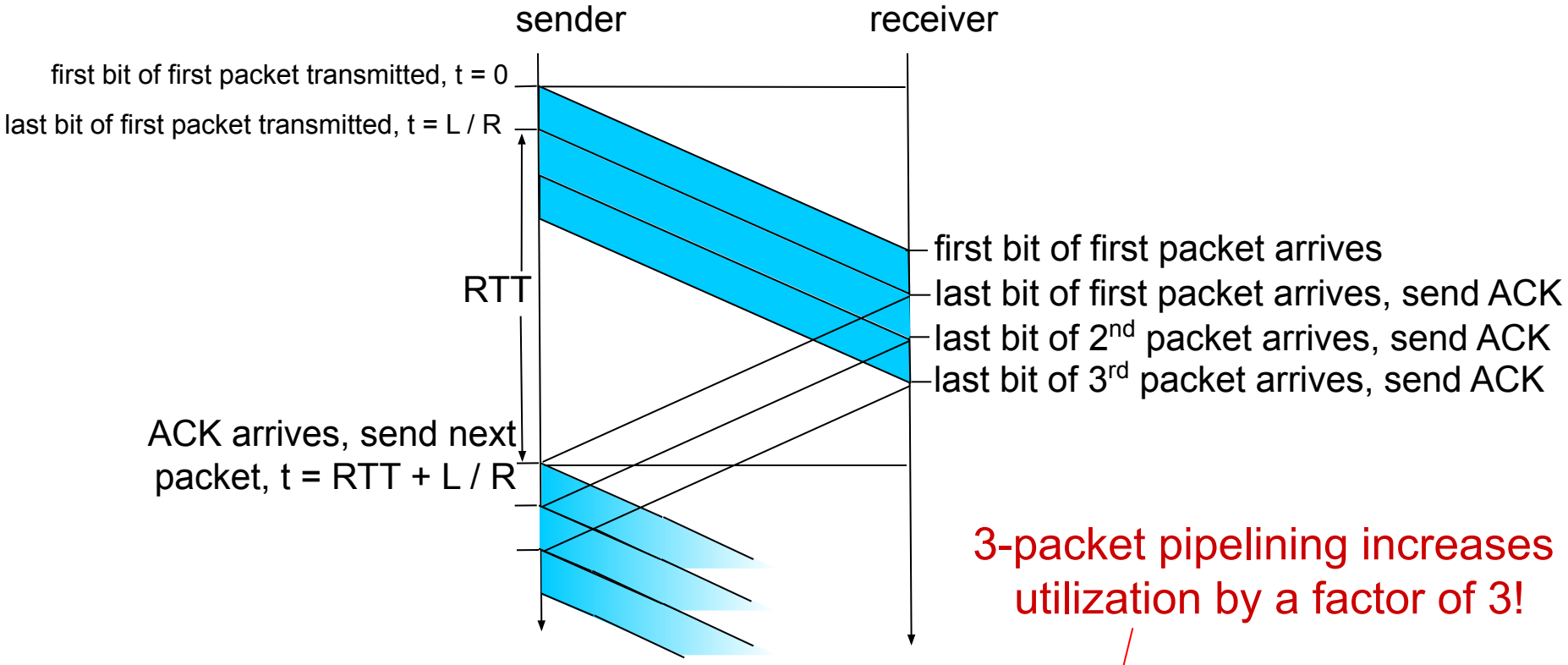


(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Pipelining: increased utilization



3-packet pipelining increases utilization by a factor of 3!

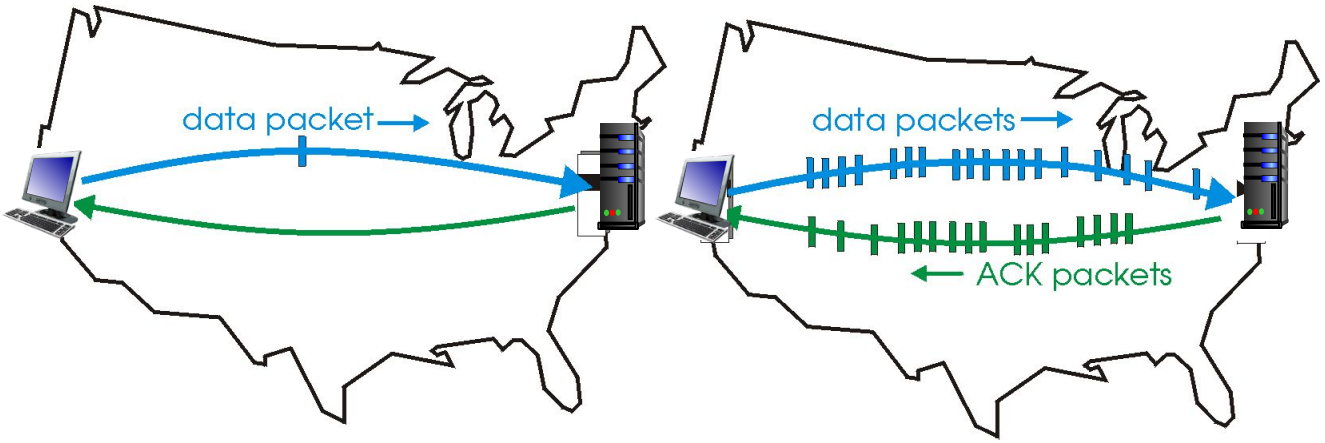
$$U_{sender} = \frac{3 * L / R}{RTT + L / R} = \frac{.024}{30.008} = 0.0008$$

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Pipelined protocols

pipelining: consequences

- range of sequence numbers must be increased
- buffering at sender and/or receiver



(a) a stop-and-wait protocol in operation

(b) a pipelined protocol in operation

- two generic forms of pipelined protocols: *go-Back-N, selective repeat*

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Pipelined protocols: overview

Go-back-N:

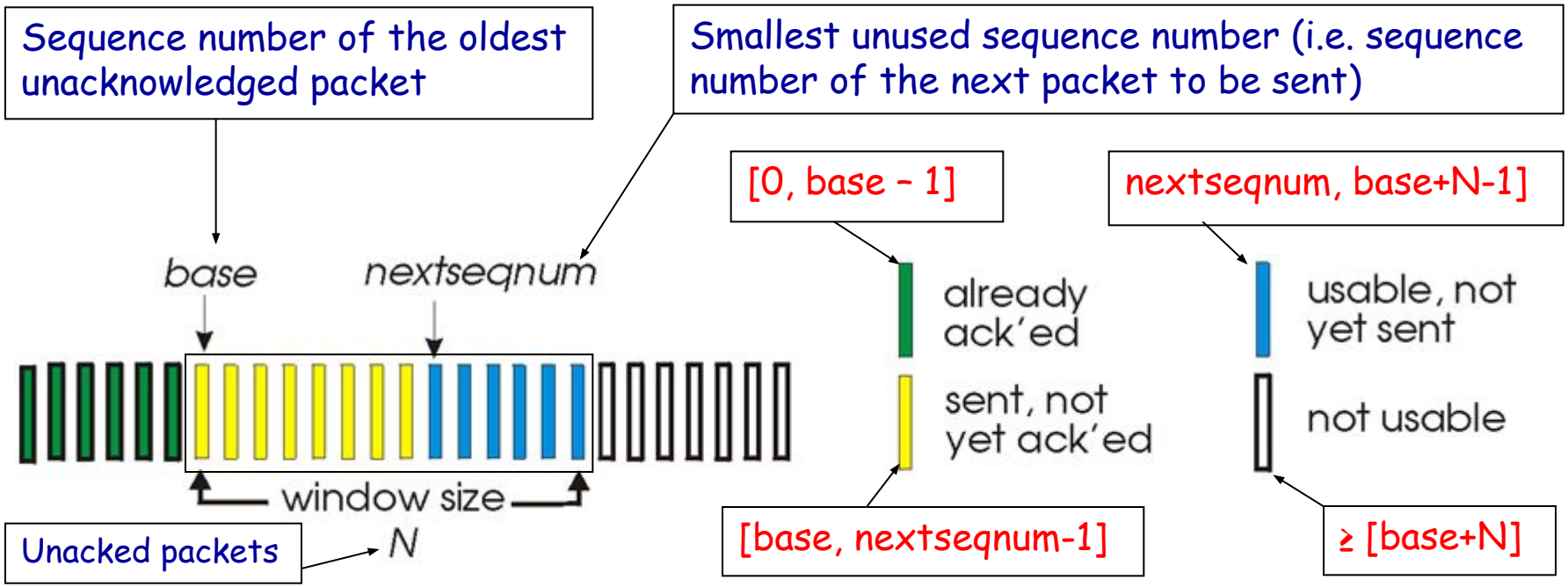
- sender can have up to N unacked packets in pipeline
- receiver only sends *cumulative ack*
  - doesn't ack packet if there's a gap
- sender has timer for oldest unacked packet
  - when timer expires, retransmit *all* unacked packets

Selective Repeat:

- sender can have up to N unack'ed packets in pipeline
- rcvr sends *individual ack* for each packet
- sender maintains timer for each unacked packet
  - when timer expires, retransmit only that unacked packet

© J.F. Kurose, K.W. Ross. All Rights Reserved

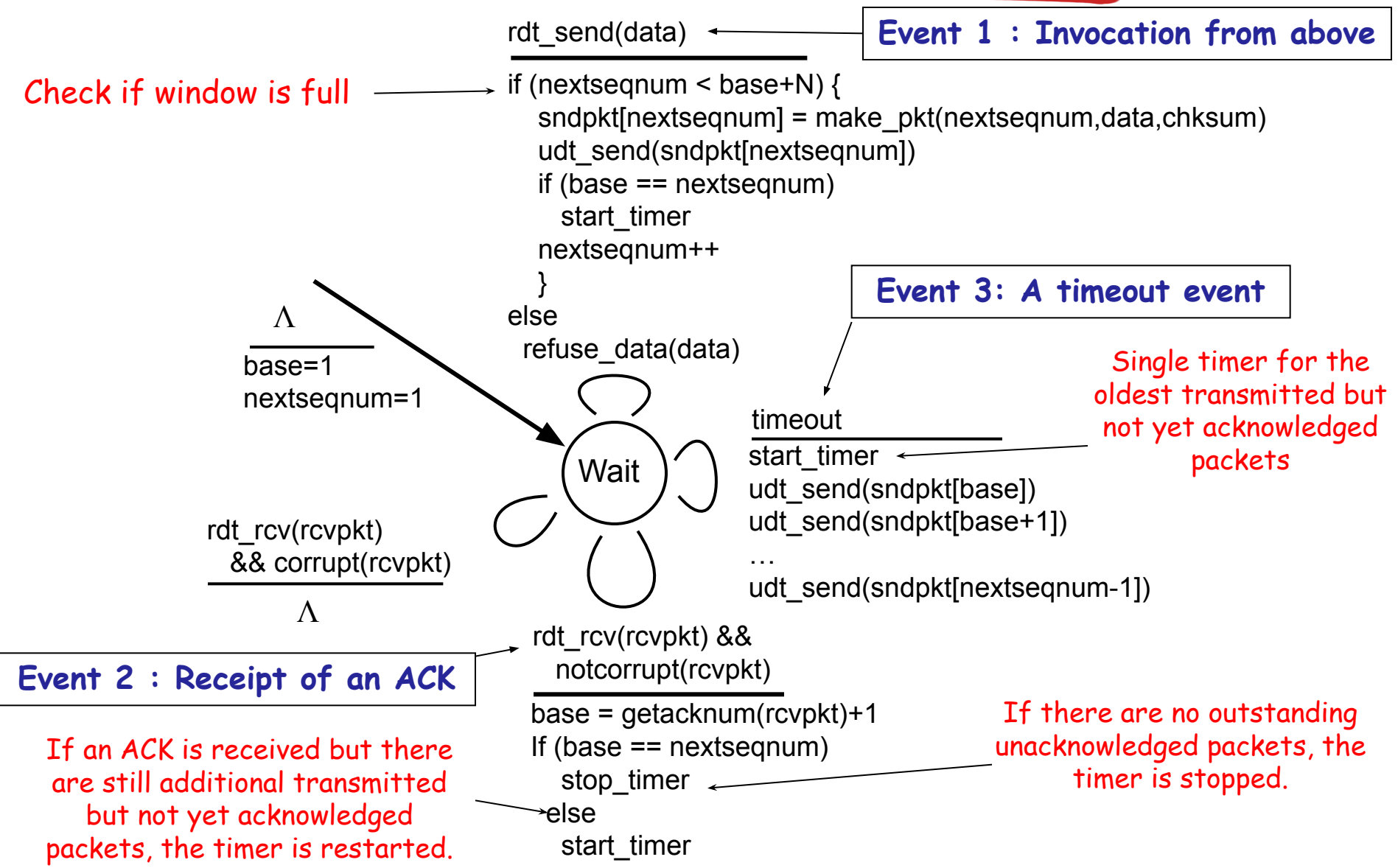
Go-Back-N: sender



- k-bit seq # in pkt header, $[0, 2^k-1]$, modulo 2^k arithmetic
- "window" of up to N, consecutive unack'ed pkts allowed
  - *Sliding window protocol*
- ACK(n): ACKs all pkts up to, including seq # n - *"cumulative ACK"*
  - may receive duplicate ACKs (see receiver)
- timer for oldest in-flight pkt
- *timeout(n):* retransmit packet n and all higher seq # pkts in window

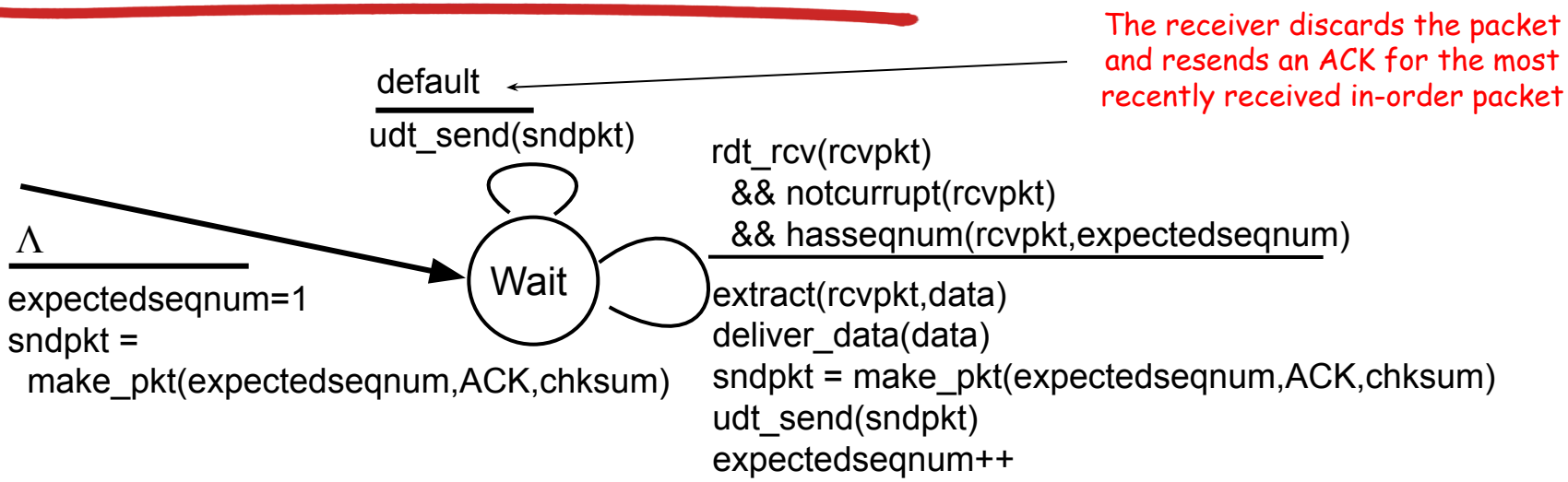
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

GBN: sender extended FSM (ACK based, NAK free)



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

GBN: receiver extended FSM



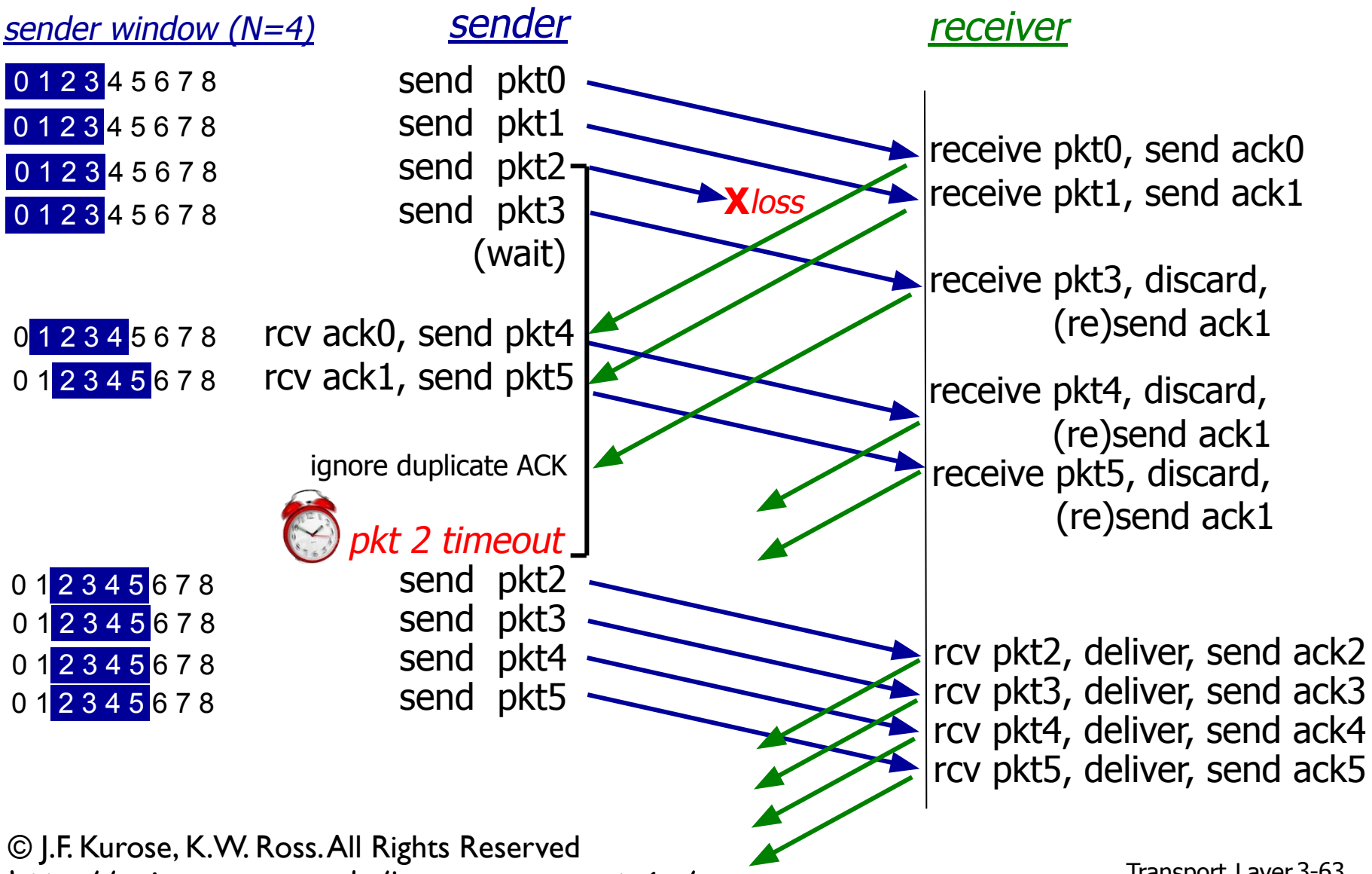
ACK-only: always send ACK for correctly-received pkt with highest *in-order* seq #

- may generate duplicate ACKs
- need only remember **`expectedseqnum`**

- out-of-order pkt:
  - discard (don't buffer) -> no receiver buffering!
  - Re-ACK pkt with highest in-order seq #

© J.F. Kurose, K.W. Ross. All Rights Reserved

GBN in action



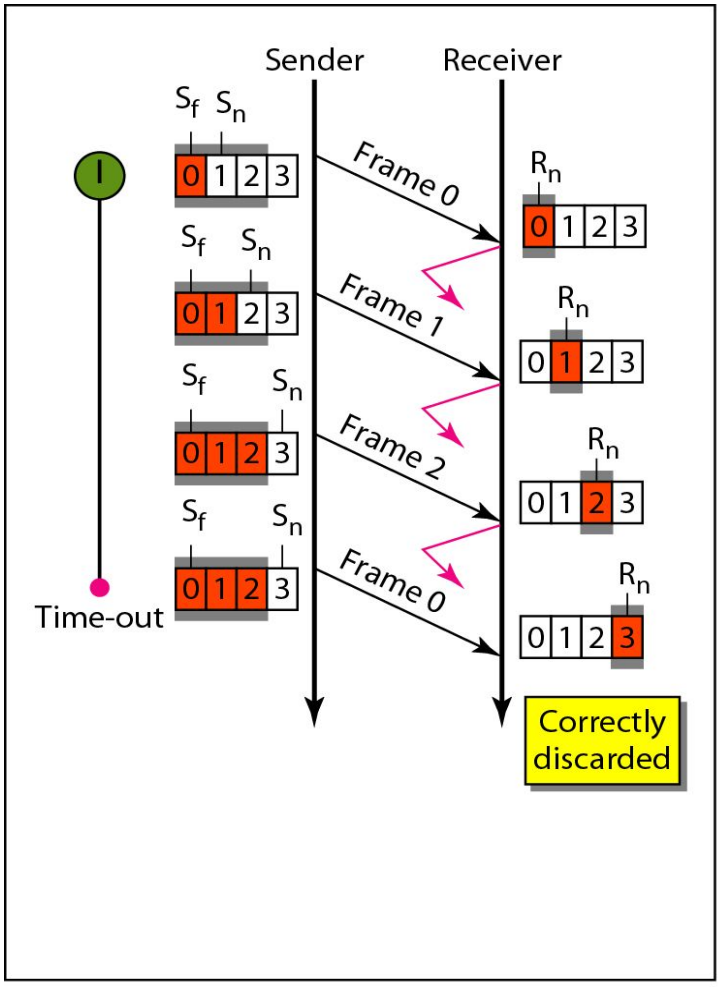
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

GBN: Performance Problems

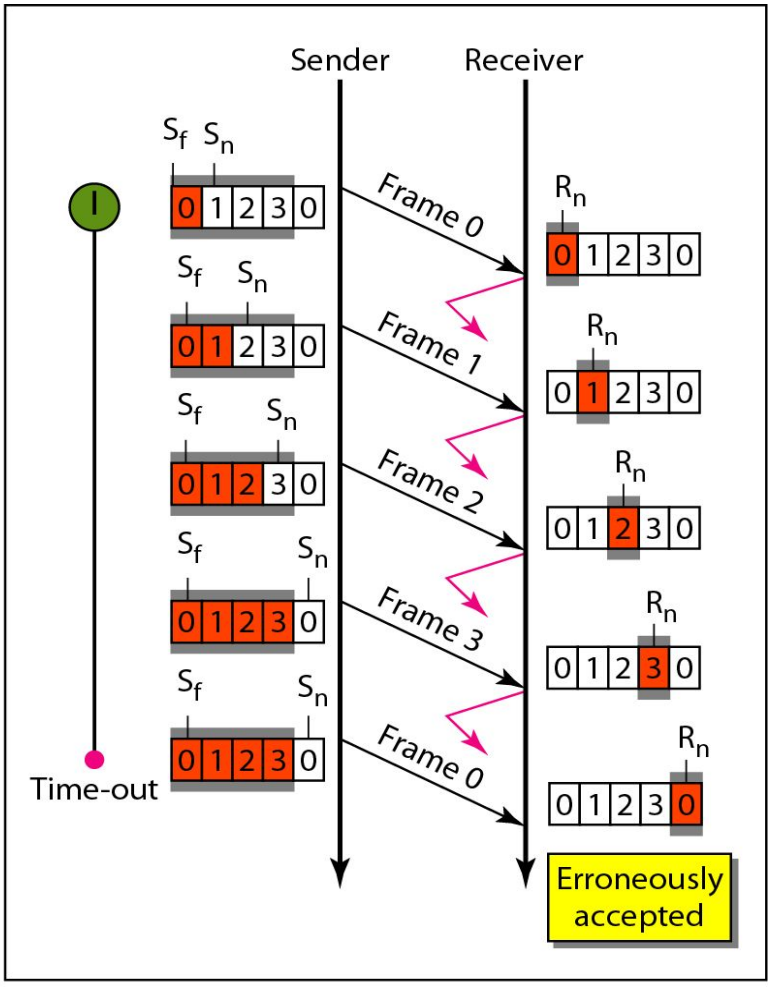
- When the windows size and bandwidth-delay product are both large
  - Many packets can be in pipeline
- *A single error can cause GBN to retransmit a large number of packets.*

© J.F. Kurose, K.W. Ross. All Rights Reserved

Window Size for GBN



a. Window size < 2^m



b. Window size = 2^m

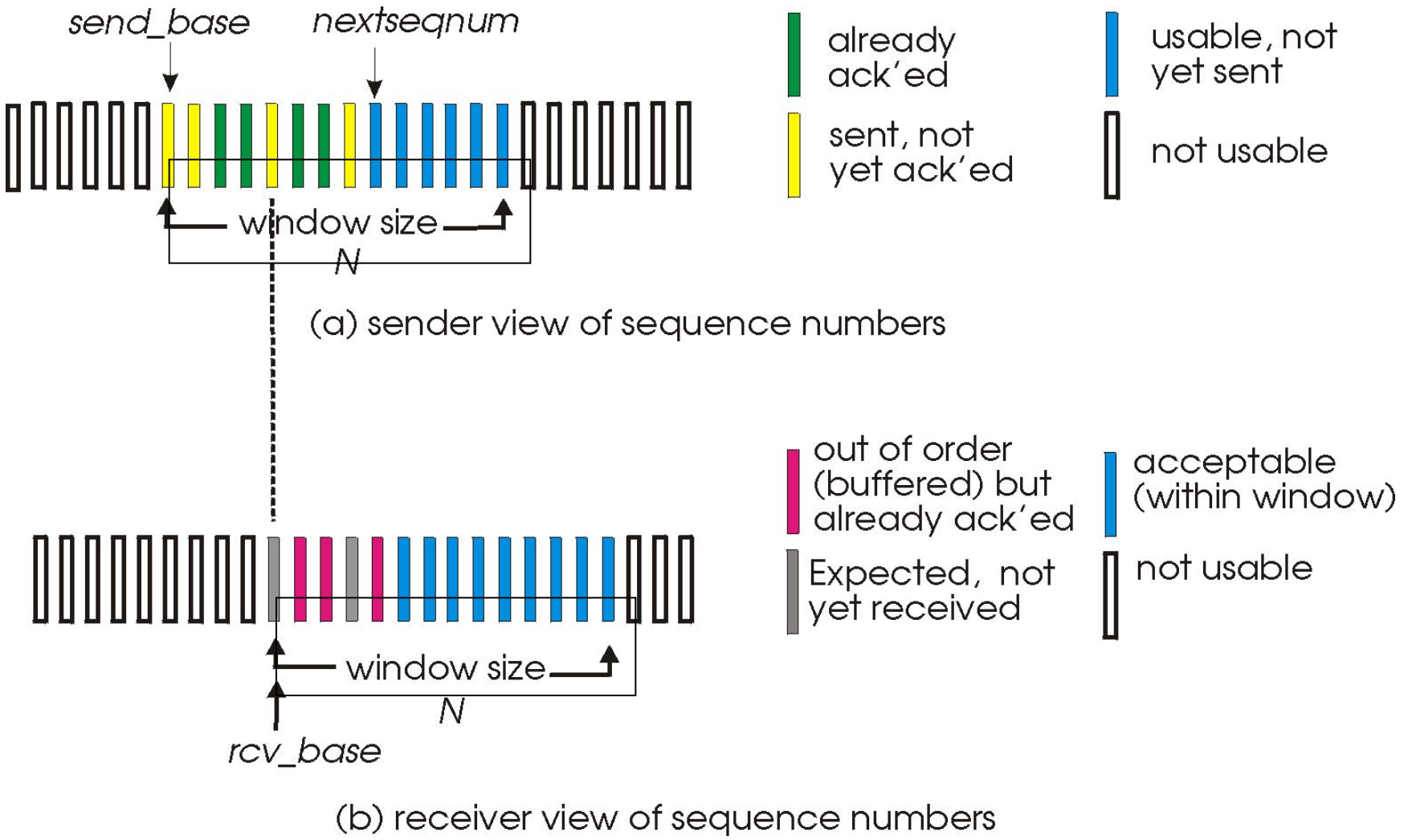
© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

Selective repeat

- receiver *individually* acknowledges all correctly received pkts
  - buffers pkts, as needed, for eventual in-order delivery to upper layer
- sender only resends pkts for which ACK not received
  - sender timer for each unACKed pkt
- sender window
  - *N* consecutive seq #'s
  - limits seq #s of sent, unACKed pkts

© J.F. Kurose, K.W. Ross. All Rights Reserved

Selective repeat: sender, receiver windows



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Selective repeat

sender

data received from above:

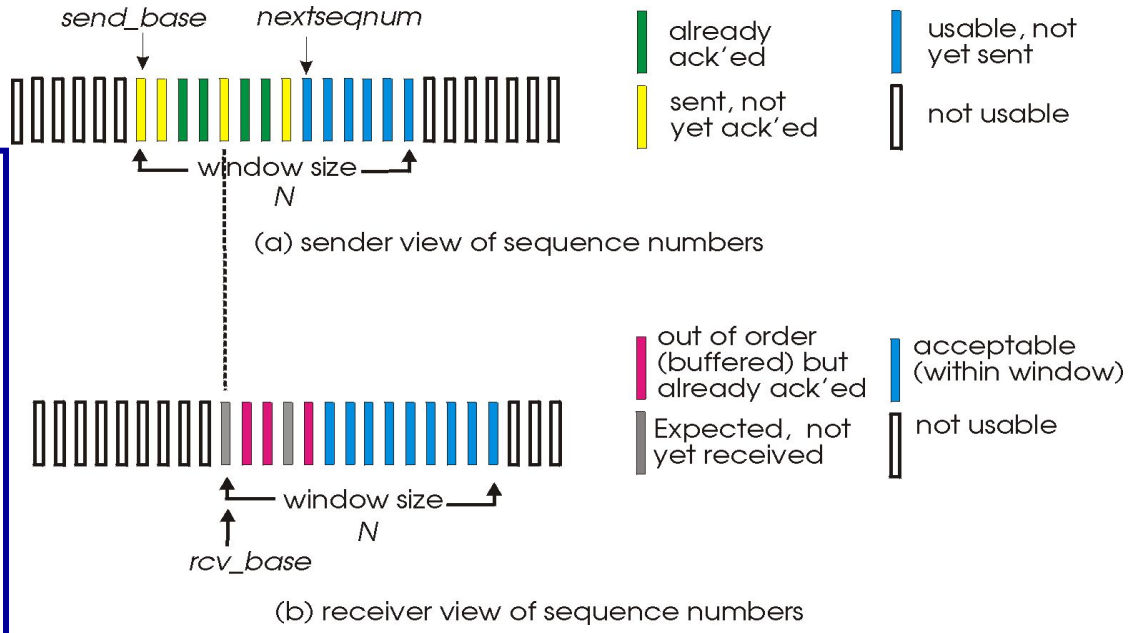
- if next available seq # in window, send pkt

timeout(n):

- Each packet must have its own logical timer
- Single H/W timer
- resend pkt n, restart timer

ACK(n) in [sendbase, sendbase+N-1]:

- mark pkt n as received
- if n smallest unACKed pkt, advance window base to next unACKed seq #



receiver

pkt n in [rcvbase, rcvbase+N-1]

- send ACK(n)
- out-of-order: buffer
- in-order: deliver (also deliver buffered, in-order pkts), advance window to next not-yet-received pkt

pkt n in [rcvbase-N,rcvbase-1]

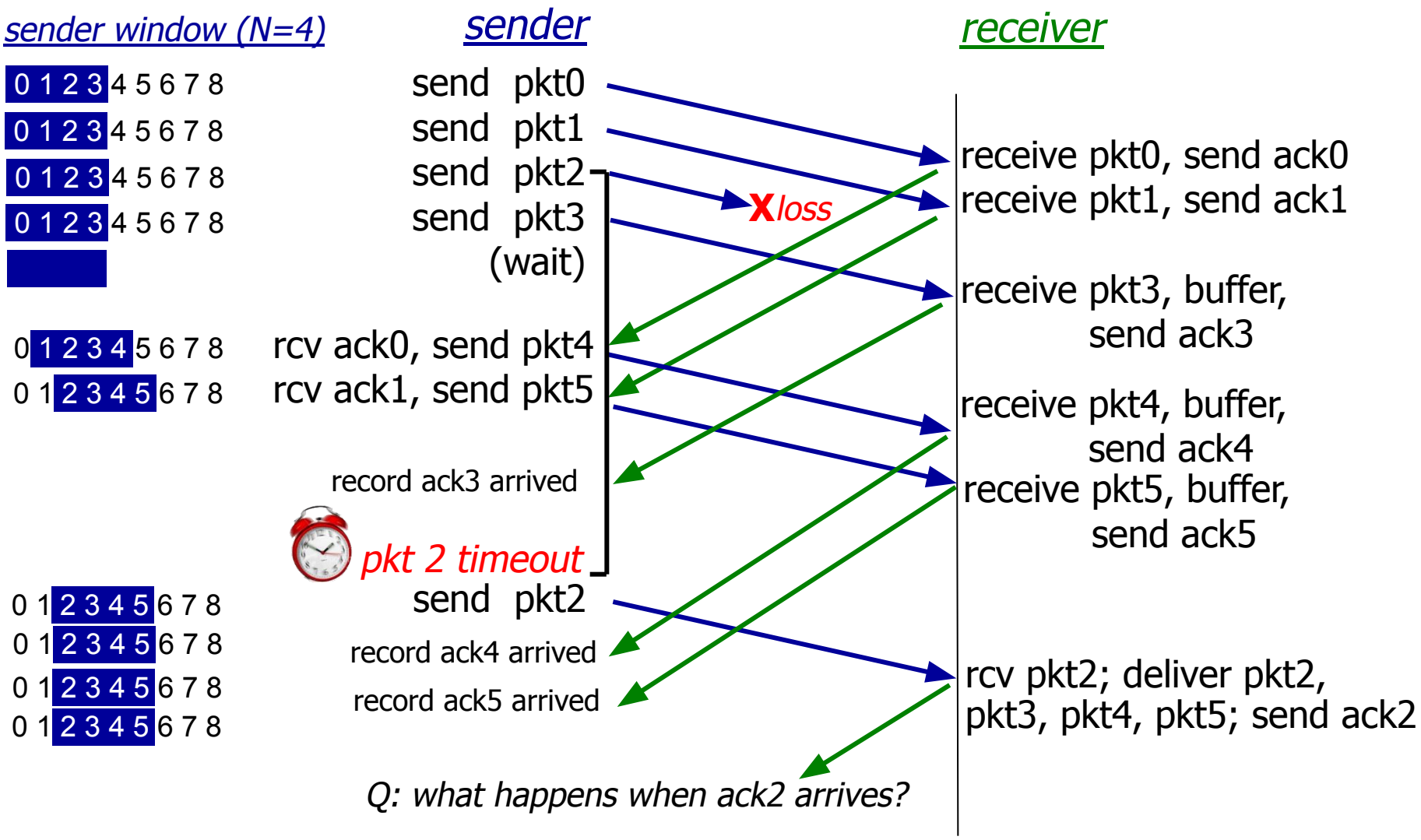
- ACK(n)

otherwise:

- ignore

© J.F. Kurose, K.W. Ross. All Rights Reserved

Selective repeat in action



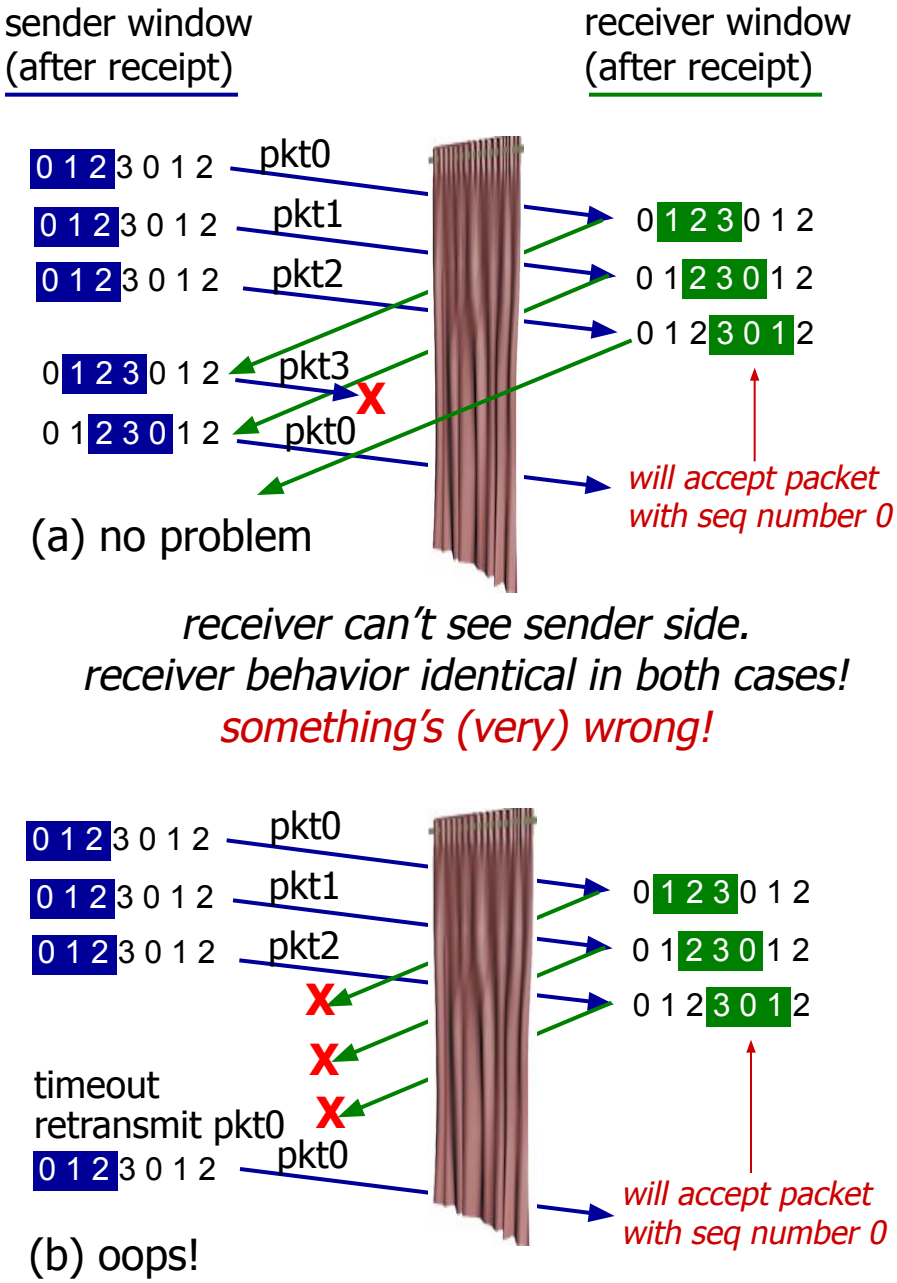
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Selective repeat: dilemma

example:

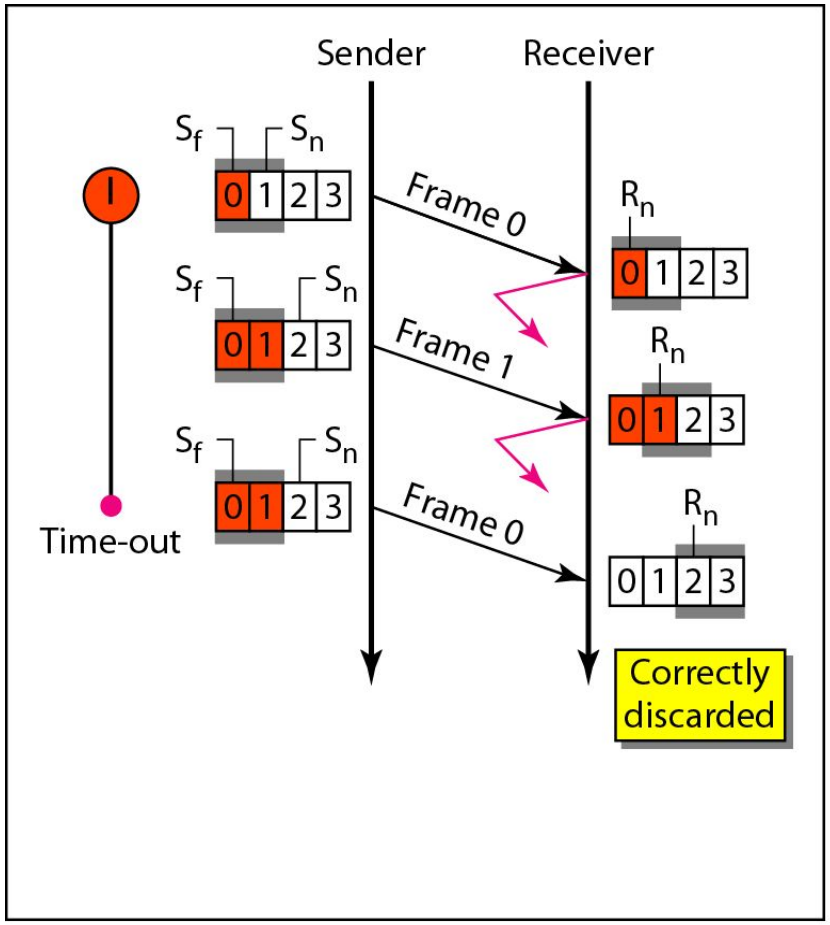
- seq #'s: 0, 1, 2, 3
- window size=3
- receiver sees no difference in two scenarios!
- duplicate data accepted as new in (b)

Q: what relationship between seq # size and window size to avoid problem in (b)?

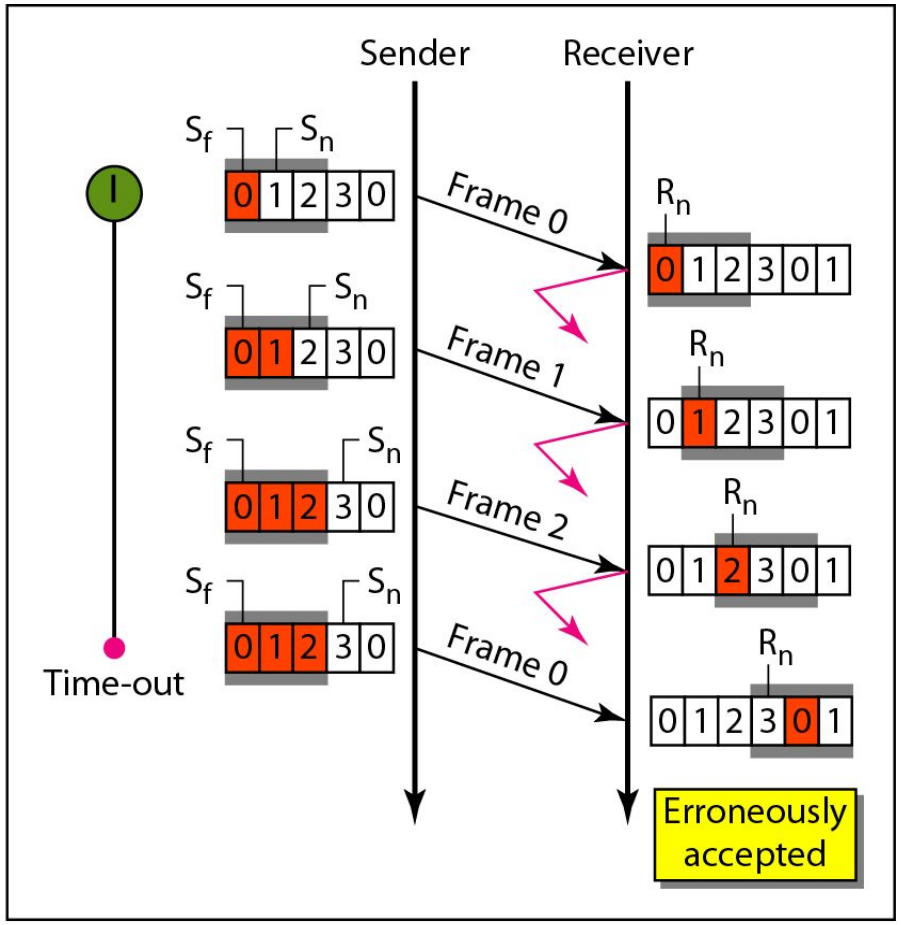


© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Window size for SR



a. Window size = 2^{m-1}



b. Window size > 2^{m-1}

© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

Summary of rdt mechanisms

Mechanism	Use, Comments
Checksum	❖ Used to detect bit errors in a transmitted packet.
Timer	❖ Used to timeout/retransmit a packet, possibly because the packet (or its ACK) was lost within the channel. ❖ Because timeouts can occur when a packet is delayed but not lost (premature timeout), or when a packet has been received by the receiver but the receiver-to-sender ACK has been lost, duplicate copies of a packet may be received by a receiver.
Sequence number	❖ Used for sequential numbering of packets of data flowing from sender to receiver. ❖ Gaps in the sequence numbers of received packets allow the receiver to detect a lost packet. ❖ Packets with duplicate sequence numbers allow the receiver to detect duplicate copies of a packet.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Summary of rdt mechanisms

Mechanism	Use, Comments
Acknowledgment	❖ Used by the receiver to tell the sender that a packet or set of packets has been received correctly. ❖ Acknowledgments will typically carry the sequence number of the packet or packets being acknowledged. ❖ Acknowledgments may be individual or cumulative, depending on the protocol.
Negative acknowledgment	❖ Used by the receiver to tell the sender that a packet has not been received correctly. ❖ Negative acknowledgments will typically carry the sequence number of the packet that was not received correctly.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Summary of rdt mechanisms

Mechanism	Use, Comments
Window, pipelining	❖ The sender may be restricted to sending only packets with sequence numbers that fall within a given range. ❖ By allowing multiple packets to be transmitted but not yet acknowledged, sender utilization can be increased over a stop-and-wait mode of operation. ❖ We'll see shortly that the window size may be set on the basis of the receiver's ability to receive and buffer messages, or the level of congestion in the network, or both.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

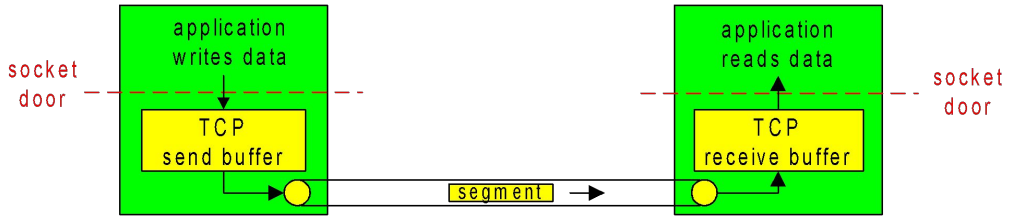
3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

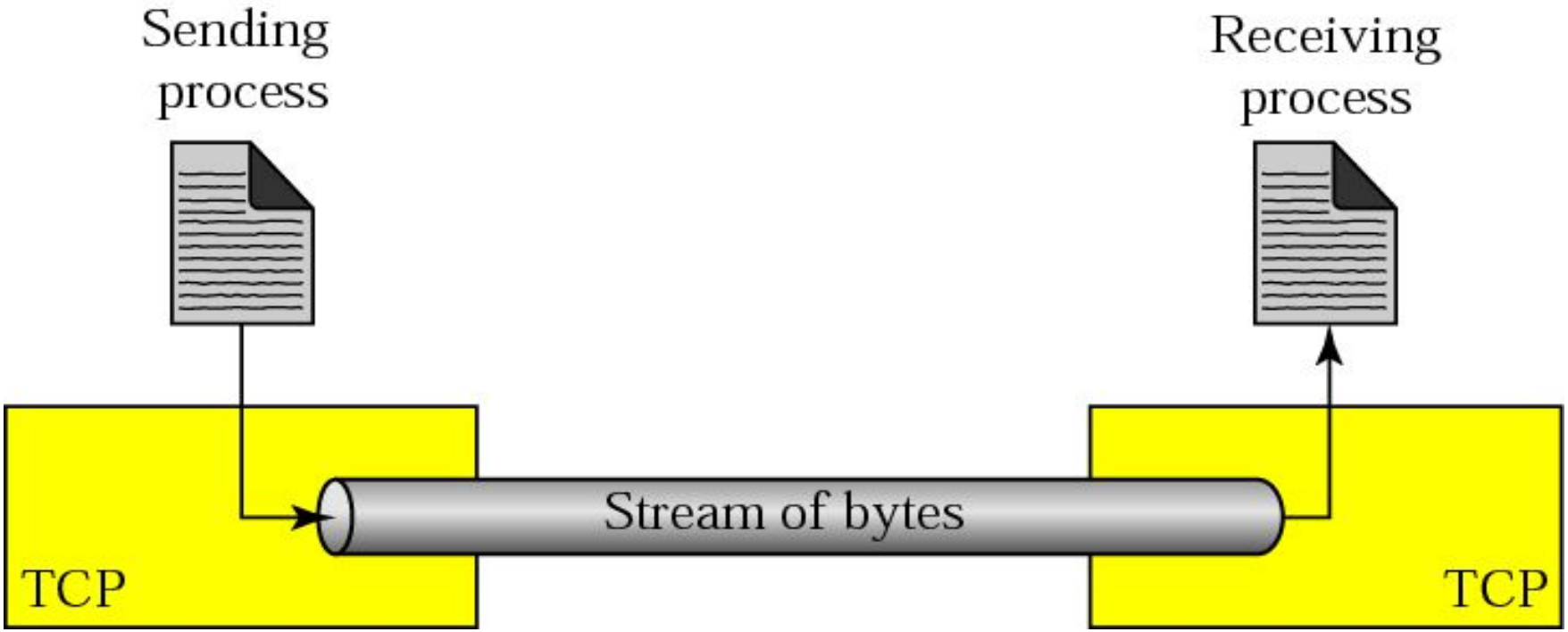
TCP: Overview RFCs: 793,1122,1323, 2018, 2581

- connection-oriented:
  - 3-way handshaking (exchange of control msgs) inits sender, receiver state before data exchange
- full duplex data:
  - bi-directional data flow in same connection
- point-to-point:
  - one sender, one receiver
- reliable, in-order *byte steam:*
  - no "message boundaries"
- *send & receive buffers:*
  - MSS: maximum segment size
  - MTU: Max transmission unit
- pipelined:
  - TCP congestion and flow control set window size
- flow controlled:
  - sender will not overwhelm receiver



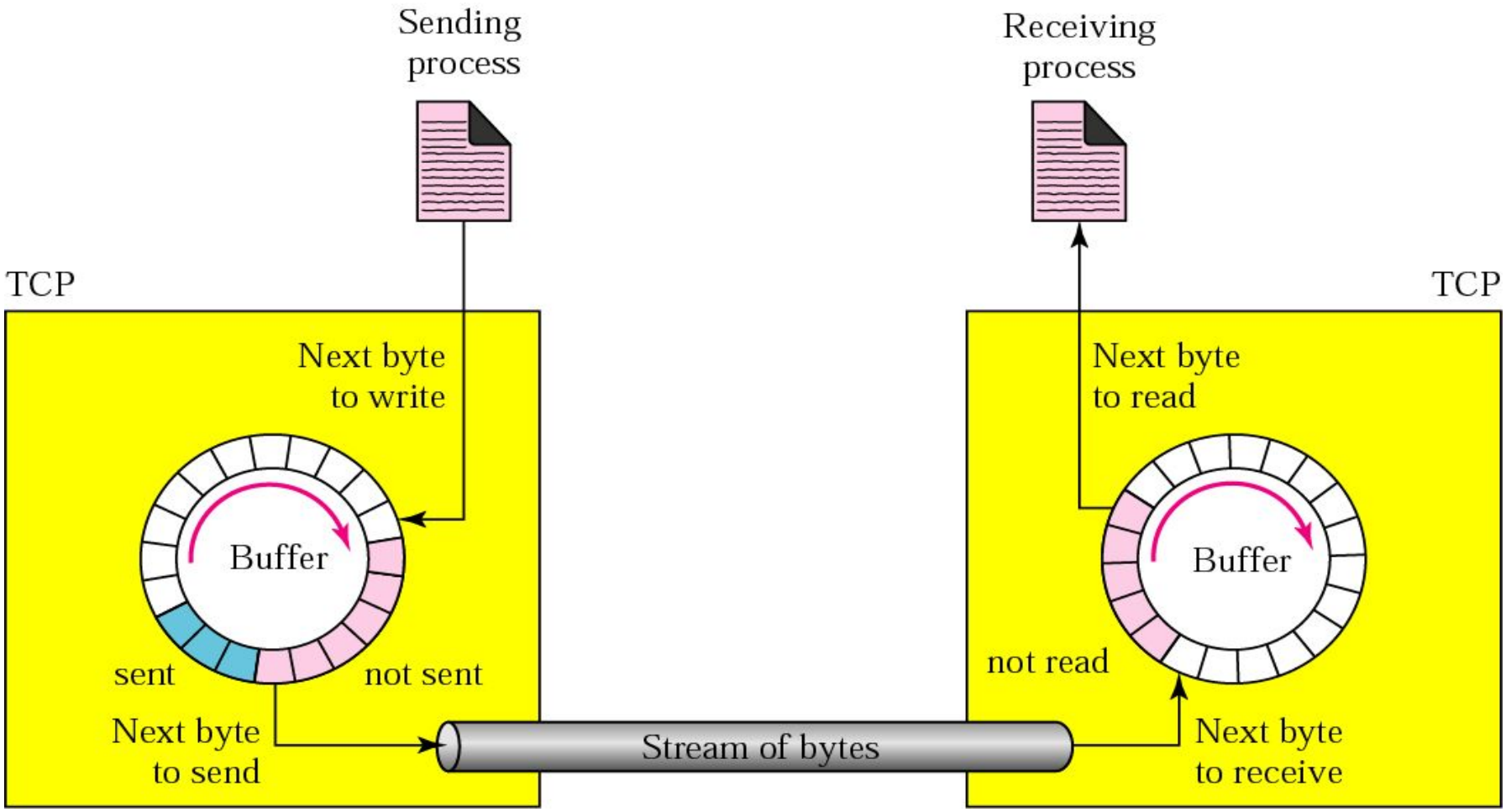
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Stream Delivery



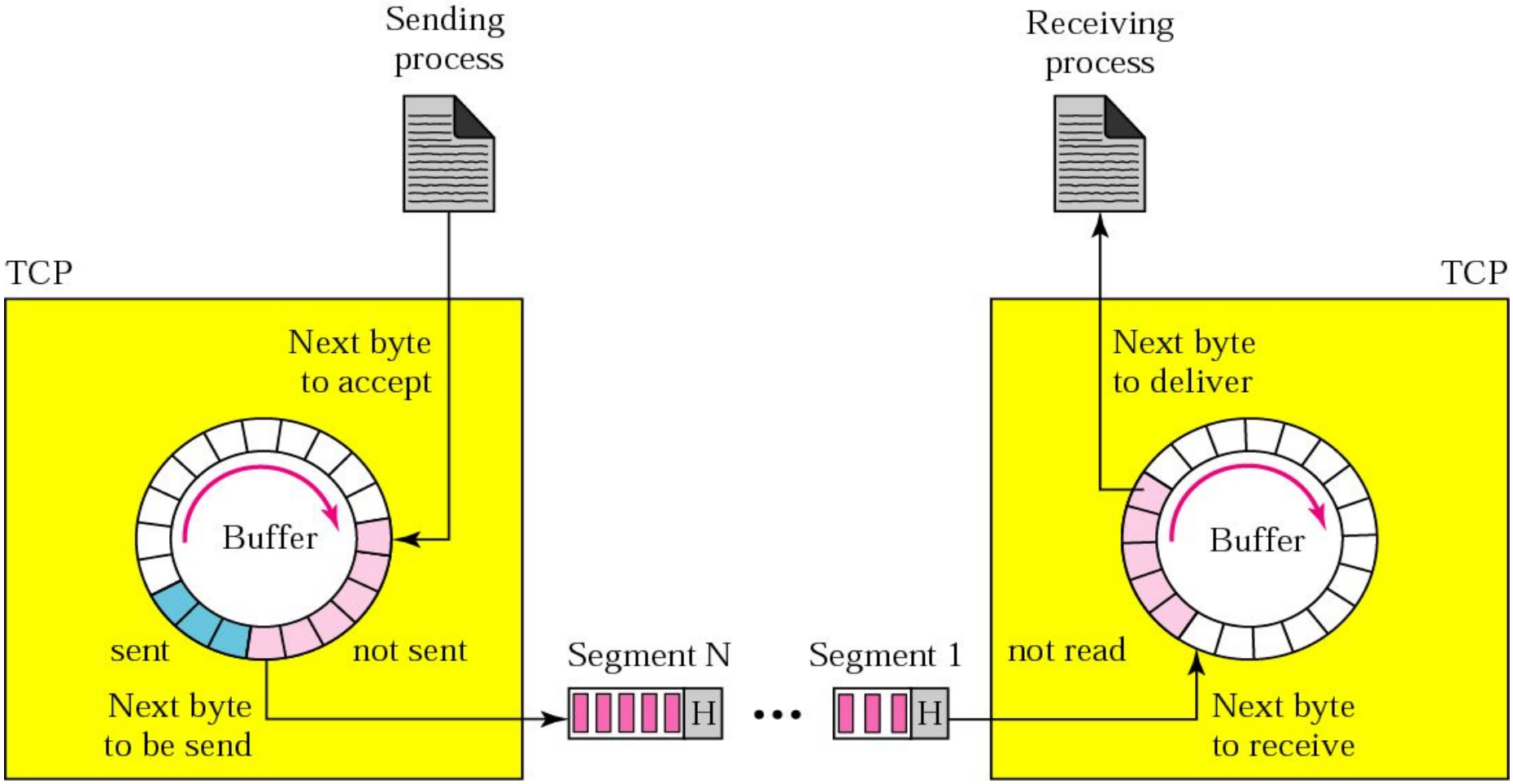
© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

Sending and Receiving buffers



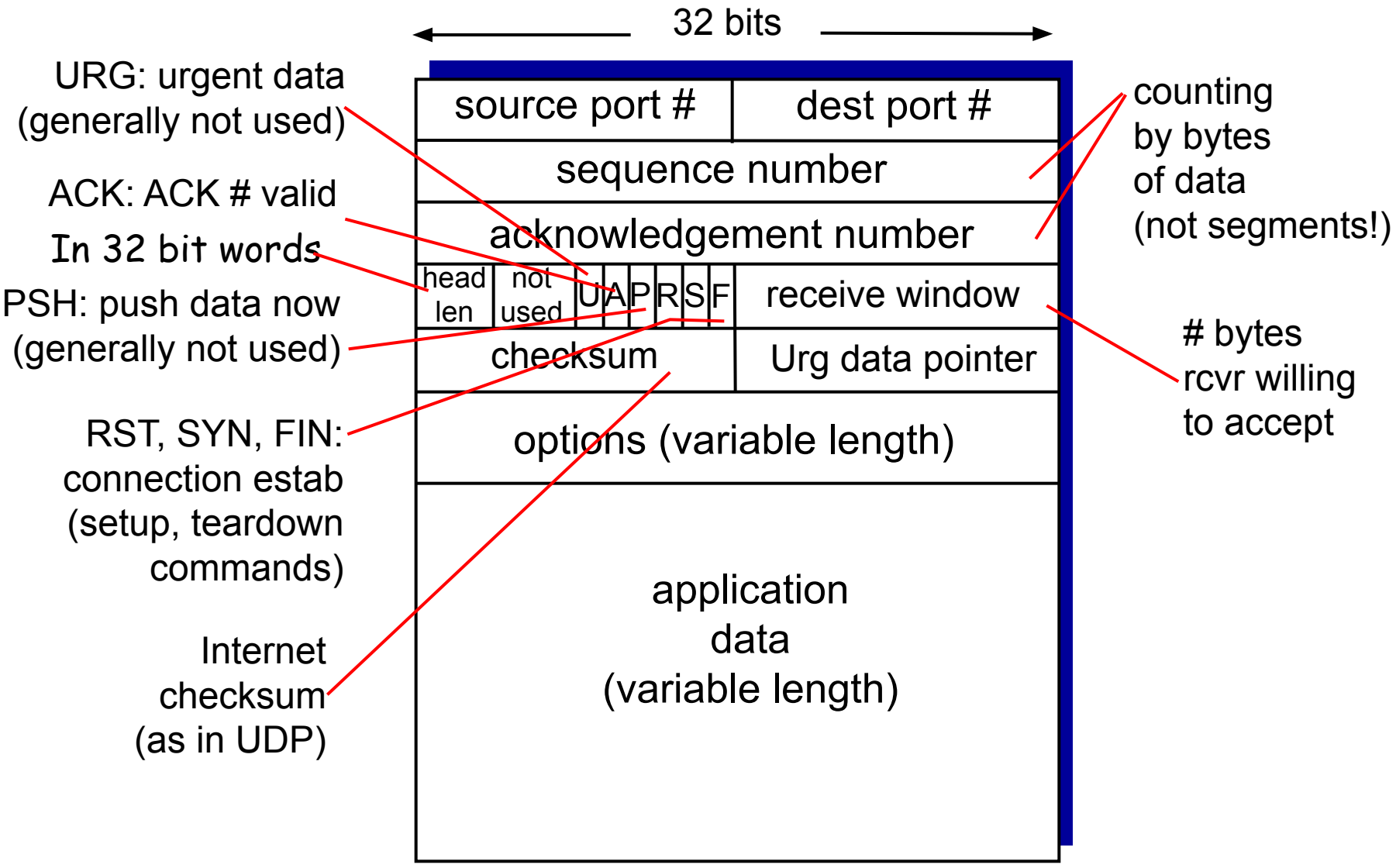
© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

TCP Segments



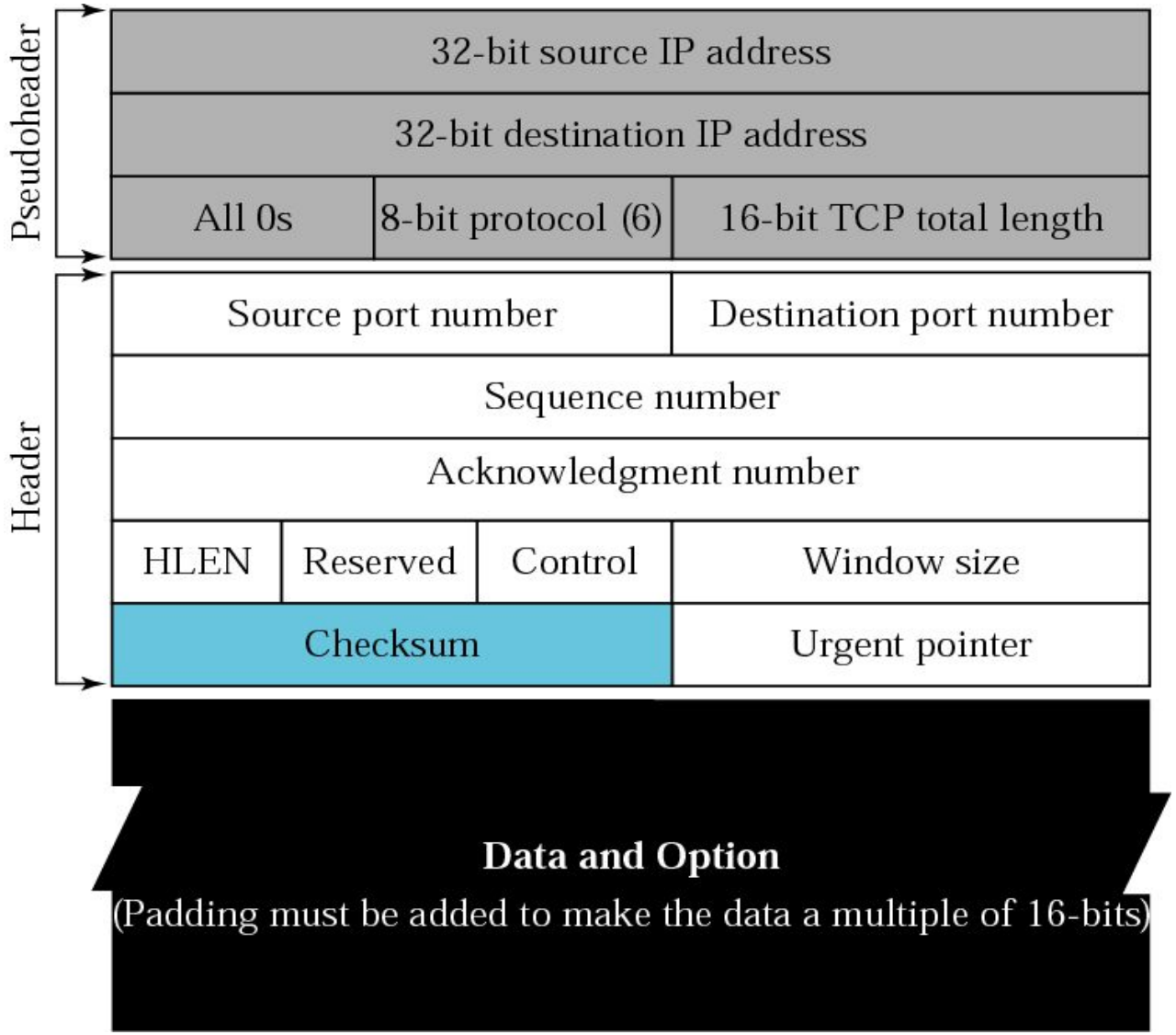
© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

TCP segment structure



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Checksum

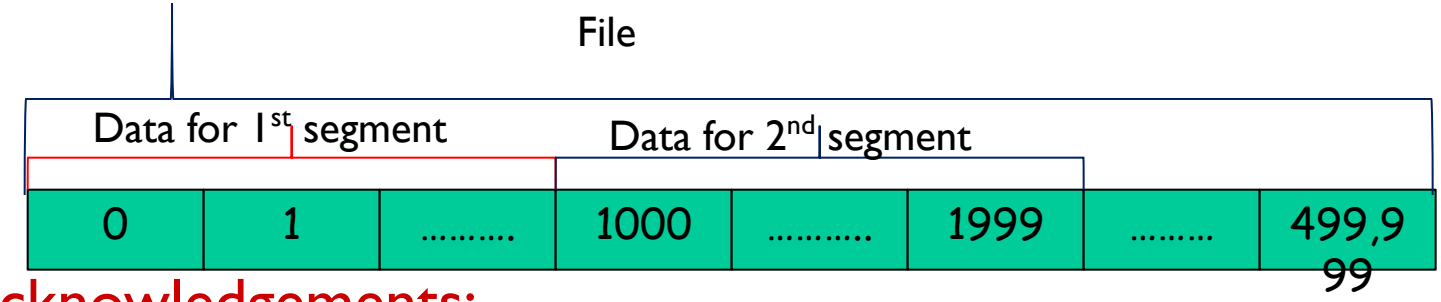


© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

TCP seq. numbers, ACKs

sequence numbers:

- byte stream "number" of first byte in segment's data
- Example
  - Host A to Host B, File consisting of 500,000 bytes, MSS is 1000 bytes, 500 segments, first byte of the data stream is numbered zero ('0')



acknowledgements:

- seq # of next byte expected from other side
- cumulative ACK

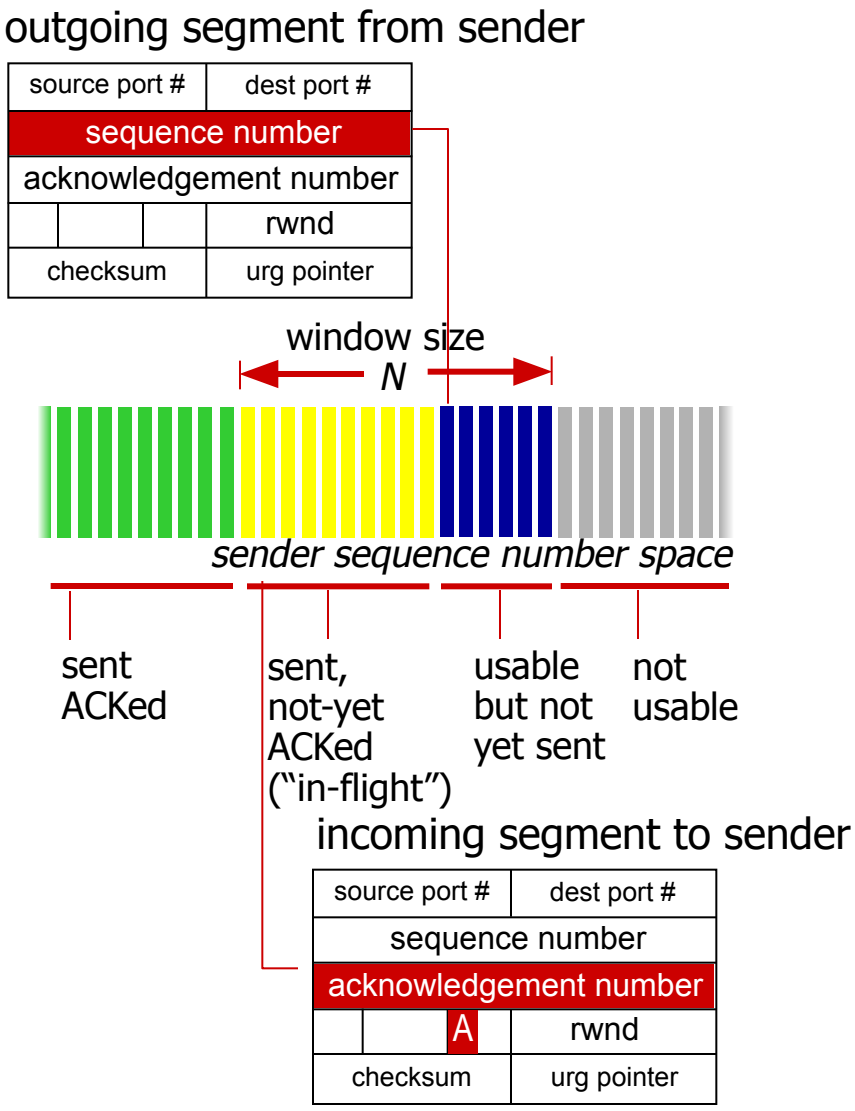
Q: how receiver handles out-of-order segments

- A: TCP spec doesn't say, - up to implementor – GBN or SR

Both sides of TCP connection randomly choose an initial seq. no.

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP seq. numbers, ACKs

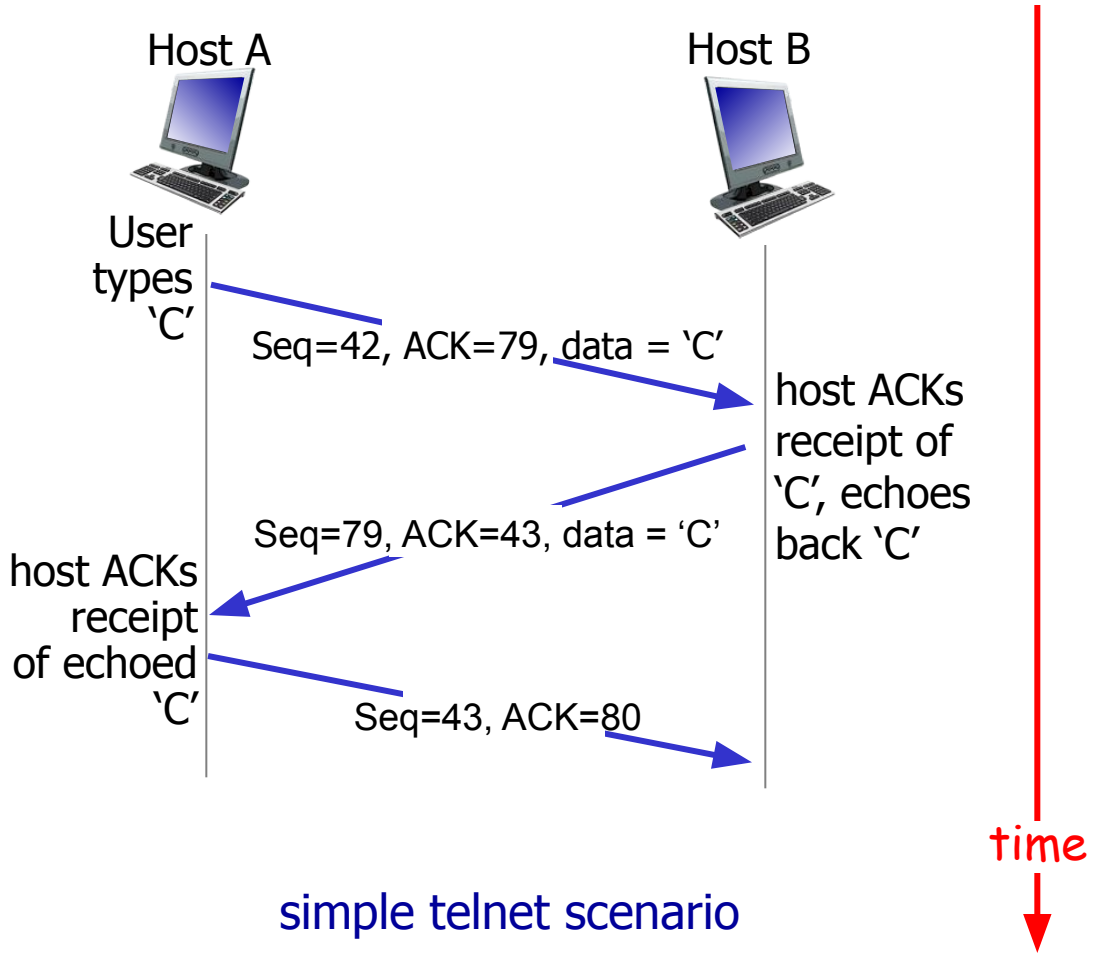


© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP seq. numbers, ACKs

telnet [RFC 854]

- interactive application
- Remote login
- Runs over TCP



simple telnet scenario

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP round trip time, timeout

Q: how to set TCP timeout value?

- longer than RTT
  - but RTT varies
- *too short:* premature timeout, unnecessary retransmissions
- *too long:* slow reaction to segment loss

Q: how to estimate RTT?

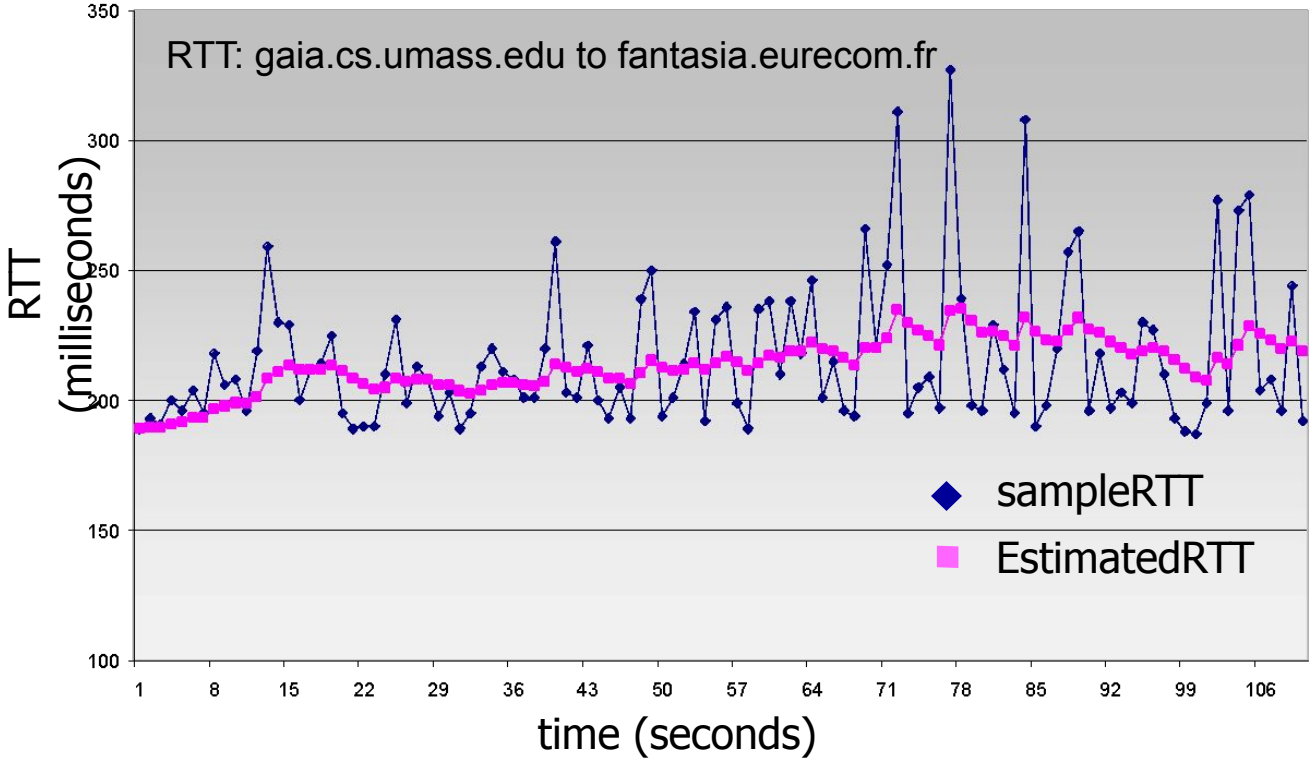
- **`SampleRTT`**: measured time from segment transmission until ACK receipt
  - ignore retransmissions
- **`SampleRTT`** will vary, want estimatedRTT "smoother"
  - average several *recent* measurements, not just current **`SampleRTT`**

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP round trip time, timeout

`EstimatedRTT = (1- α)*EstimatedRTT + α*SampleRTT`

- exponential weighted moving average (EWMA)
- influence of past sample decreases exponentially fast
- typical value: $\alpha = 0.125$ [RFC 6298]



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP round trip time, timeout

- timeout interval: **`EstimatedRTT`** plus "safety margin"
  - large variation in **`EstimatedRTT`** -> larger safety margin
- estimate SampleRTT deviation from EstimatedRTT:

```
DevRTT = (1-β)*DevRTT +
                 β*|SampleRTT-EstimatedRTT|

                 (typically, β = 0.25)
```

```
TimeoutInterval = EstimatedRTT + 4*DevRTT
```



estimated RTT

"safety margin"

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP reliable data transfer

- TCP creates rdt service on top of IP's unreliable service
  - pipelined segments
  - cumulative acks
  - single retransmission timer
- retransmissions triggered by:
  - timeout events - TCP Tahoe
  - 3 duplicate acks - TCP Reno

let's initially consider simplified TCP sender:

- Data is being sent in only one direction
- ignore duplicate acks
- ignore flow control, congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP sender events:

data rcvd from app:

- create segment with seq #
- seq # is byte-stream number of first data byte in segment
- start timer if not already running
  - think of timer as for oldest unacked segment
  - expiration interval: `TimeOutInterval`

timeout:

- retransmit segment that caused timeout
- restart timer

ack rcvd:

- if ack acknowledges previously unacked segments
  - update what is known to be ACKed
  - start timer if there are still unacked segments

© J.F. Kurose, K.W. Ross. All Rights Reserved

```
NextSeqNum = InitialSeqNum
SendBase = InitialSeqNum

loop (forever) {
  switch(event)

  event: data received from application above
      create TCP segment with sequence number NextSeqNum
      if (timer currently not running)
          start timer                for oldest unacknowledged segment
      pass segment to IP
      NextSeqNum = NextSeqNum + length(data)

  event: timer timeout
      retransmit not-yet-acknowledged segment with
          smallest sequence number
      start timer

  event: ACK received, with ACK field value of y
      if (y > SendBase) {
          SendBase = y               seq. no. of oldest unacked byte
          if (there are currently not-yet-acknowledged segments)
              start timer
          }

} /* end of loop forever */
```

TCP sender (simplified)

Comment:

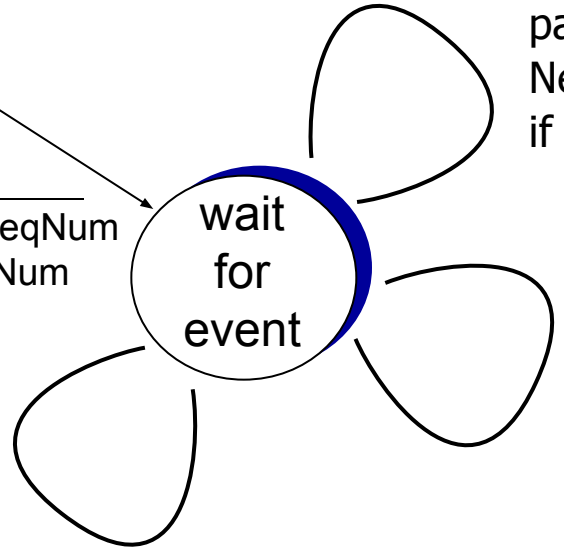
- SendBase-1: last cumulatively ack'ed byte

Example:

- SendBase-1 = 71; y= 73, so the rcvr wants 73+ ; y > SendBase, so that new data is acked

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP sender (simplified)



Λ

NextSeqNum = InitialSeqNum
SendBase = InitialSeqNum

data received from application above

create segment, seq. #: NextSeqNum
pass segment to IP (i.e., "send")
NextSeqNum = NextSeqNum + length(data)
if (timer currently not running)
    start timer

timeout

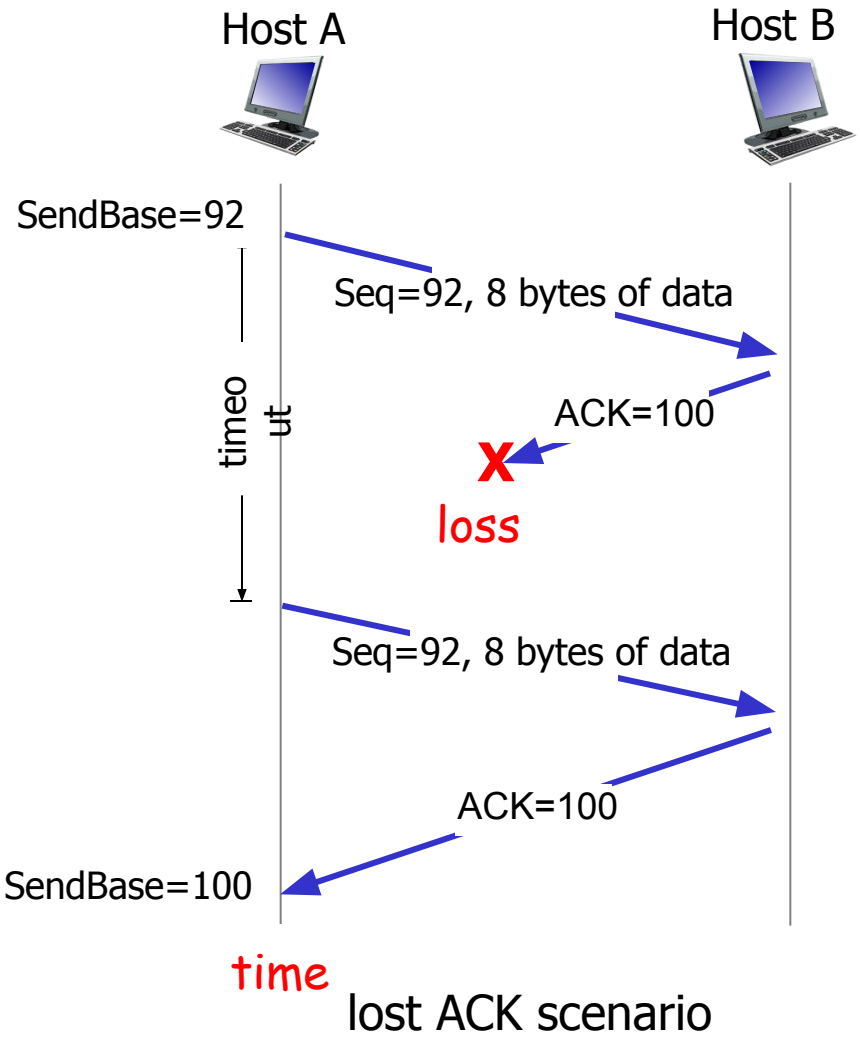
retransmit not-yet-acked segment
        with smallest seq. #
start timer

ACK received, with ACK field value y

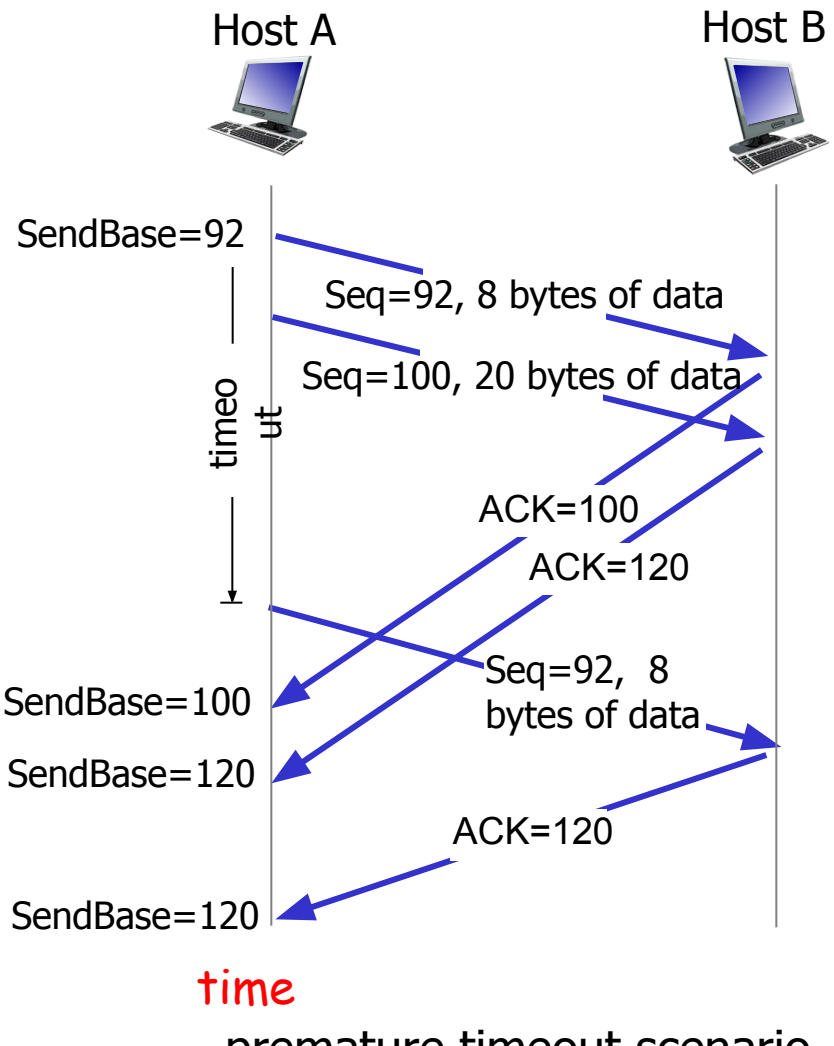
```
if (y > SendBase) {
    SendBase = y
    /* SendBase–1: last cumulatively ACKed byte */
    if (there are currently not-yet-acked segments)
        start timer
      else stop timer
     }
```

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP: retransmission scenarios



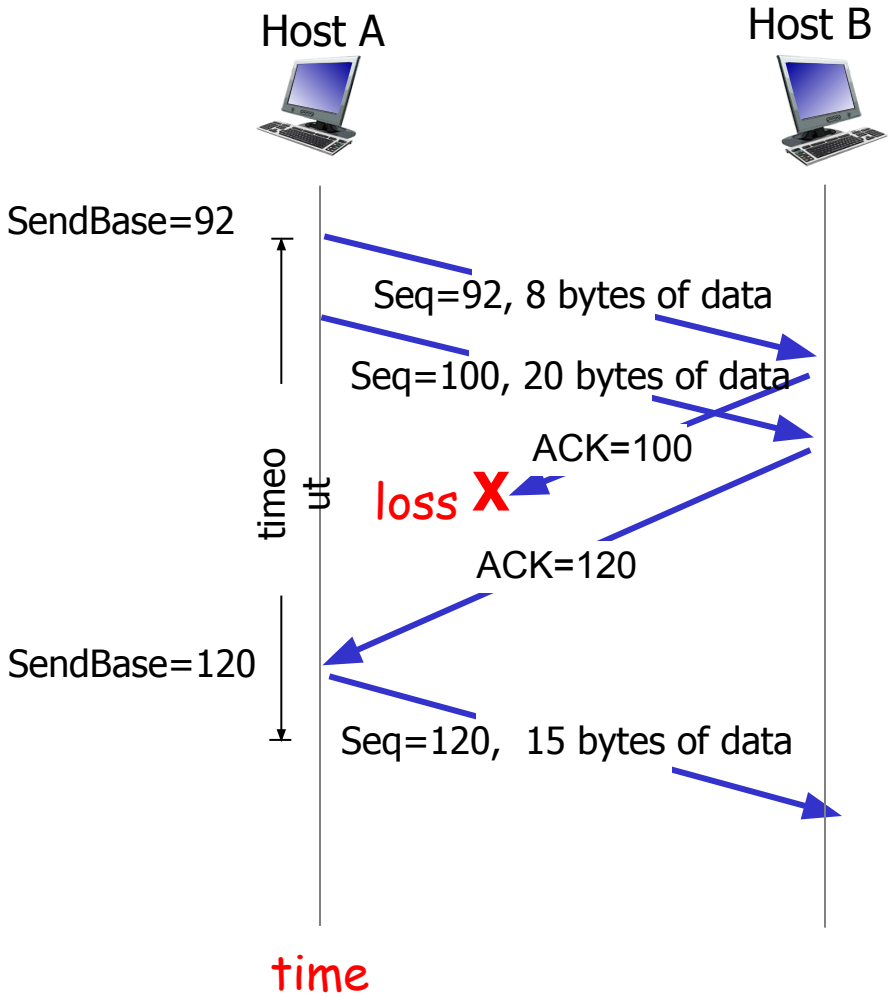
lost ACK scenario



premature timeout scenario

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP: retransmission scenarios



cumulative ACK scenario

© J.F. Kurose, K.W. Ross. All Rights Reserved

Doubling the time out interval:

- *Length of the timeout interval after a timer expiration?*
- Each time TCP retransmits, it sets the next *timeout* interval to twice the previous value, rather than deriving it from **`EstimtedRTT and DevRTT`.**
- Example
  - `TimeoutInterval=0.75 sec`
  - `Expiration time=1.5sec`
  - `new expiration time=3.0sec`
- Intervals grow exponentially after each retransmission.
- However, whenever the timer started after either of the two other events
  - *Data received from application above*
  - *ACK received*

  `TimeoutInterval` is derived from the most recent values of `EstimatedRTT` and `DevRTT`)
- Limited form of congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP fast retransmit

- time-out period often relatively long:
  - long delay before resending lost packet
- detect lost segments via duplicate ACKs.
- Duplicate ACK
  - An ACK that reacknowledges a segment for which the sender has already received an earlier acknowledgement.

- Why the receiver sends a duplicate ACK ?

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP ACK generation [RFC 1122, RFC 2581]

event at receiver	*TCP receiver action*
arrival of in-order segment with expected seq #. All data up to expected seq # already ACKed	delayed ACK. Wait up to 500ms for next segment. If no next segment, send ACK
arrival of in-order segment with expected seq #. One other segment has ACK pending	immediately send single cumulative ACK, ACKing both in-order segments
arrival of out-of-order segment higher-than-expect seq. # . Gap detected	immediately send *duplicate ACK*, indicating seq. # of next expected byte
arrival of segment that partially or completely fills gap	immediate send ACK, provided that segment starts at lower end of gap

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP fast retransmit

- detect lost segments via duplicate ACKs.
  - sender often sends many segments back-to-back
  - if segment is lost, there will likely be many duplicate ACKs.

TCP fast retransmit

if sender receives 3 ACKs for same data ("triple duplicate ACKs"), resend unacked segment with smallest seq #

- likely that unacked segment lost, so don't wait for timeout

© J.F. Kurose, K.W. Ross. All Rights Reserved

Fast retransmit algorithm:

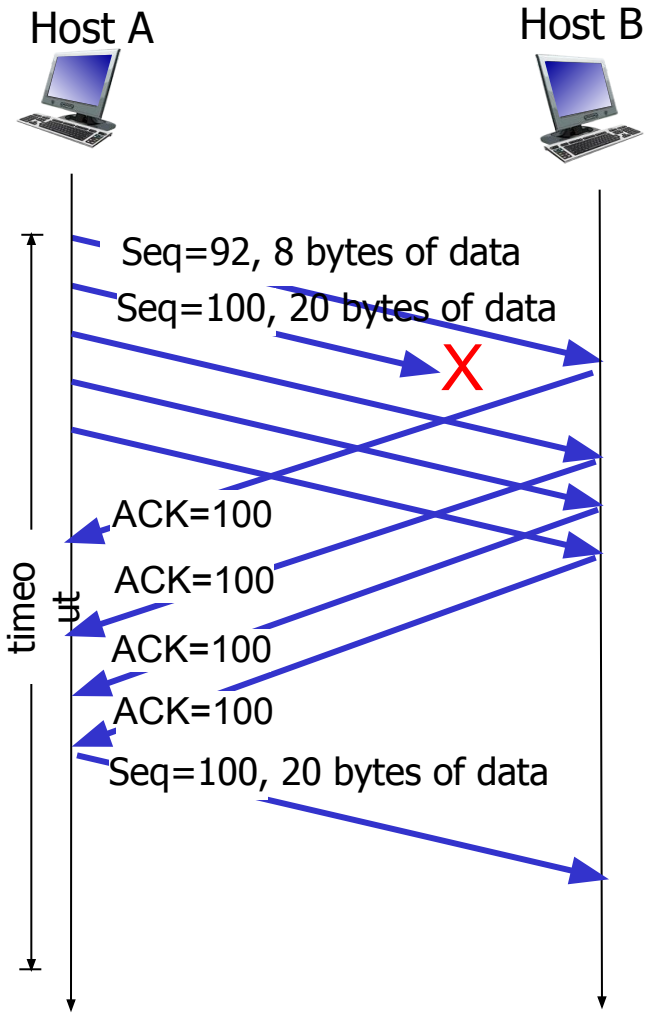
```
event: ACK received, with ACK field value of y
            if (y > SendBase) {
                SendBase = y
                if (there are currently not-yet-acknowledged segments)
                    start timer
              }
            else {
                increment count of dup ACKs received for y
                if (count of dup ACKs received for y == 3) {
                    resend segment with sequence number y
                }
```

a duplicate ACK for already ACKed segment

fast retransmit

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP fast retransmit



fast retransmit after sender receipt of triple duplicate ACK

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Go-Back-N or Selective Repeat?

- GBN style protocol
  - TCP acknowledgements are cumulative
  - Correctly received but out-of-order segments are not individually acknowledged
  - Need to maintain **SendBase** and **NextSeqNum**

© J.F. Kurose, K.W. Ross. All Rights Reserved

Go-Back-N or Selective Repeat?

- Differences between TCP and Go-Back-N
  - Buffer correctly received but out-of-order segments
- Example
  - Sender sends segments 1, 2, . . . , *N*
  - All of the segments arrive in order without error at the receiver.
  - Suppose that the acknowledgment for packet $n < N$ gets lost, but the remaining $N - 1$ acknowledgments arrive at the sender before their respective timeouts.
  - GBN would retransmit not only packet *n*, but also all of the subsequent packets $n + 1, n + 2, \ldots, N$.
  - TCP, on the other hand, would retransmit at most one segment, namely, segment *n*.
  - TCP would not even retransmit segment *n* if the acknowledgment for segment $n + 1$ arrived before the timeout for segment *n*.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Go-Back-N or Selective Repeat?

- Selective Acknowledgement - SACK
  - Acknowledge out-of-order segments selectively rather than just cumulatively acknowledging the last correctly received in-order segment.
- Hybrid of Go-Back-N and SR

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

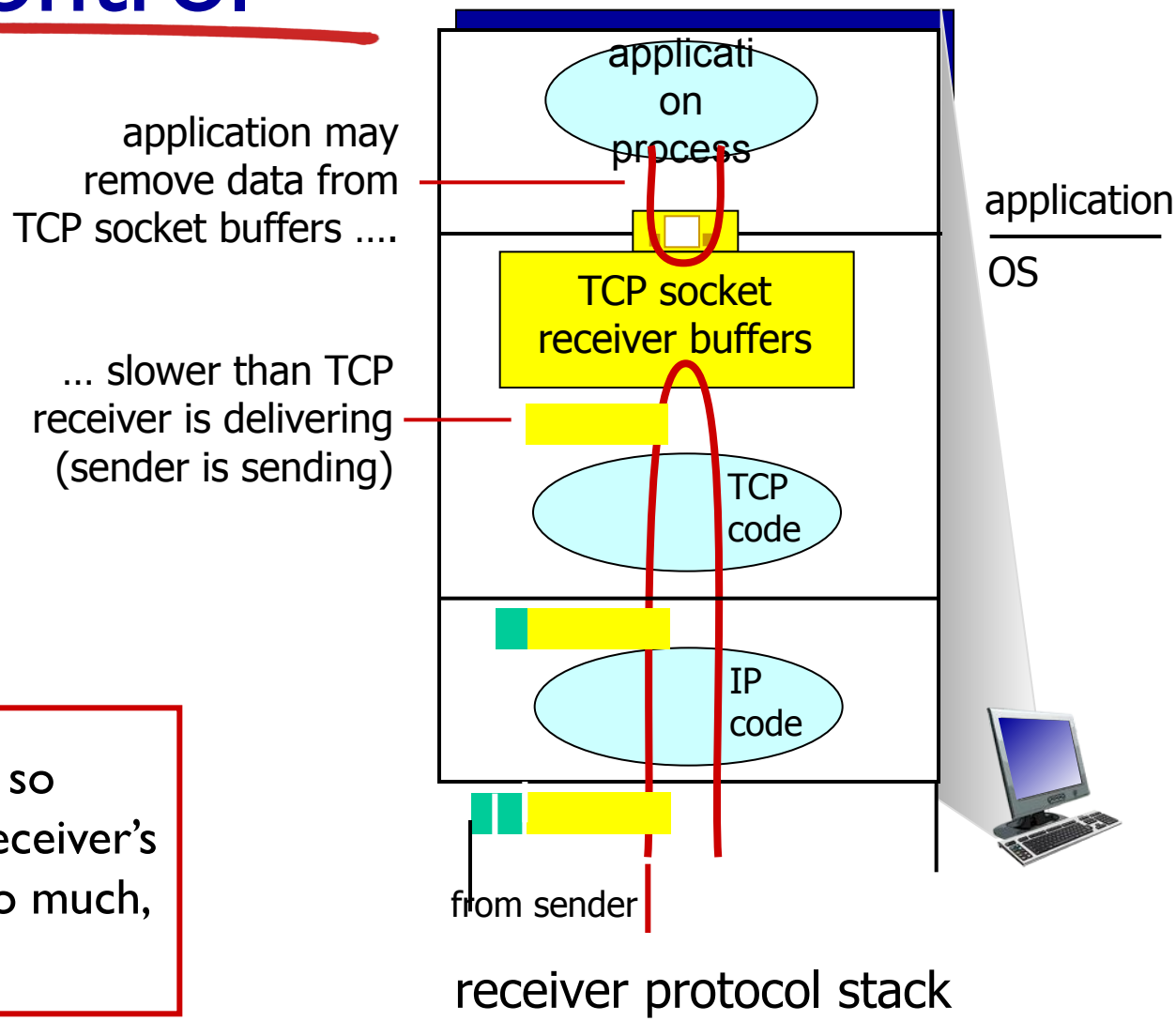
3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP flow control

- speed-matching service: matching the send rate to the receiving app's drain rate



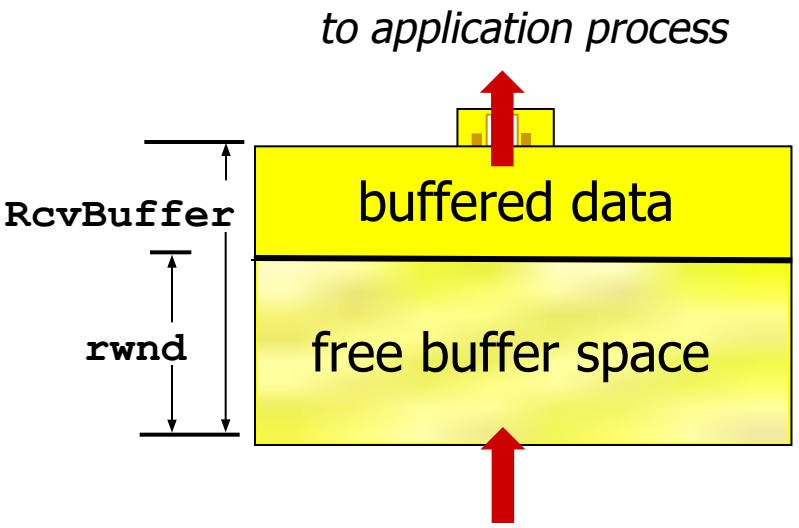
flow control

receiver controls sender, so sender won't overflow receiver's buffer by transmitting too much, too fast

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP flow control

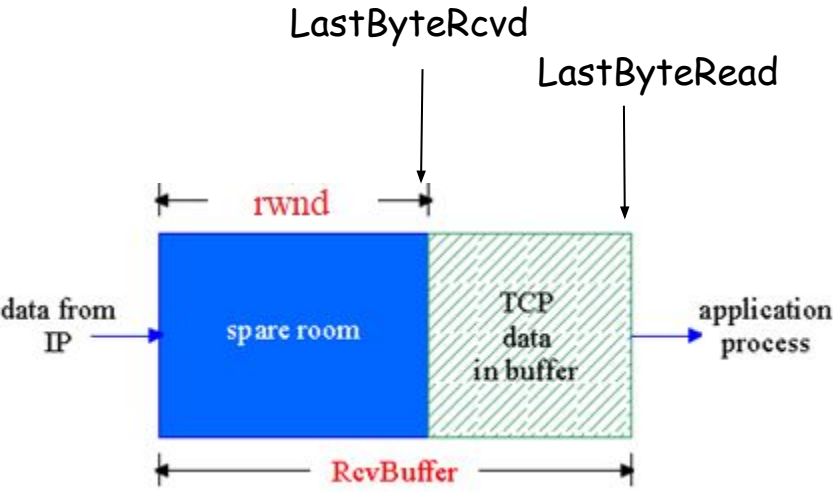
- receiver "advertises" free buffer space by including **`rwnd`** value in TCP header of receiver-to-sender segments
  - **`RcvBuffer`** size set via socket options (typical default is 4096 bytes)
  - many operating systems autoadjust **`RcvBuffer`**
- sender limits amount of unacked ("in-flight") data to receiver's **`rwnd`** value
- guarantees receive buffer will not overflow



receiver-side buffering

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Flow control: how it works



Host B

LastByteRead

The number of the last byte in the data stream read from the buffer by the application process in host B

LastByteRcvd

The number of the last byte in the data stream that has arrived from the network and has been placed in the receive buffer at B

- File transfer
  - Host A is sending a large file to Host B over a TCP connection.
  - Host B allocates a receive buffer to this connection : **RcvBuffer**
- TCP is not permitted to overflow the allocated buffer

$$LastByteRcvd - LastByteRead \leq RcvBuffer$$

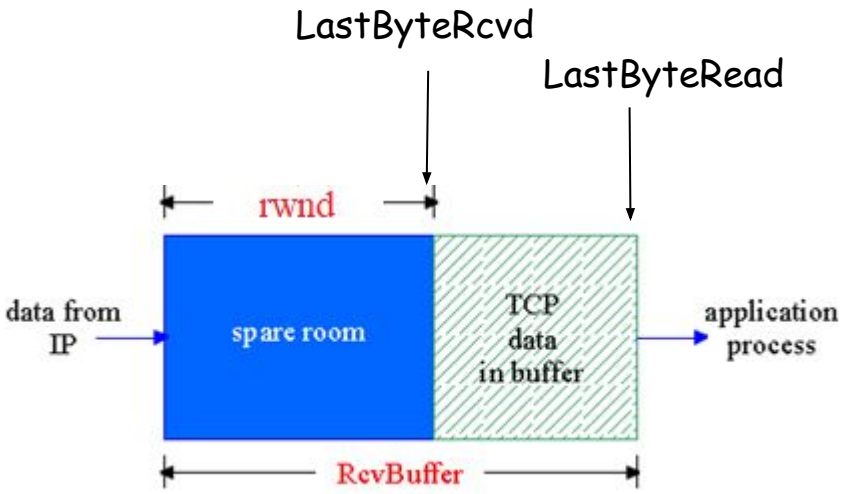
- The receive window is set to the amount of spare room in buffer

$$rwnd = RcvBuffer - [LastByteRcvd - LastByteRead]$$

- *rwnd* is dynamic because the spare room changes with time

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Flow control: how it works



- *rwnd* provide the flow control
  - Host B tells Host A how much spare room it has in the connection buffer by placing its current value of *rwnd* in the receive window field of every segment it sends to A.
  - Initially, Host B sets ***rwnd = RcvBuffer***

Host A

- The ***amount of unacknowledged data*** that A has sent into connection ***LastByteSent – LastByteAcked***
- By keeping the amount of unacknowledged data less than the value of *rwnd*, Host A is assured that it is not overflowing the receive buffer at Host B.

LastByteSent – LastByteAcked ≤ rwnd

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Flow control: Technical Problem

Problem:

- Host B's receive buffer is full

  rwnd = 0 sent to Host A
- *Assume that B has nothing to send to A*
- Application process at B empties the buffer, TCP does not send new segments with new *rwnd* values to Host A
- TCP sends a segment to Host A only
  - if it has data to send

    or
  - if it has an acknowledgement to send
- Host A is never informed that some space has opened up in Host B's receive buffer – Host A is blocked and can transmit no more data.

Solution:

- Host A continue to send segments with one data byte when B's receive window is zero.
- These segments will be acknowledged by the receiver.
- Eventually the buffer will begin to empty and the acknowledgements will contain a nonzero *rwnd* value.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

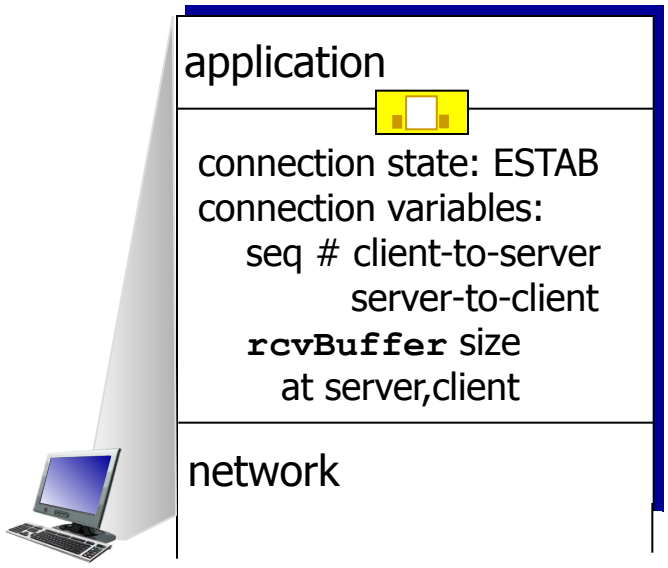
3.7 TCP congestion control

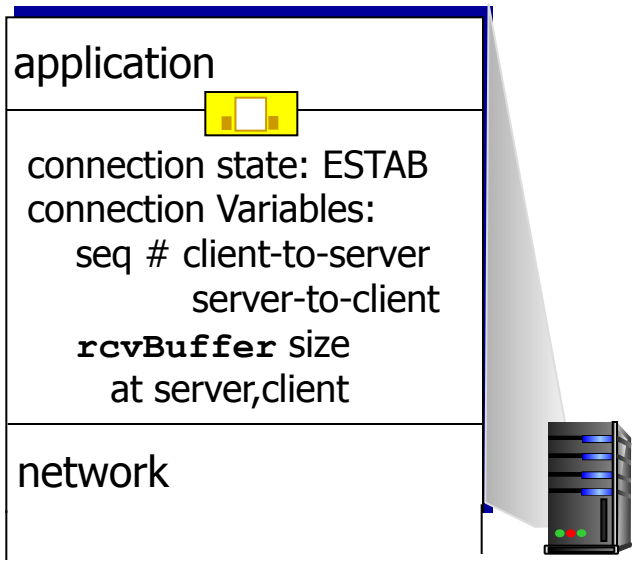
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Connection Management

before exchanging data, sender/receiver "handshake":

- agree to establish connection (each knowing the other willing to establish connection)
- agree on connection parameters



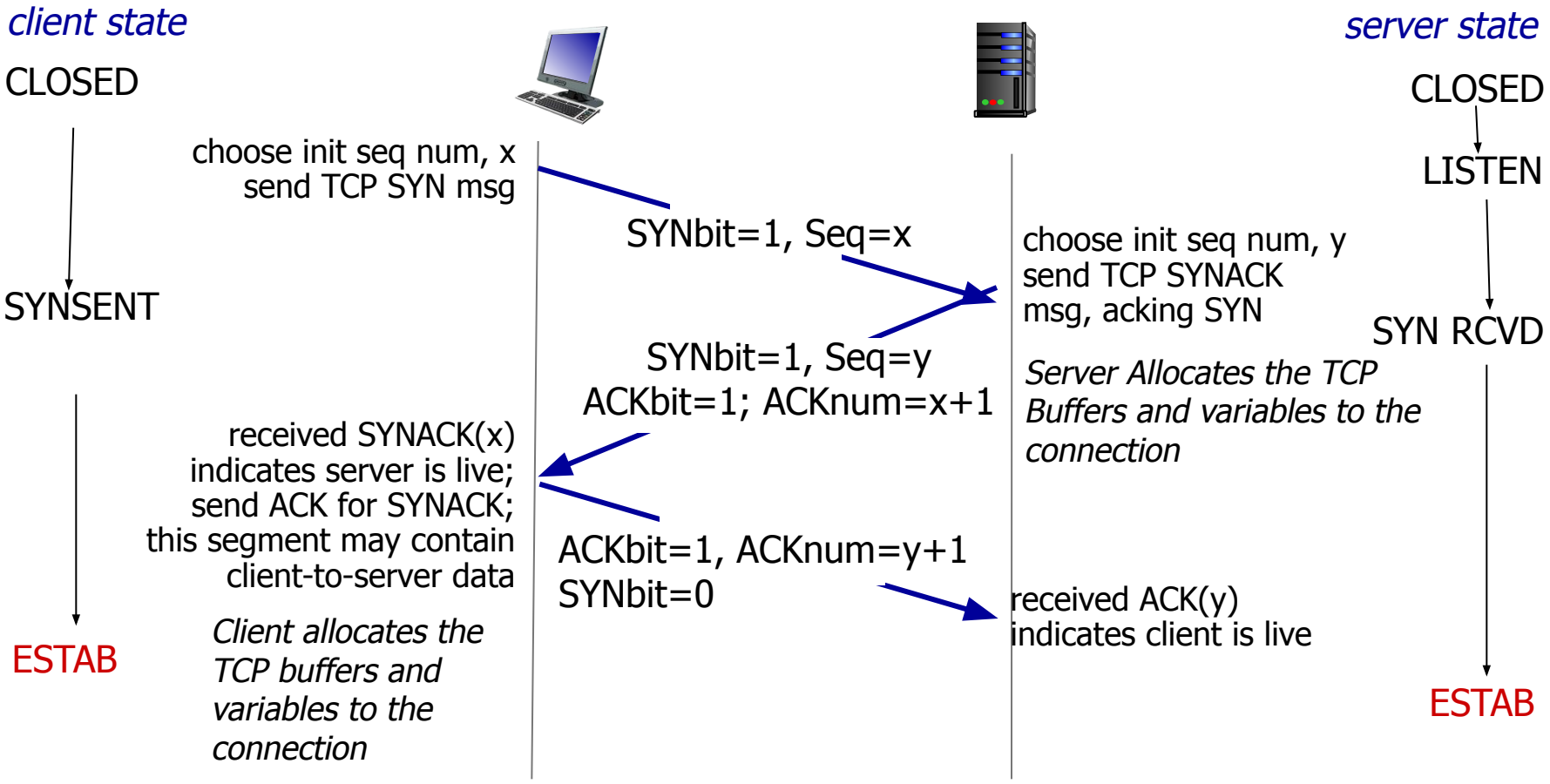



```
Socket clientSocket =
  newSocket("hostname","port number");
```

```
Socket connectionSocket =
  welcomeSocket.accept();
```

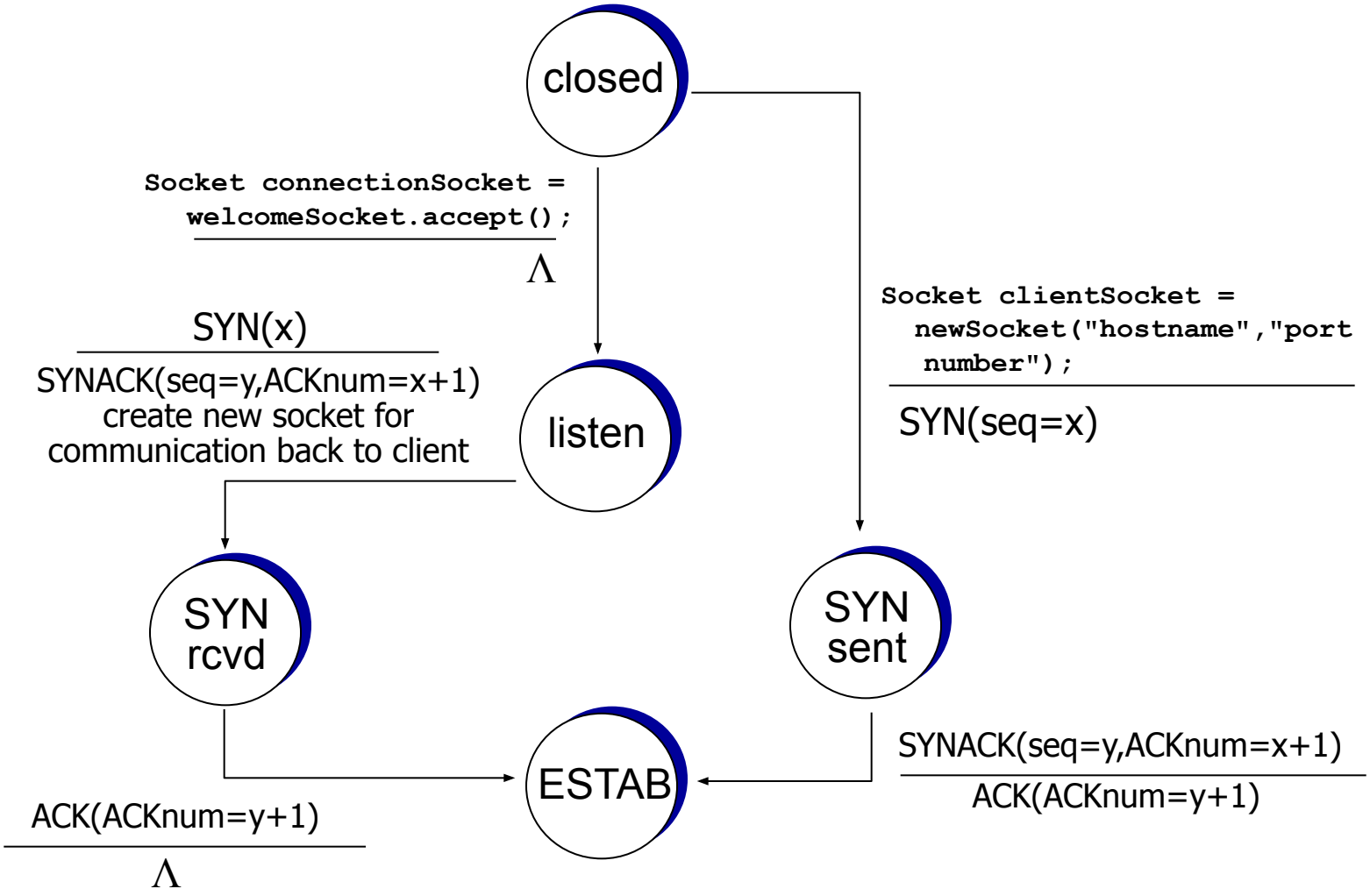
© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP 3-way handshake



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP 3-way handshake: FSM



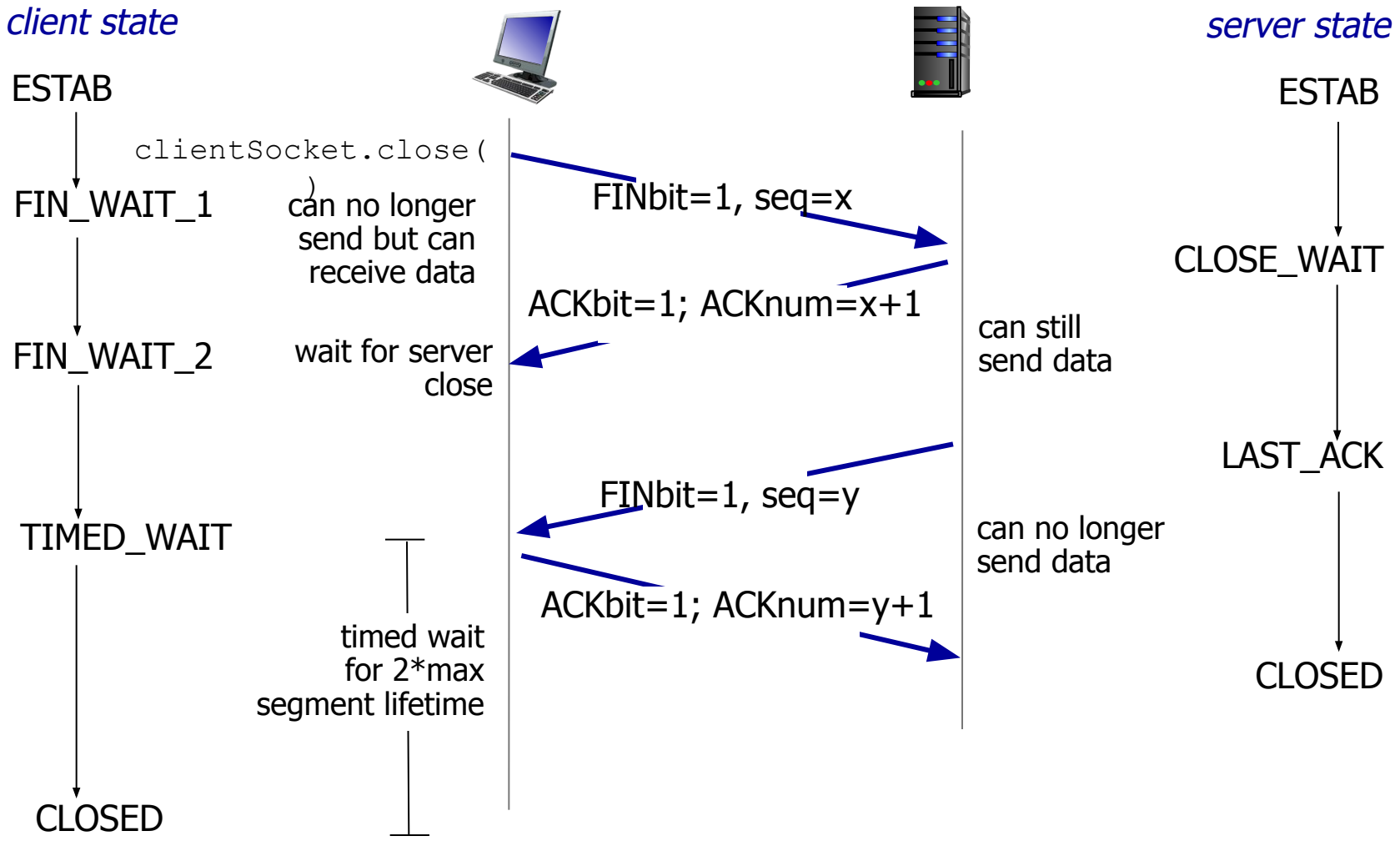
© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP: closing a connection

- client, server each close their side of connection
  - send TCP segment with FIN bit = 1
- respond to received FIN with ACK
  - on receiving FIN, ACK can be combined with own FIN

© J.F. Kurose, K.W. Ross. All Rights Reserved

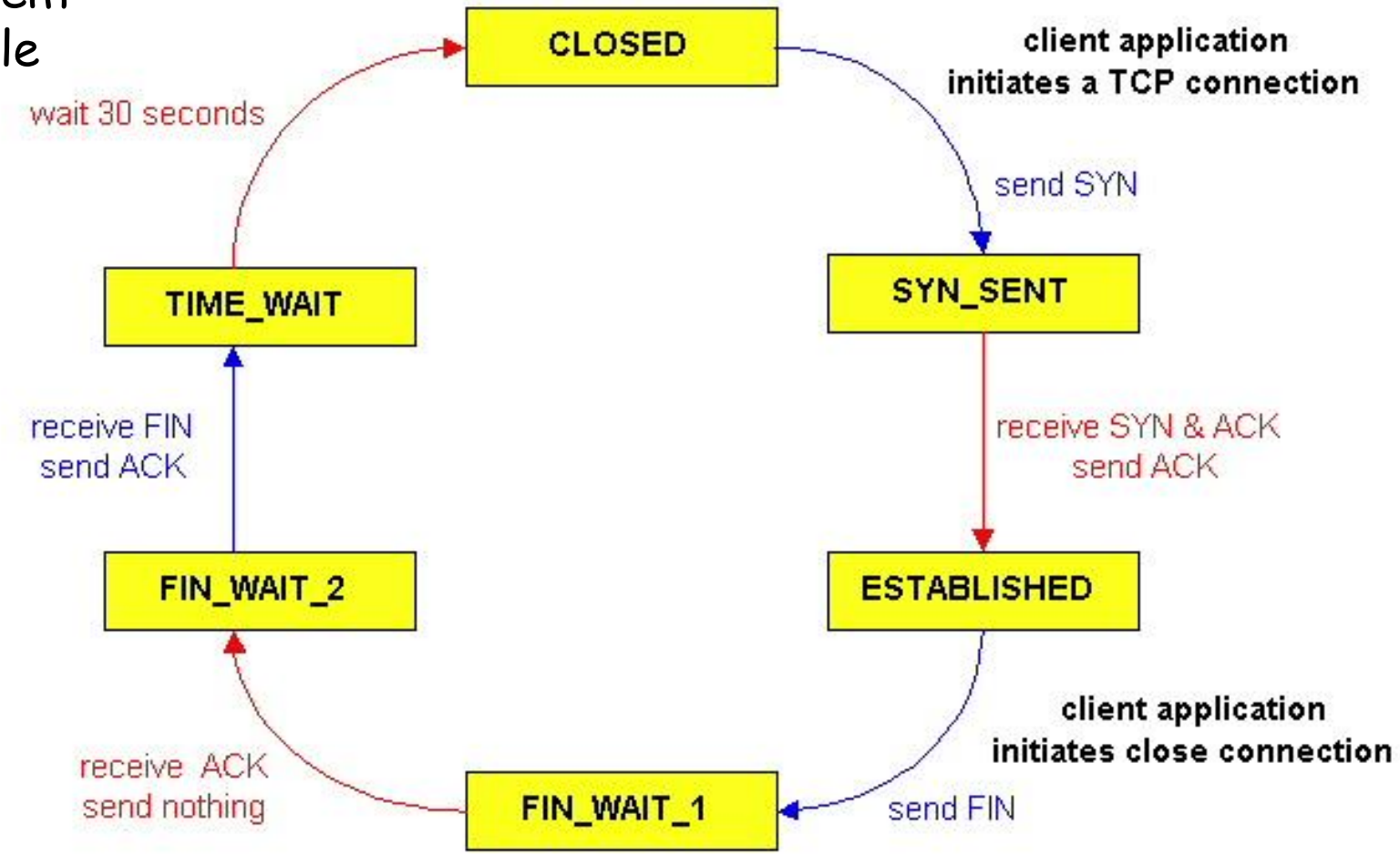
TCP: closing a connection



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Client States

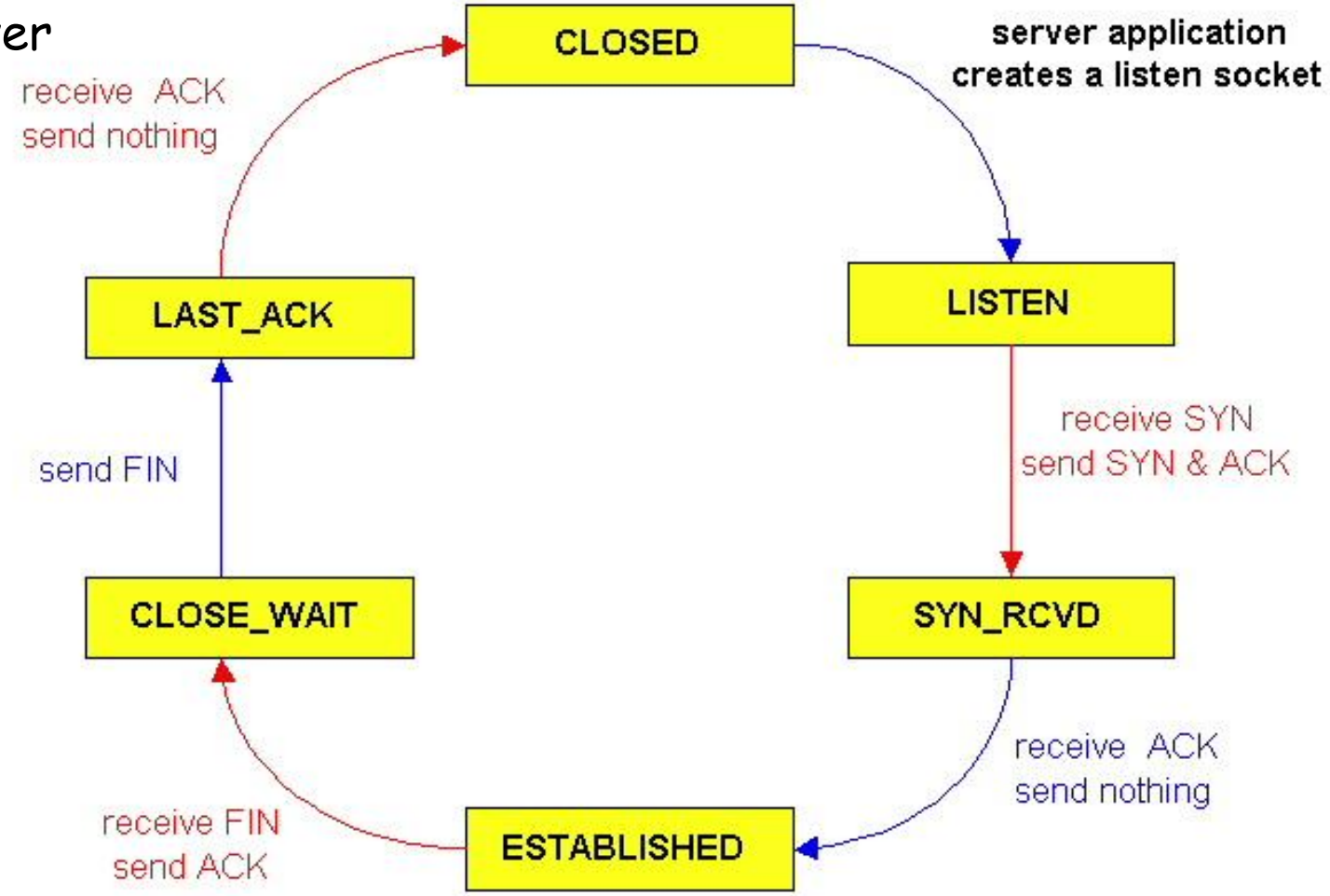
TCP client lifecycle



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Server States

TCP server lifecycle



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP: Resetting a connection

- When a host receives a TCP segment whose port numbers or source IP address do not match with any of the ongoing sockets in the host.
  - For example, suppose a host receives a TCP SYN packet with destination port 80, but the host is not accepting connections on port 80 (that is, it is not running a Web server on port 80).
  - Then the host will send a special reset segment to the source. This TCP segment has the RST flag bit set to 1.
  - Thus, when a host sends a reset segment, it is telling the source "I don't have a socket for that segment. Please do not resend the segment."

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP: Resetting a connection

- When a host receives a UDP packet whose destination port number doesn't match with an ongoing UDP socket
  - The host sends a special ICMP datagram

© J.F. Kurose, K.W. Ross. All Rights Reserved

nmap: port scanning tool

- Explore a specific TCP port, say port 6789, on a target host
  - nmap will send a TCP SYN segment with destination port 6789 to that host
- Possible outcomes:
  - source host receives a TCP SYNACK segment from the target host
  - source host receives a TCP RST segment from the target host
  - source receives nothing

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

Principles of congestion control

congestion:

- informally: "too many sources sending too much data too fast for *network* to handle"
- different from flow control!
- manifestations:
  - lost packets (buffer overflow at routers)
  - long delays (queueing in router buffers)
- a top-10 problem!

© J.F. Kurose, K.W. Ross. All Rights Reserved

Causes and Costs of Congestion

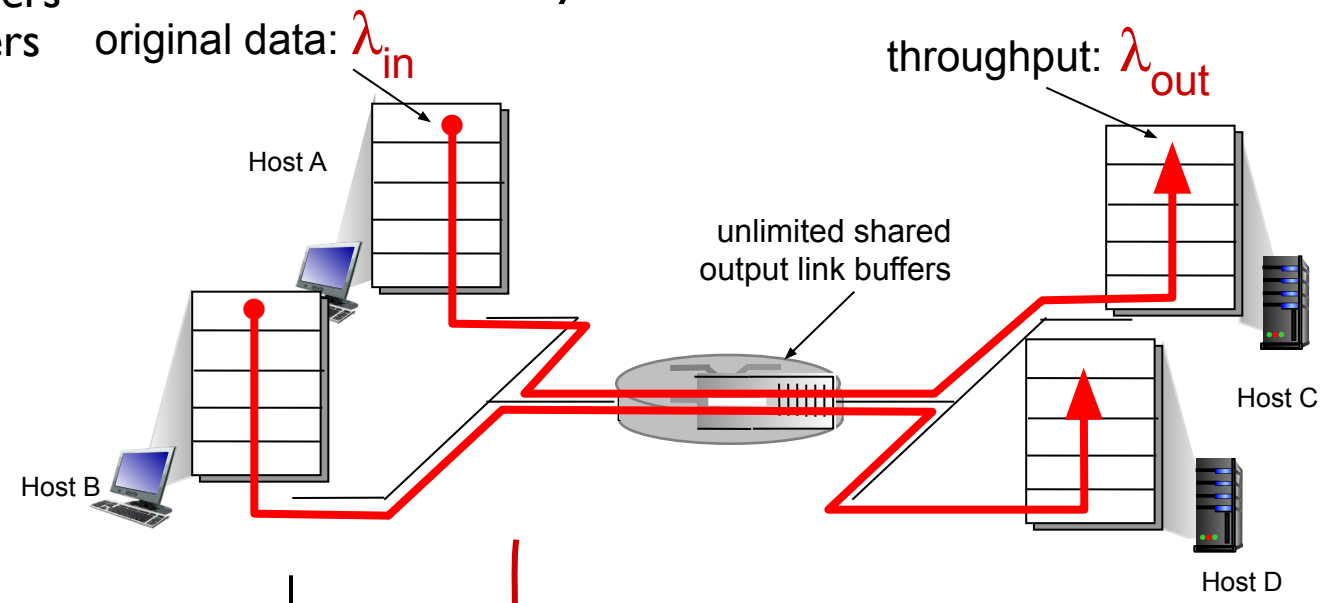
- Three increasingly complex scenarios
  - Causes - Why congestion occurs?
  - Costs of Congestion
    - Resources not fully utilized
    - Poor performance received by the end systems
- Ignore
  - How to react or avoid congestion?
- Focus on
  - What happens as hosts increase their transmission rate and the network becomes congested

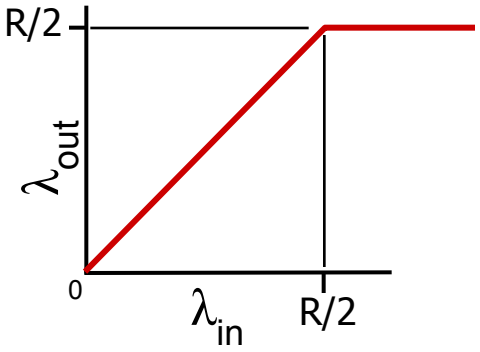
© J.F. Kurose, K.W. Ross. All Rights Reserved

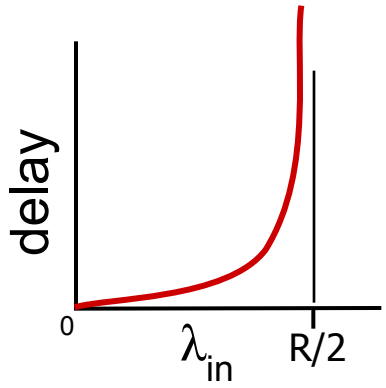
Scenario 1: Two senders, a Router with Infinite Buffers

- two senders, two receivers
- one router, infinite buffers
- output link capacity: R
- Average rate = λ_{in} Bytes/sec
- no retransmission
  - no error recovery, flow or congestion control
- Ignore additional overhead

- Extremely idealized scenario







- Cost: large queuing delays are experienced as the packet arrival rate nears the link capacity

- maximum **per-connection throughput**: R/2
- large delays as arrival rate, λ_{in}, approaches capacity

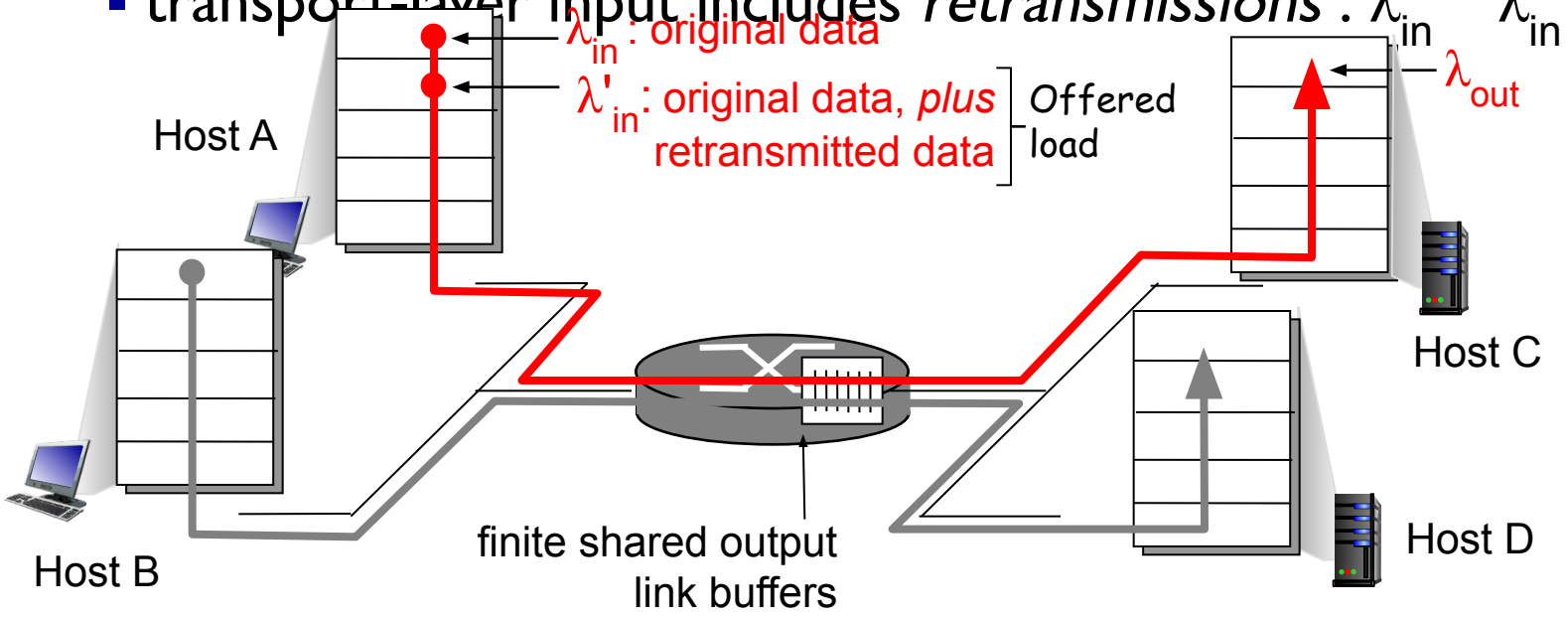
Throughput and Delay as a function of host sending Rate

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Scenario 2: Two senders, a Router with finite Buffers

- one router, *finite buffers* – *packet dropped if buffer full*
- Connection is reliable

  sender *retransmission* of timed-out packet
  - *performance depends on how retransmission is performed*
  - transport-layer input includes *retransmissions* : $\lambda'_{in} \geq \lambda_{in}$

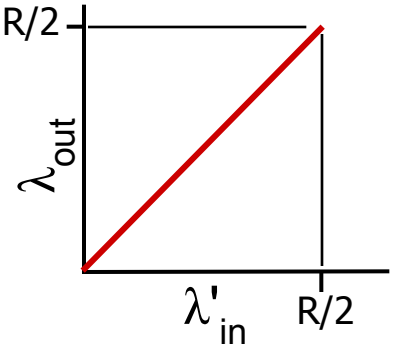


© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

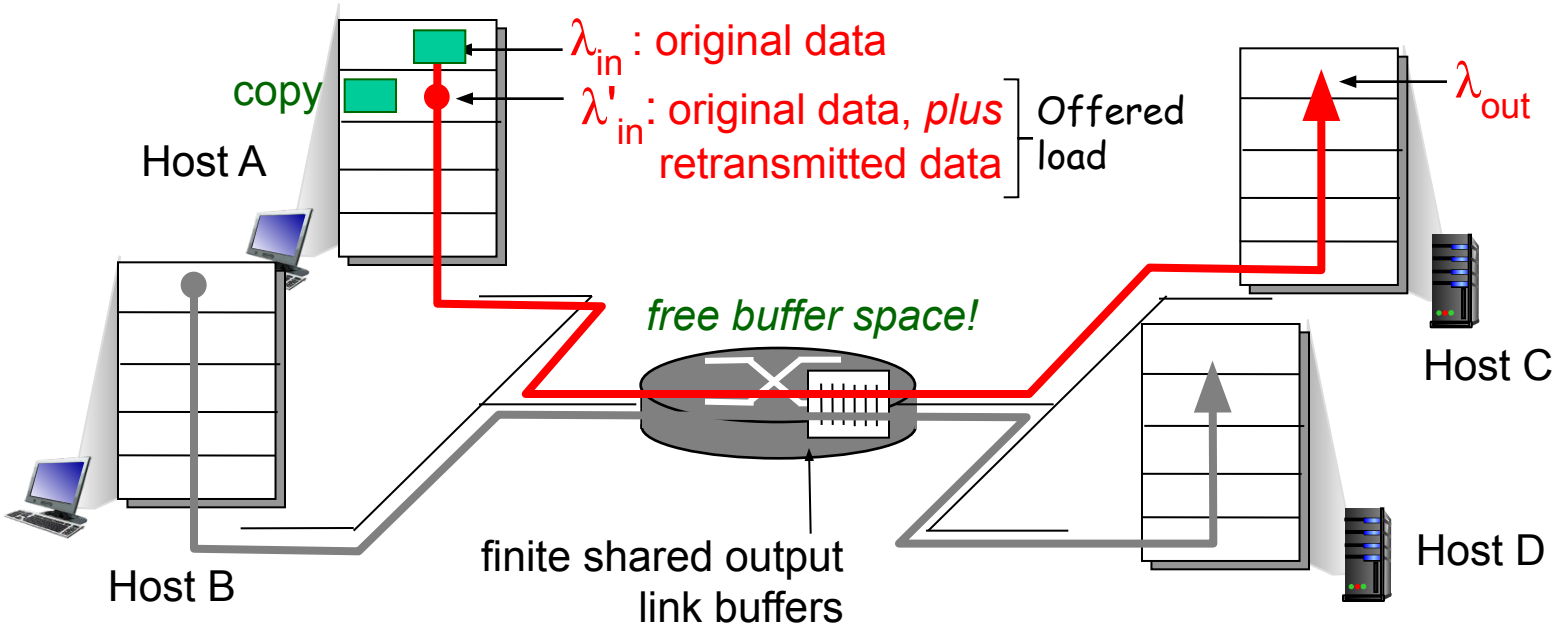
Scenario 2: Two senders, a Router with finite Buffers

idealization: perfect knowledge

- sender sends only when router buffers available
- No loss $\lambda_{in} = \lambda'_{in}$
- Throughput $\lambda_{out} = \lambda_{in}$



Cost: Average host sending rate can not exceed R/2



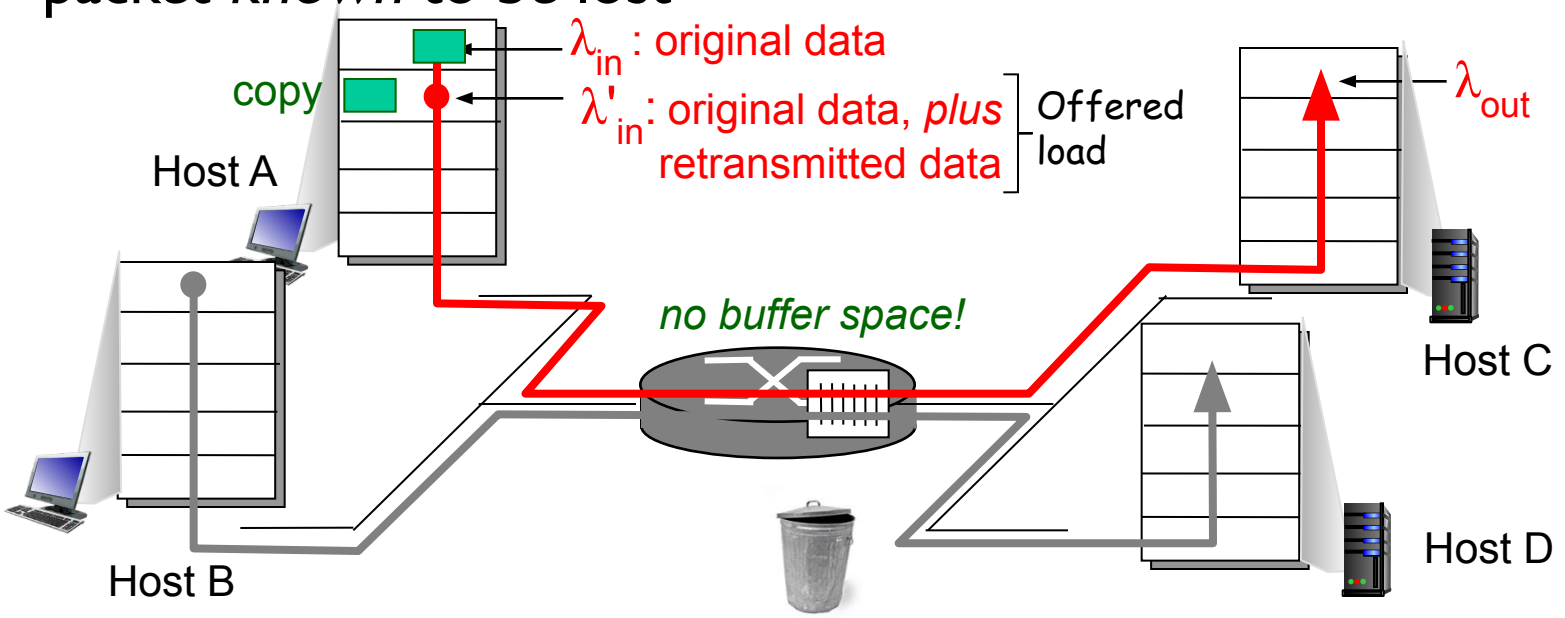
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Scenario 2: Two senders, a Router with finite Buffers

Idealization: known

loss packets can be lost, dropped at router due to full buffers

- sender only resends if packet *known* to be lost

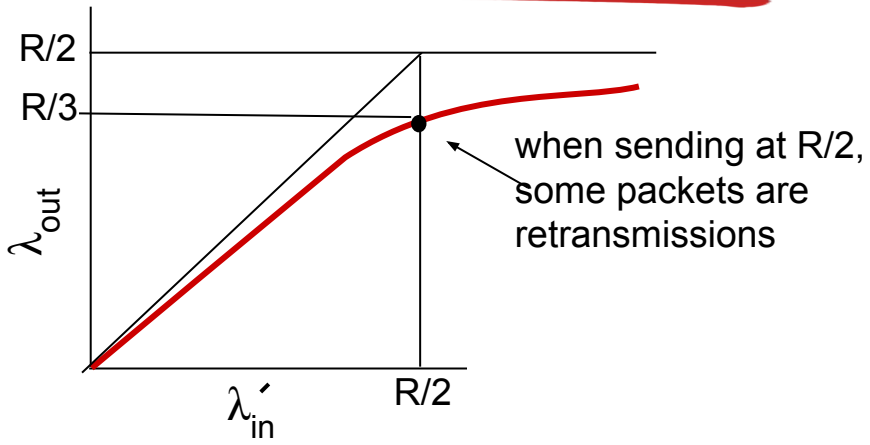


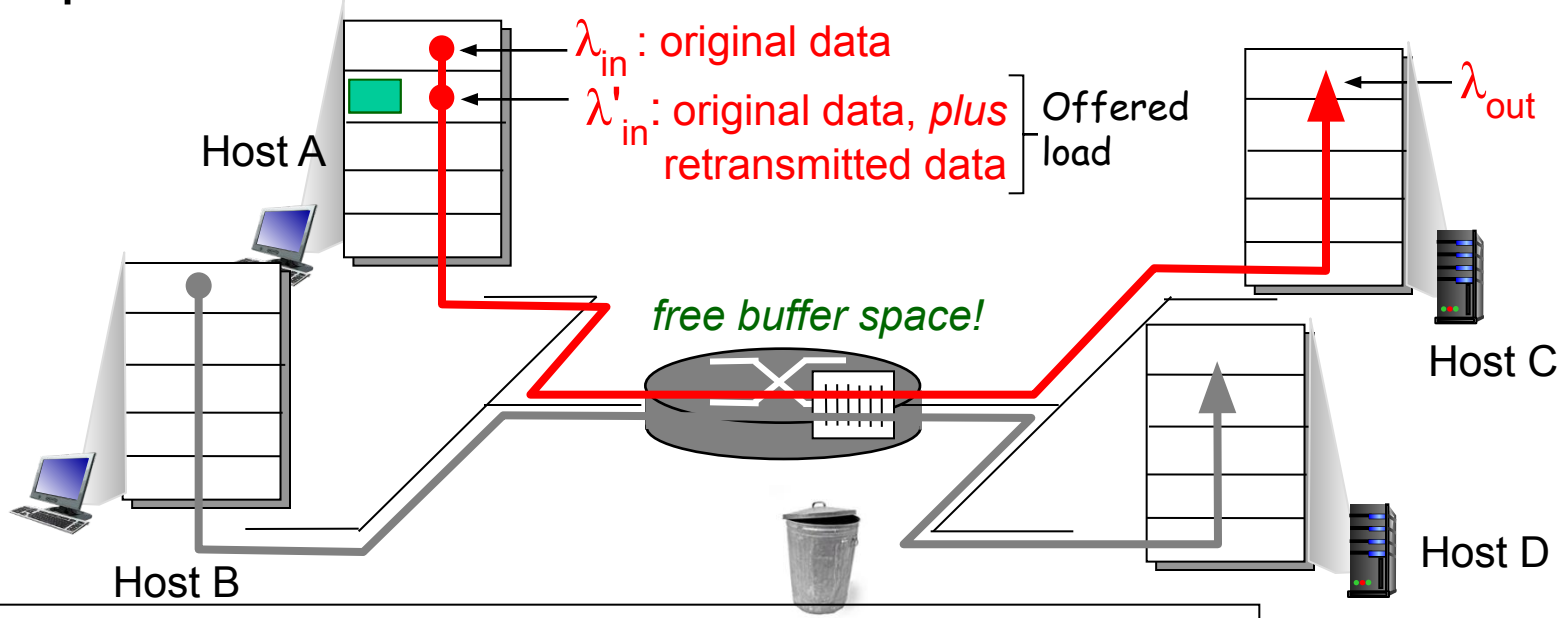
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Scenario 2: Two senders, a Router with finite Buffers

Idealization: known *loss* packets can be lost, dropped at router due to full buffers

- sender only resends if packet *known* to be lost





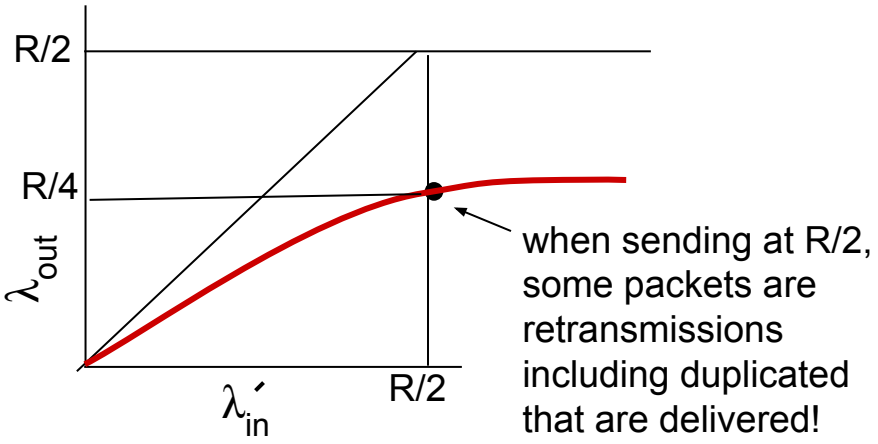
Cost: the sender must perform retransmissions in order to compensate for dropped (lost) packets due to buffer overflow

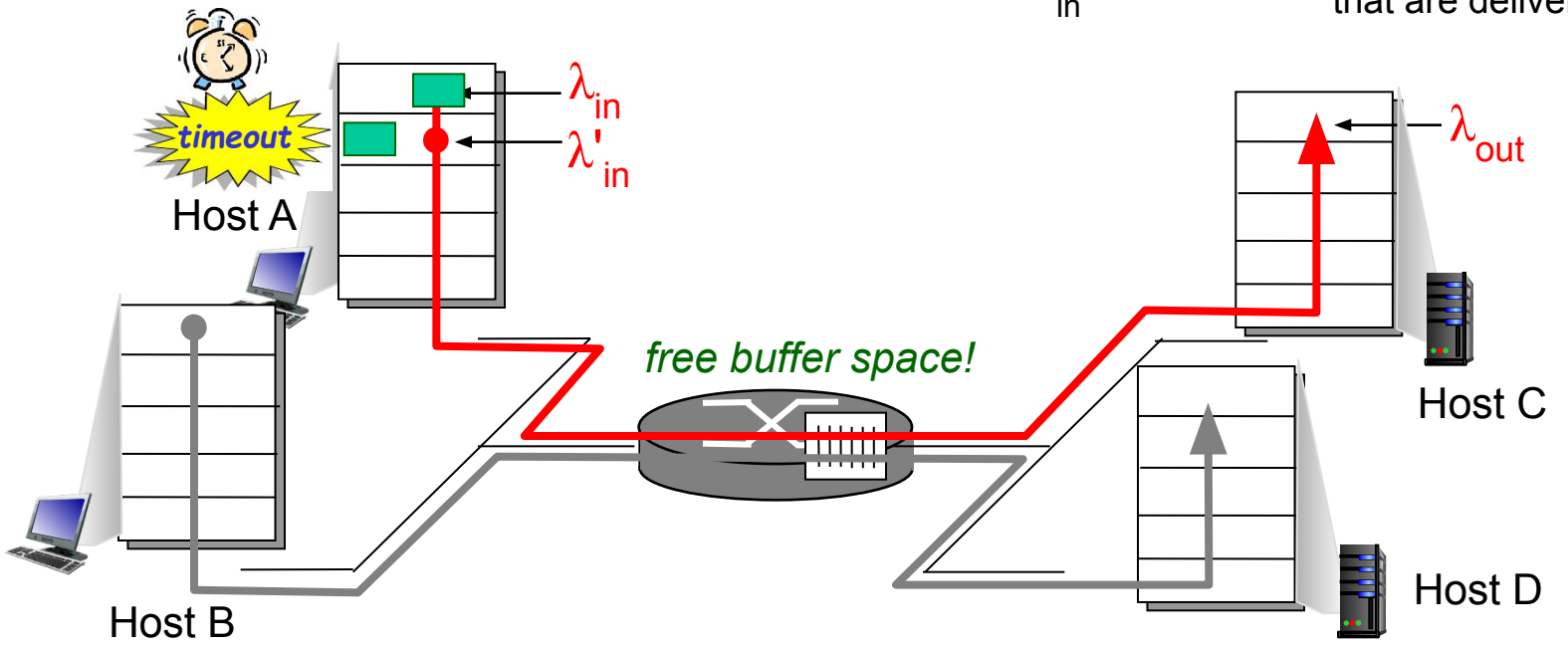
© J.F. Kurose, K.W. Ross. All Rights Reserved

Scenario 2: Two senders, a Router with finite Buffers

Realistic: duplicates

- *sender times out prematurely*, sending *two* copies, both of which are delivered



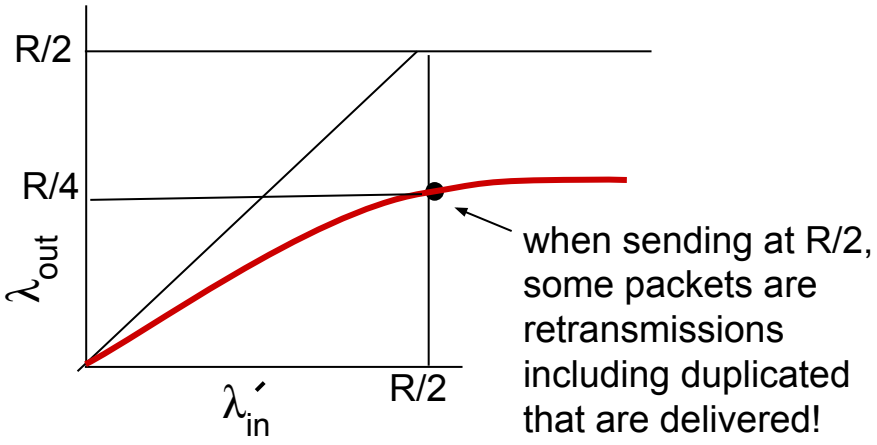


© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Scenario 2: Two senders, a Router with finite Buffers

Realistic: duplicates

- *sender times out prematurely*, sending *two* copies, both of which are delivered



"costs" of congestion:

- more work (retrans) for given "goodput"
- unneeded retransmissions: link carries multiple copies of pkt
  - decreasing goodput

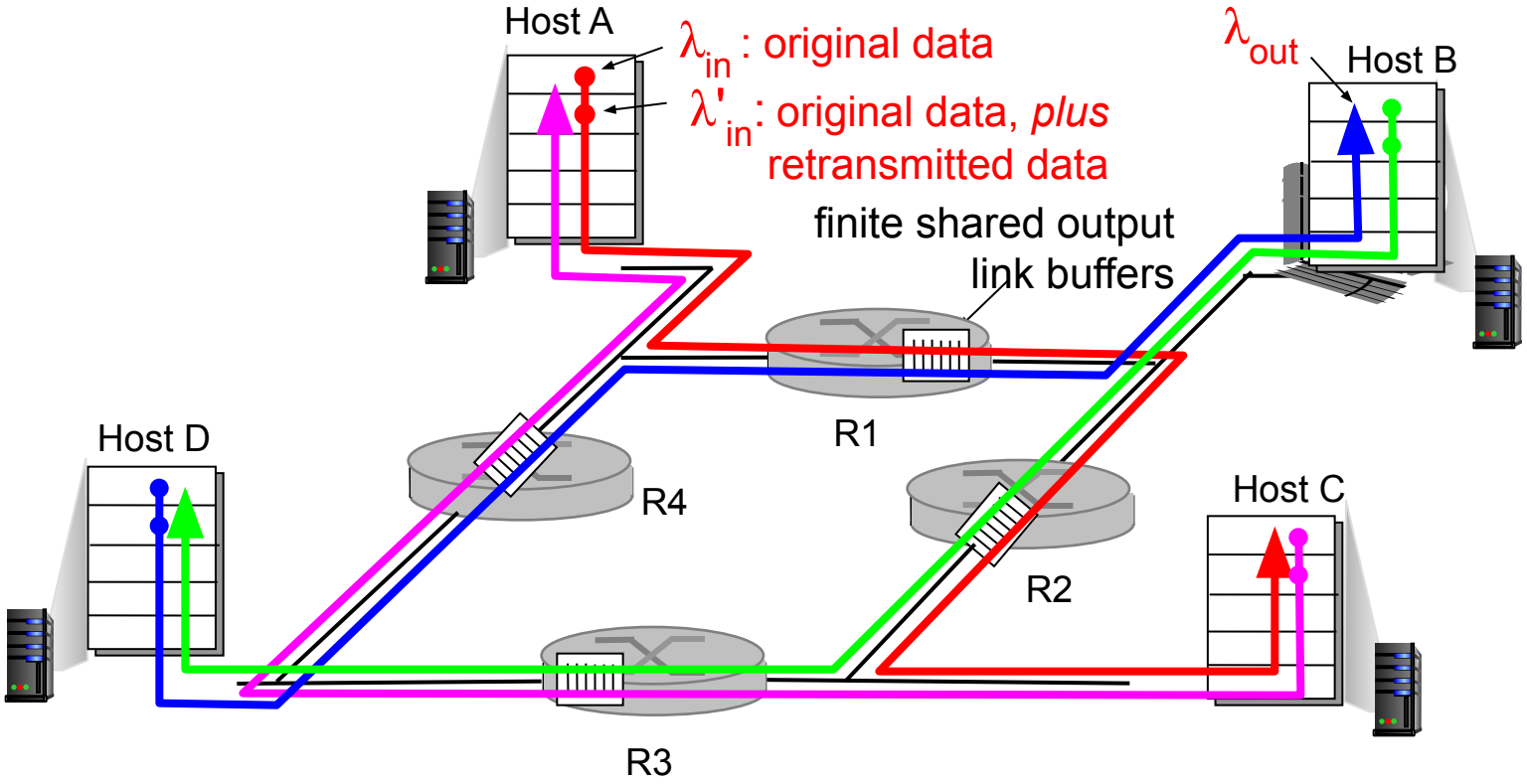
© J.F. Kurose, K.W. Ross. All Rights Reserved

Scenario 3: Four senders, Routers with finite Buffers & Multihop Paths

- four senders
- multihop paths
  - Overlapping two-hop paths
- timeout/retransmit
  - For reliable data transfer

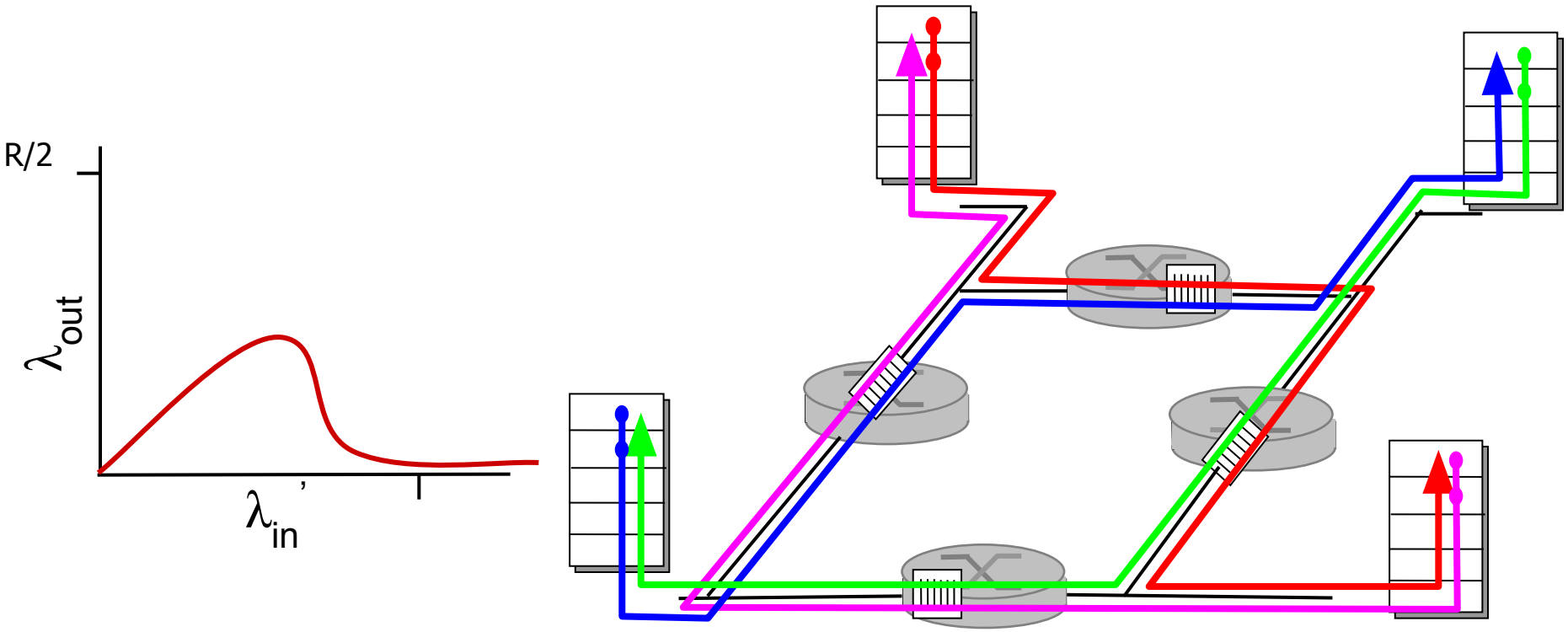
Q: what happens as λ_{in} and λ_{in}' increase ?

A: as red λ_{in}' increases, all arriving blue pkts at upper queue are dropped, blue throughput → 0



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Scenario 3: Four senders, Routers with finite Buffers & Multihop Paths



another "cost" of congestion:

- when packet dropped, any "upstream transmission capacity used for that packet was wasted!

© J.F. Kurose, K.W. Ross. All Rights Reserved

Approaches towards congestion control

two broad approaches towards congestion control:

end-end congestion control:

- no explicit feedback from network
- congestion inferred from end-system observed loss, delay
- approach taken by TCP

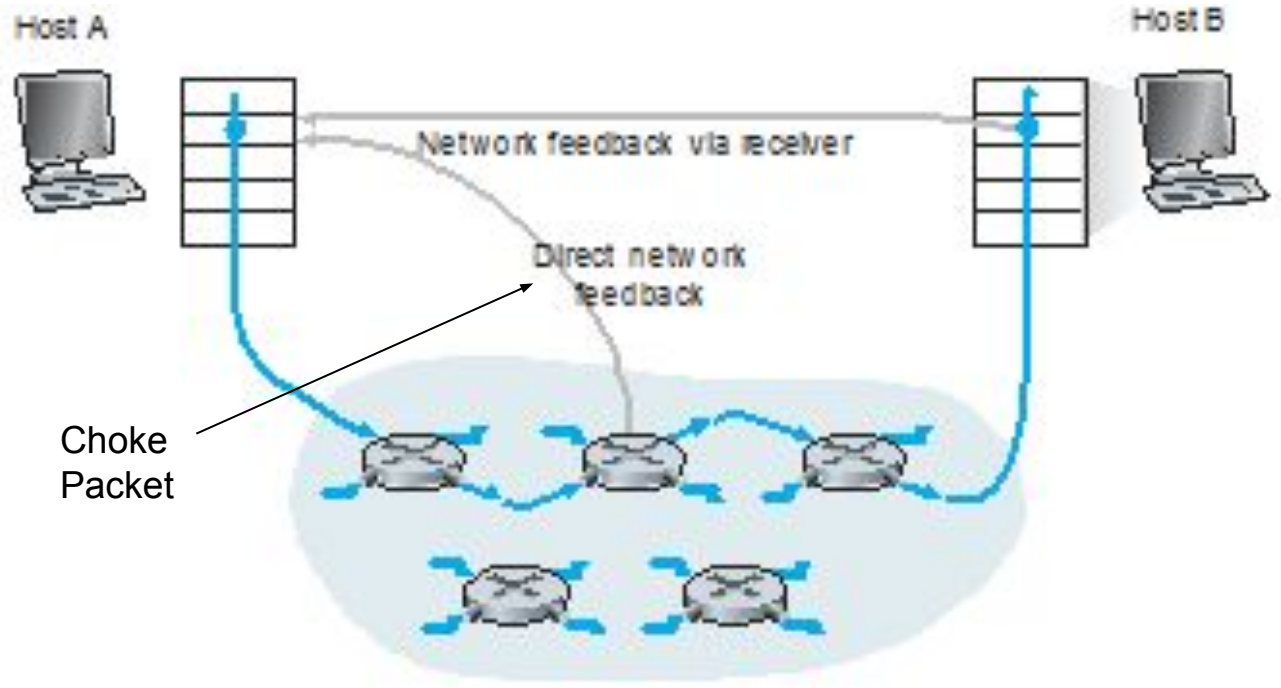
network-assisted congestion control:

- routers provide feedback to end systems
  - single bit indicating congestion (SNA, DECbit, TCP/IP ECN, ATM)
  - explicit rate for sender to send at

© J.F. Kurose, K.W. Ross. All Rights Reserved

Approaches towards congestion control

Network-assisted congestion control: congestion information is typically fed back from the network to the sender in one of two ways, as shown in the Figure below.



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Case study: ATM ABR congestion control

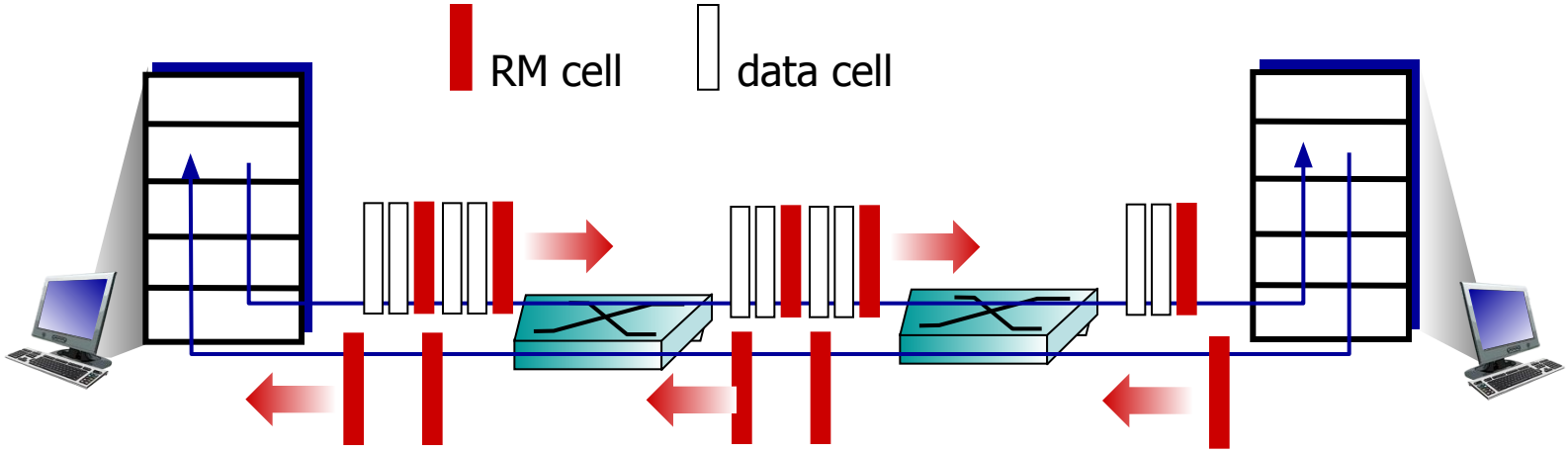
- Network Assisted congestion control
- ATM – Virtual Circuit based approach
- Rate Based Approach
  - Sender explicitly computes a maximum rate at which it can send and regulates itself accordingly

ABR: available bit rate:

- "elastic service"
- if sender's path "underloaded":
  - sender should use available bandwidth
- if sender's path congested:
  - sender throttled to minimum guaranteed rate

© J.F. Kurose, K.W. Ross. All Rights Reserved

Case study: ATM ABR congestion control

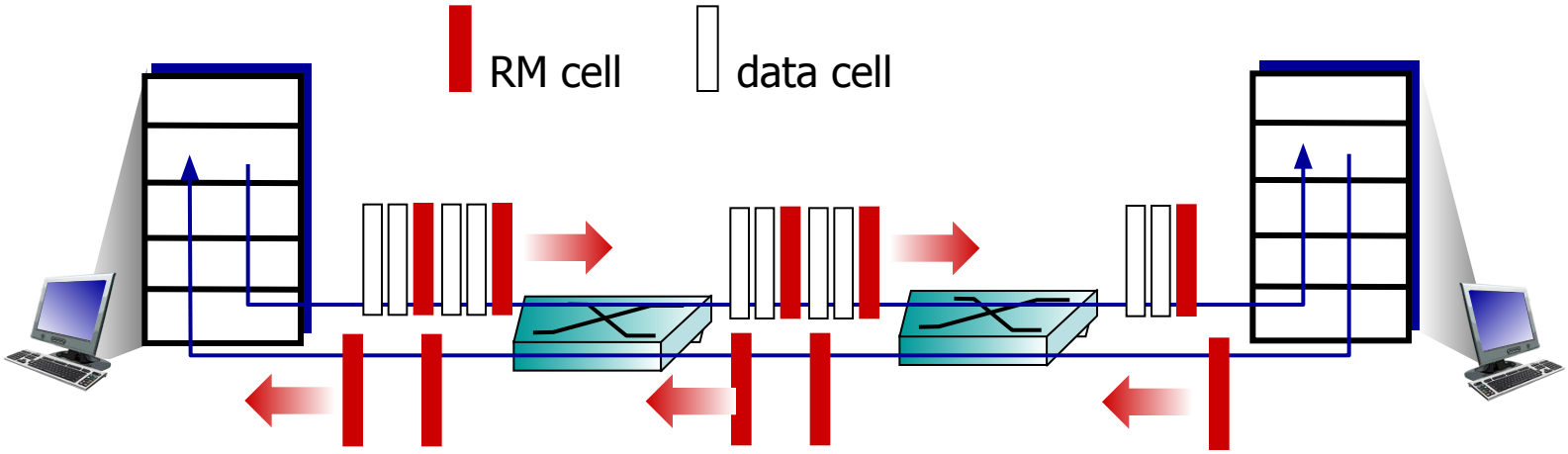


RM (resource management) cells:

- sent by sender, interspersed with data cells
- RM cells used to convey congestion related information
- Network feedback via receiver
  - RM cells turned around by destination
- Direct network feedback
  - Switch generate RM cells and send directly to source

© J.F. Kurose, K.W. Ross. All Rights Reserved

Case study: ATM ABR congestion control

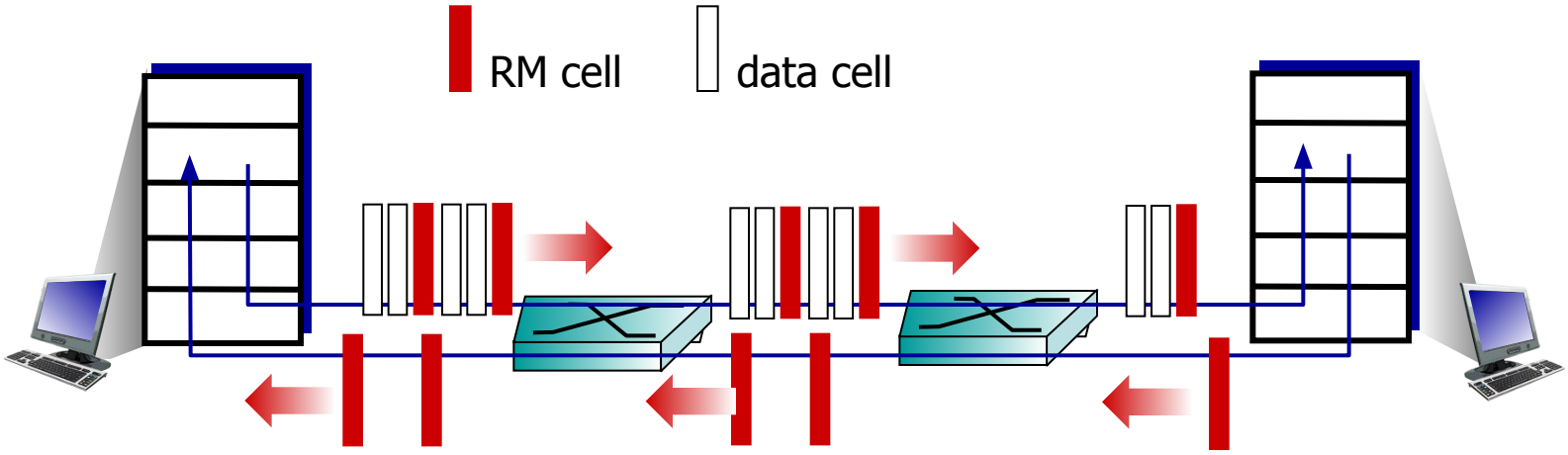


Three Mechanisms:

- EFCI bit – Explicit Forward Congestion Indication
- CI & NI Bits - Congestion Indication and No Increase
- ER Setting – Explicit Rate

© J.F. Kurose, K.W. Ross. All Rights Reserved

Case study: ATM ABR congestion control

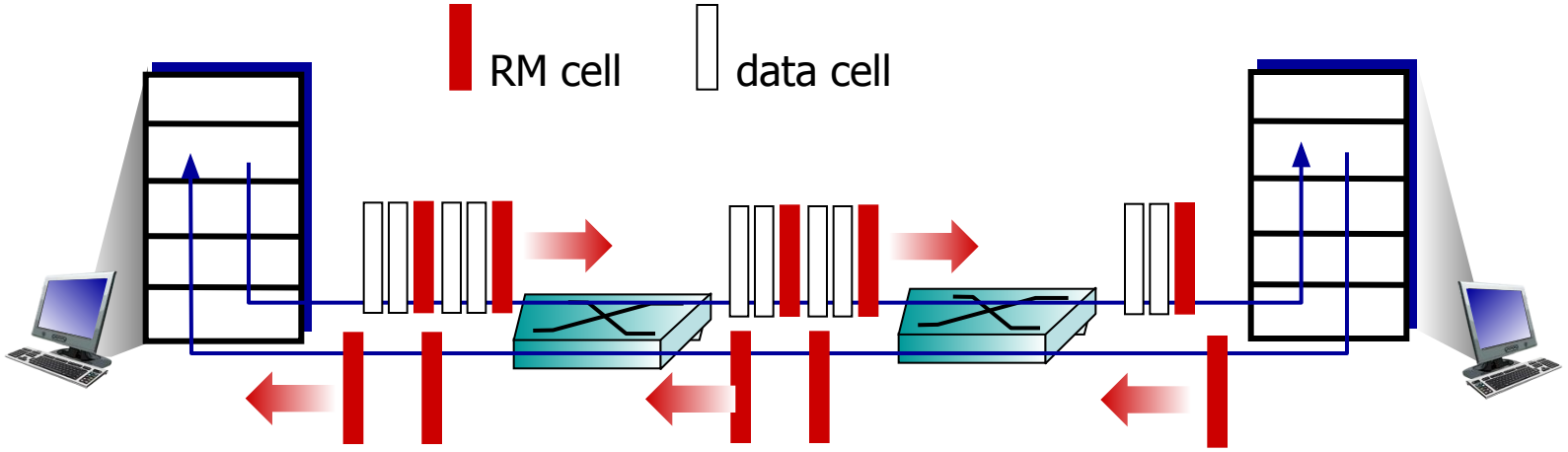


EFCI Bit:

- EFCI bit in data cells : set to 1 in congested switch
- if data cell preceding RM cell has EFCI set, receiver sets CI bit in returned RM cell

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Case study: ATM ABR congestion control

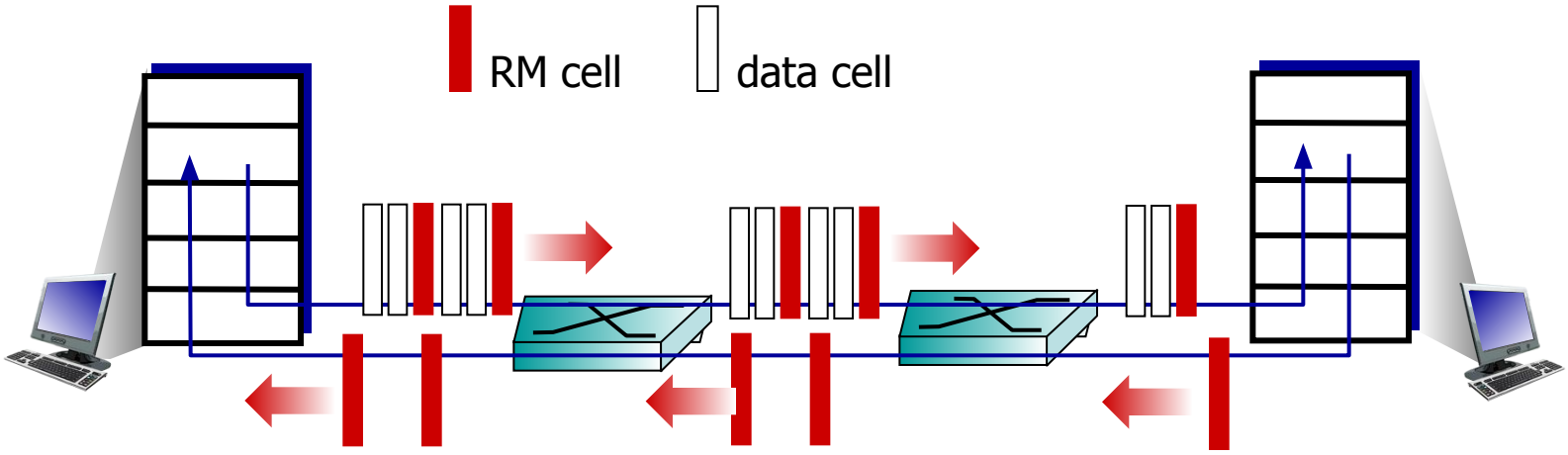


CI & NI Bits:

- RM cells sent by sender, interspersed with data cells
- One RM cell every 32 data cells
- bits in RM cell set by switches ("*network-assisted*")
  - *NI bit:* no increase in rate (mild congestion)
  - *CI bit:* congestion indication (severe congestion)
- RM cells returned to sender by receiver, with bits intact

© J.F. Kurose, K.W. Ross. All Rights Reserved

Case study: ATM ABR congestion control

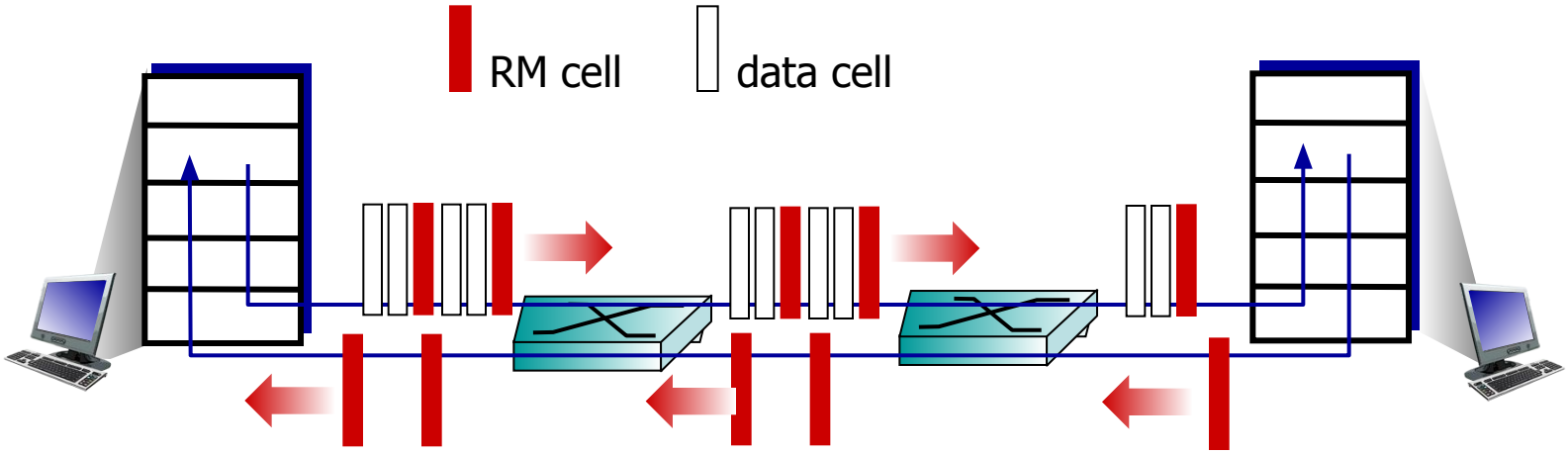


ER Setting:

- two-byte ER (explicit rate) field in RM cell
- congested switch may lower ER value in cell
- senders' send rate thus max supportable rate on path

© J.F. Kurose, K.W. Ross. All Rights Reserved

Case study: ATM ABR congestion control



Rate Adjustment:

- Source adjusts rate at which it can send cells as a function of the CI, NI and ER values in a returned RM cell

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3 outline

3.1 transport-layer services

3.2 multiplexing and demultiplexing

3.3 connectionless transport: UDP

3.4 principles of reliable data transfer

3.5 connection-oriented transport: TCP

- segment structure
- reliable data transfer
- flow control
- connection management

3.6 principles of congestion control

3.7 TCP congestion control

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Congestion Control: Overview

- end-end control (no network assistance)
- Approach
  - Limit send rate when network is congested
  - Mild congestion – increase rate
  - Severe congestion – reduce rate
- Questions
  - How to limit send rate?
  - How to perceive congestion?
  - How to change send rate?

  (What congestion control algorithm?)

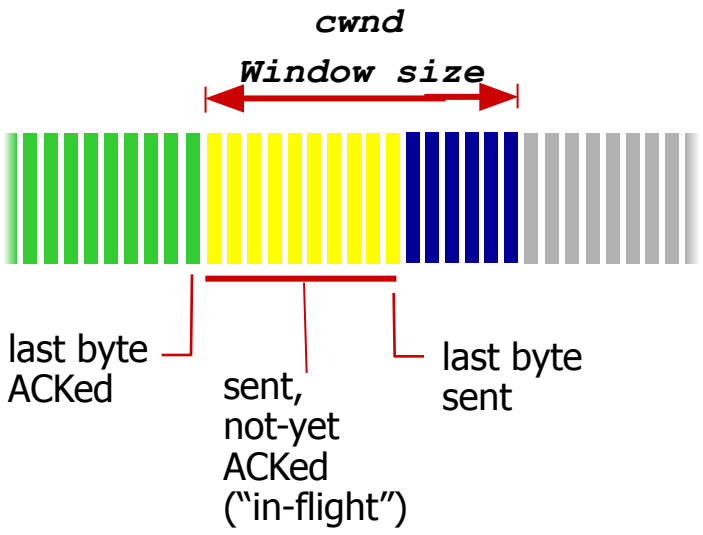
© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Congestion Control : Overview

How does a sender limit its rate?

- Flow Control - each side of a TCP connection
  - receive buffer, send buffer, and several variables (LastByteRead, rwnd, and so on).
- Congestion control
  - congestion window (cwnd)

sender sequence number space



- *sender limits transmission:*
  - Amount of unacknowledged data

$$\texttt{LastByteSent-LastByteAcked} \leq \texttt{min(cwnd, rwnd)}$$

- **rcvbuffer large, rwnd >cwnd**

$$\texttt{LastByteSent-LastByteAcked} \leq \texttt{cwnd}$$

- **cwnd** is dynamic, function of perceived network congestion

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

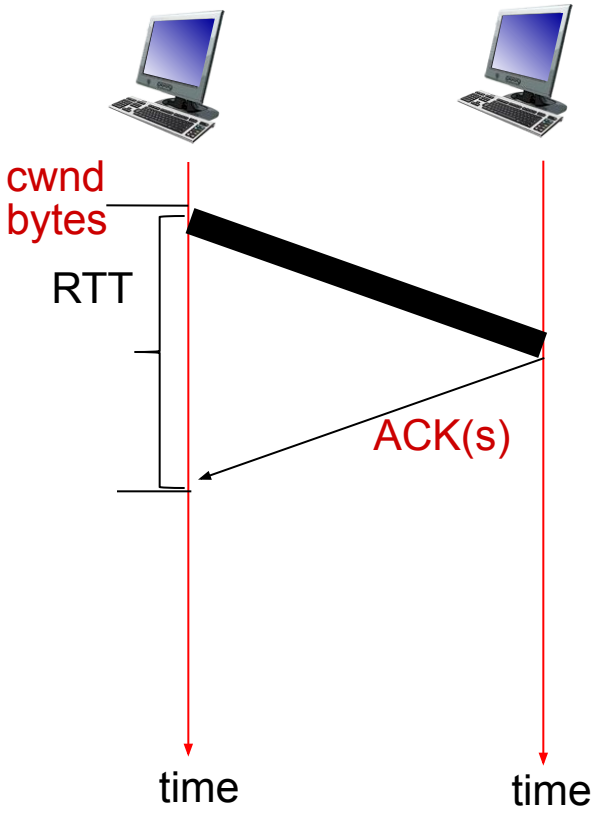
TCP Congestion Control : Overview

How does a sender limit its rate?

TCP sending rate:

- Assume loss and transmission delays are negligible
- *roughly:* send cwnd bytes, wait RTT for ACKS, then send more bytes

$$\text{rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$



© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Congestion Control: Overview

How does sender perceive congestion?

how congestion is detected?

- loss event = timeout *or* 3 duplicate acks
- Excessive congestion
  - router buffer overflow
  - datagram dropped
  - Loss event – timeout or 3 ACKs

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Congestion Control: Overview

TCP is self clocking

- *No congestion* – no loss event
- acks for previously unacked segments
  - increase its congestion window size --- its transmission rate
- acks arrive at a slow rate
  - High end-to-end delay
  - Low-bandwidth link

  ⇒ cwnd will be increased at a relatively slow rate
- acks arrive at a high rate

  ⇒ cwnd will be increased more quickly
- TCP uses acks to trigger (or *clock*) its increase in congestion window size

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Congestion Control: Overview

How should a TCP sender determine the rate at which it should send?

- If TCP senders send too fast
  - congest the network, leading to the congestion collapse
- If TCP senders send too slowly
  - under utilize the bandwidth in the network
- Guiding principles:
  - *lost segment implies congestion – reduce rate (timeout or 3ACKs)*
  - *acknowledged segment implies no congestion – increase rate*
  - *Bandwidth probing*
    - TCP sender increases its transmission rate to probe for the rate that at which congestion onset begins, backs off from that rate, and then to begins probing again to see if the congestion onset rate has changed.
    - Analogy – child who requests (and gets) more and more goodies

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Congestion Control: Overview

What congestion control algorithm?

AIMD – Additive Increase Multiplicative Decrease

three mechanisms :

- slow start
- congestion avoidance
- fast recovery

© J.F. Kurose, K.W. Ross. All Rights Reserved

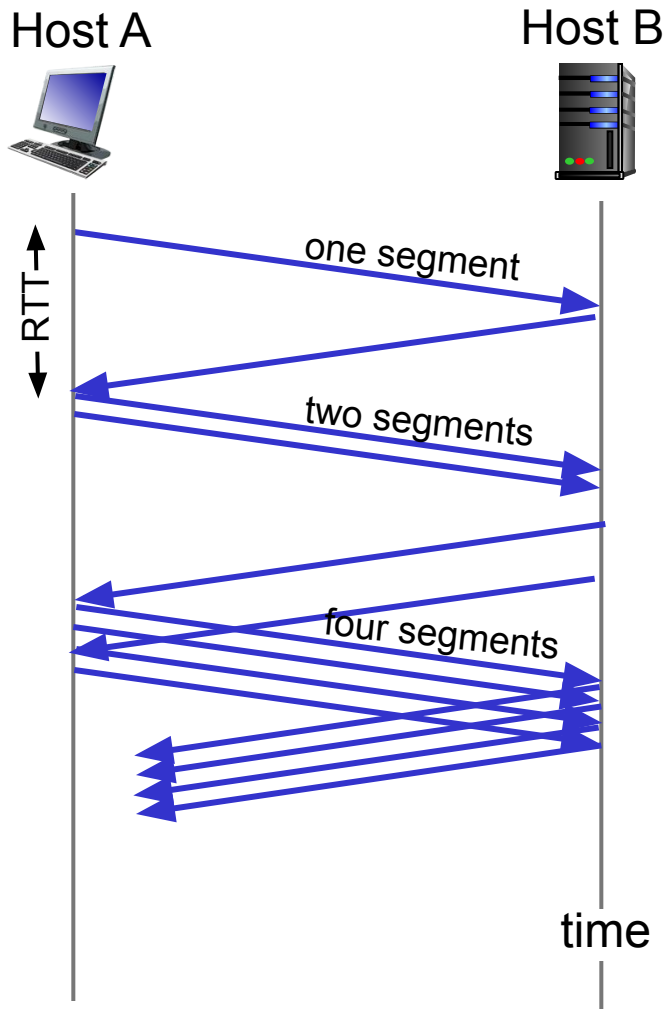
TCP Slow Start

- When connection begins, cwnd = 1 MSS [RFC 3390]
  initial sending rate= MSS/RTT
  - Example:
    MSS = 500 bytes
    RTT = 200 msec
    initial rate = 20 kbps
- available bandwidth may be >> MSS/RTT
  - desirable to quickly ramp up to respectable rate

© J.F. Kurose, K.W. Ross. All Rights Reserved

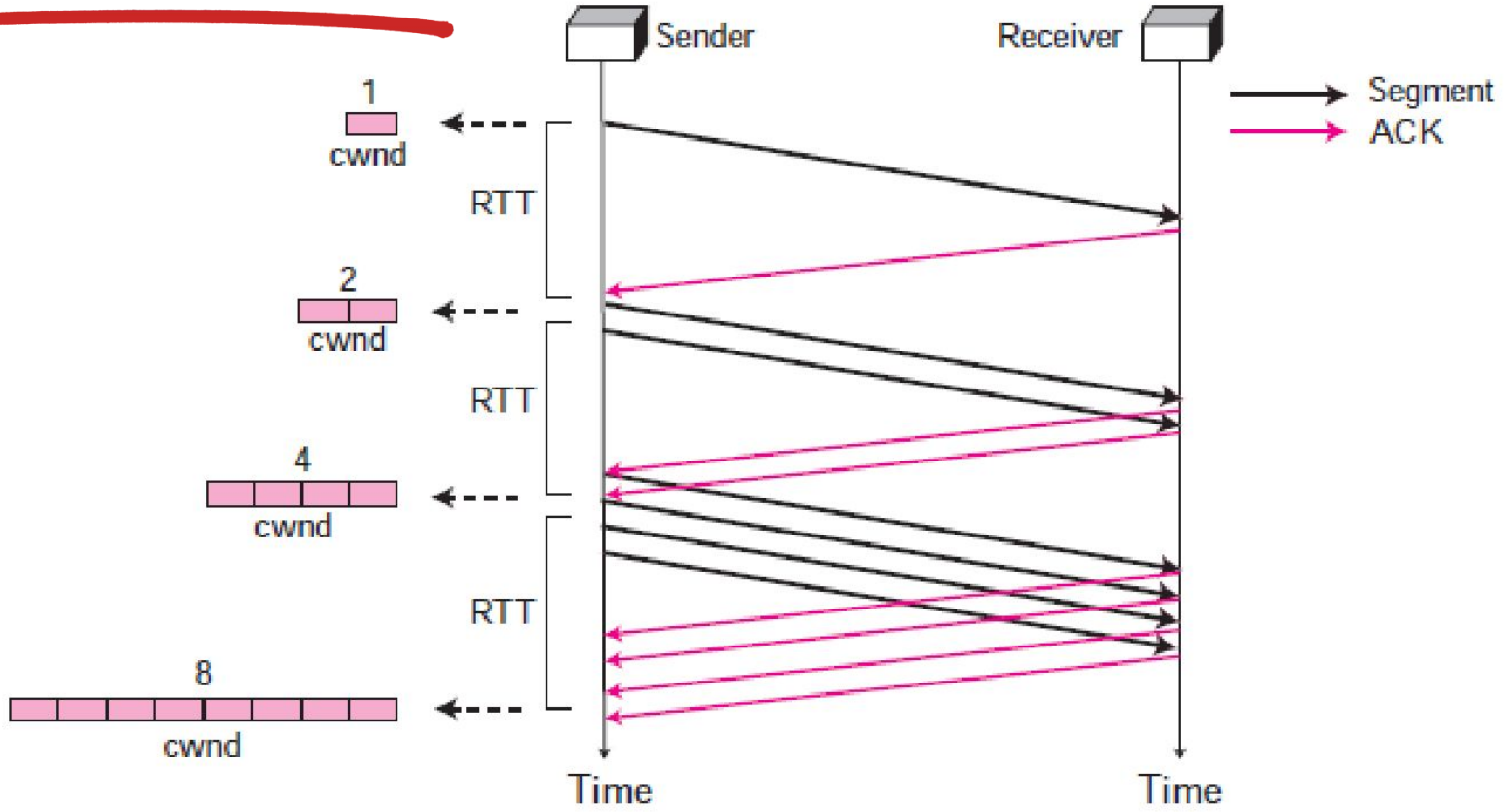
TCP Slow Start

- when connection begins, increase rate exponentially until first loss event:
  - initially **`cwnd`** = 1 MSS
  - double **`cwnd`** every RTT
  - done by incrementing **`cwnd`** for every ACK received
- *summary:* initial rate is slow but ramps up exponentially fast



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Slow Start



Start	→	cwnd = 1
After 1 RTT	→	cwnd = 1 × 2 = 2 → 2^1
After 2 RTT	→	cwnd = 2 × 2 = 4 → 2^2
After 3 RTT	→	cwnd = 4 × 2 = 8 → 2^3

© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

TCP Slow Start

When should this exponential growth end?

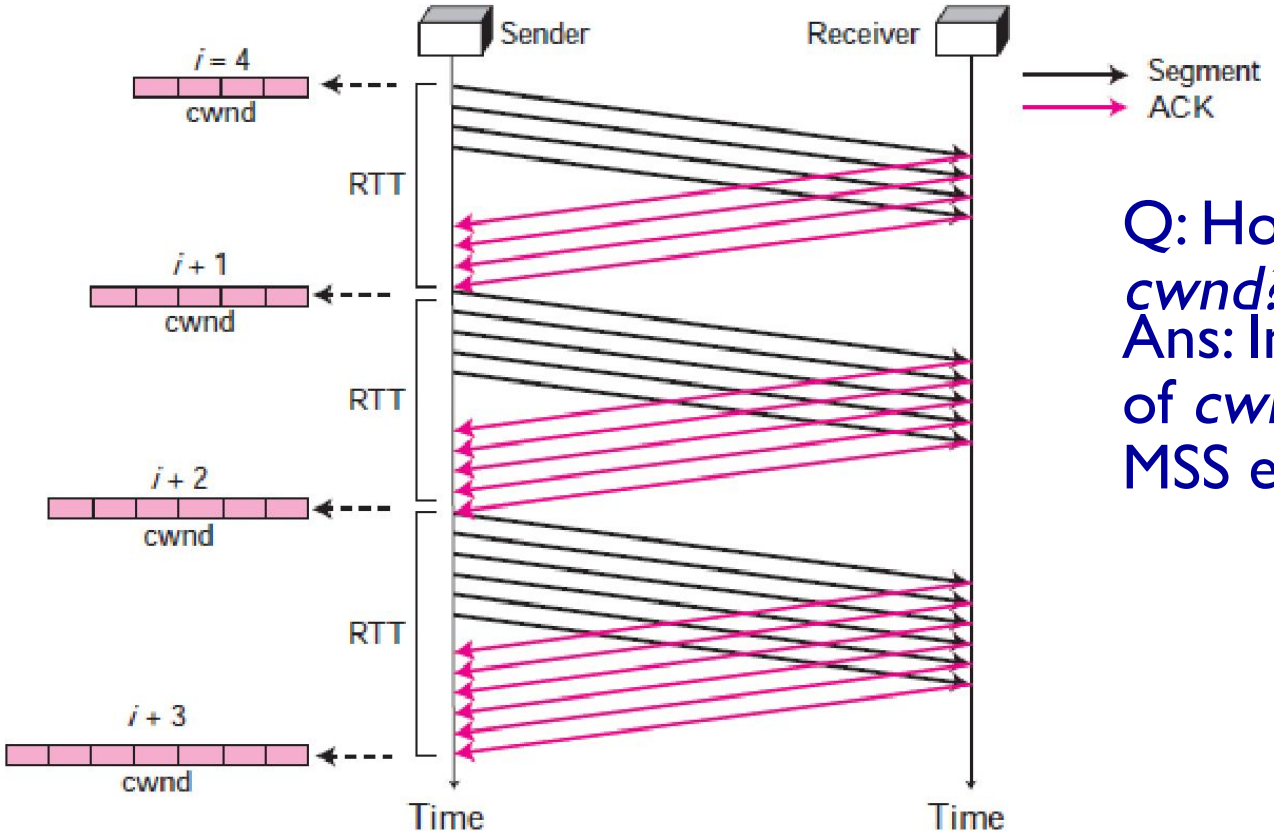
- Loss Event i.e. congestion (*Timeout*)
  - set cwnd=1 & ssthresh = **`cwnd/2`**
  - begin slow start process anew
- ssthresh
  - ssthresh = **`cwnd/2`**
  - when cwnd = ssthresh
  - slow start ends and TCP transitions into congestion avoidance mode
- Loss Event i.e. congestion (*3ACKs*)
  - TCP performs Fast Retransmit
  - TCP enters Fast recovery state

*ssthresh (slow start threshold)

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP congestion avoidance

Additive Increase:



Q: How to increase *cwnd?*
Ans: Increases the value of *cwnd* by just a single MSS every RTT

Start	→	**cwnd = *i***
After 1 RTT	→	**cwnd = *i* + 1**
After 2 RTT	→	**cwnd = *i* + 2**
After 3 RTT	→	**cwnd = *i* + 3**

© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

TCP congestion avoidance

Common approach

- Increase cwnd by MSS bytes (MSS/cwnd) whenever new ACK arrives
- For example
  - If MSS = 1460 bytes and cwnd = 14600 bytes
  - Then 10 segments are being sent within an RTT
  - Each arriving ACK(assuming one ACK per segment) increases the cwnd size by 1/10 MSS (cwnd += MSS/cwnd)
  - value of the cwnd will have increased by one MSS after ACKs when all 10 segments have been received.

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP congestion avoidance

When should congestion avoidance's linear increase (of 1 MSS per RTT) end?

- timeout occurs
- 3ACKs event

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP congestion detection

Multiplicative Decrease:

- timeout event
  - cwnd=1
  - ssthresh = **cwnd/2**
  - It starts the slow start phase again.
- 3ACKs event
  - ssthresh = **cwnd/2**
  - cwnd= ssthresh (plus 3MSS)
  - It starts the fast recovery state (AI)

© J.F. Kurose, K.W. Ross. All Rights Reserved

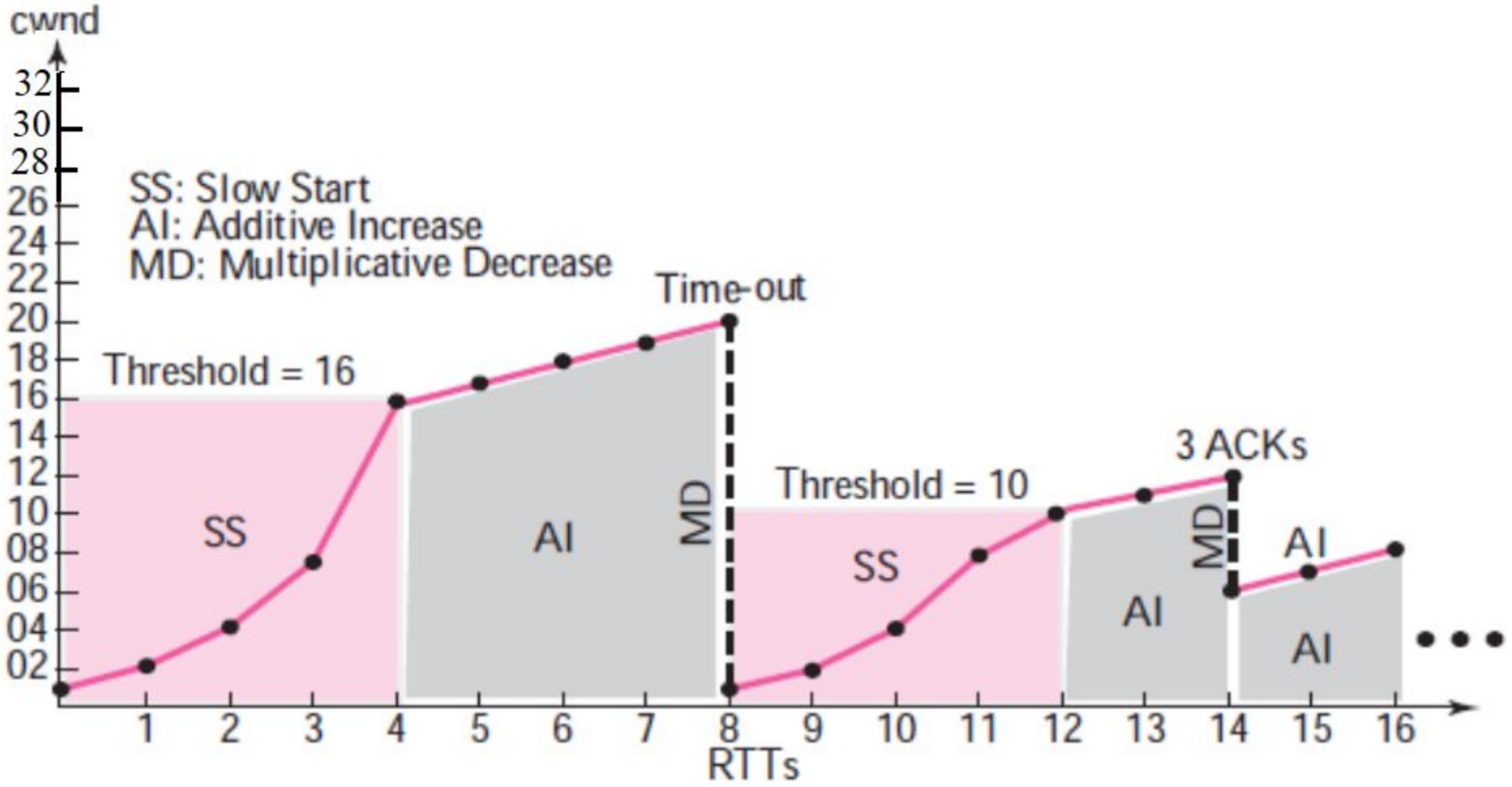
TCP fast recovery

Recommended but not required

- 3ACKs event
  - cwnd is increased by 1 MSS for every duplicate ACK received for the missing segment that caused TCP to enter the fast-recovery state.
- ACK arrives
  - when an ACK arrives for the missing segment, TCP enters the *congestion-avoidance state* after deflating cwnd.
- timeout event
  - fast recovery transitions to the *slow-start state* after performing the same actions as in slow start and congestion avoidance
  - cwnd = 1 MSS
  - ssthresh = **`cwnd/2`**

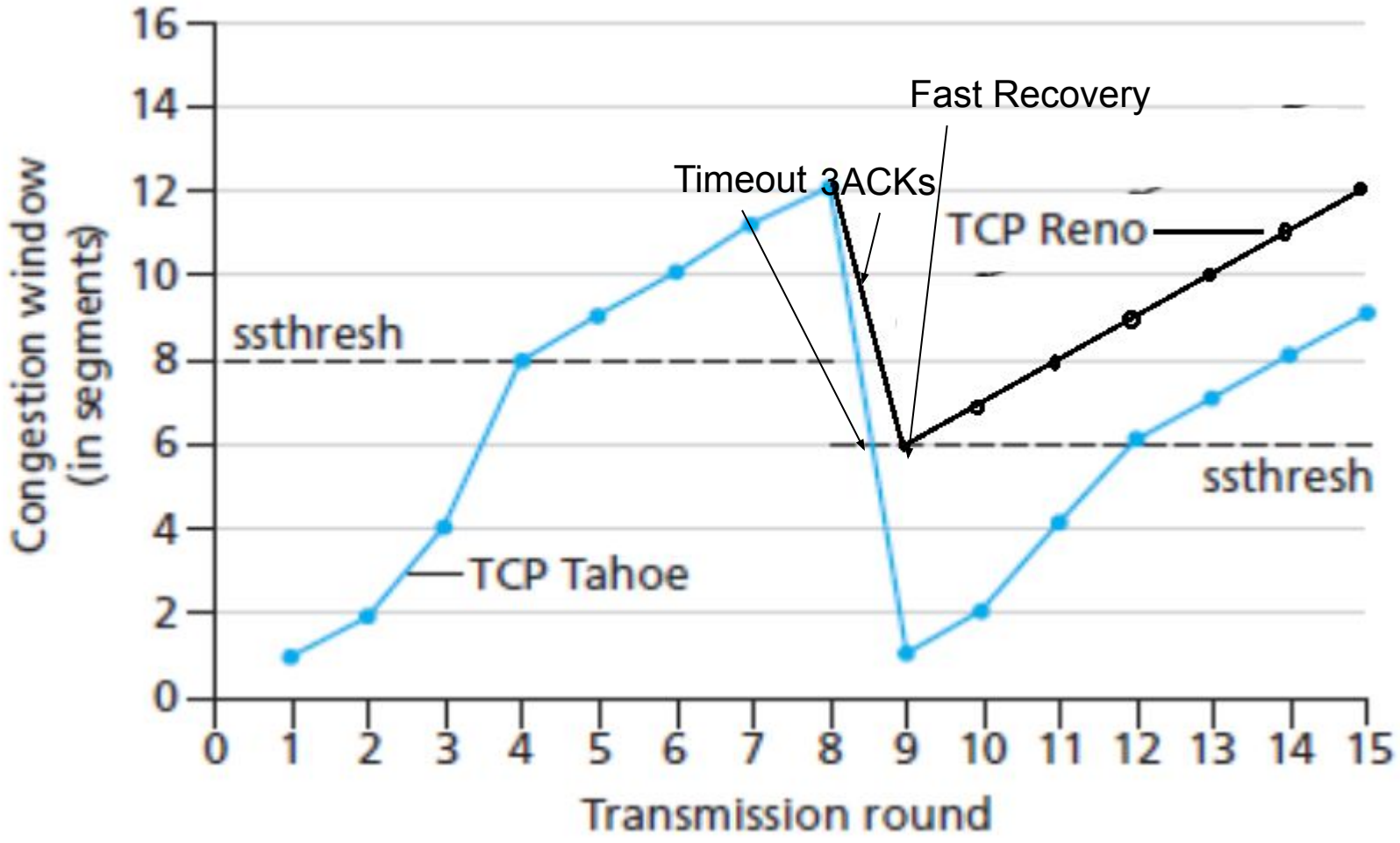
© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP congestion example



© Behrouz A. Forouzan. All Rights Reserved
http://highered.mheducation.com

Popular Flavors of TCP



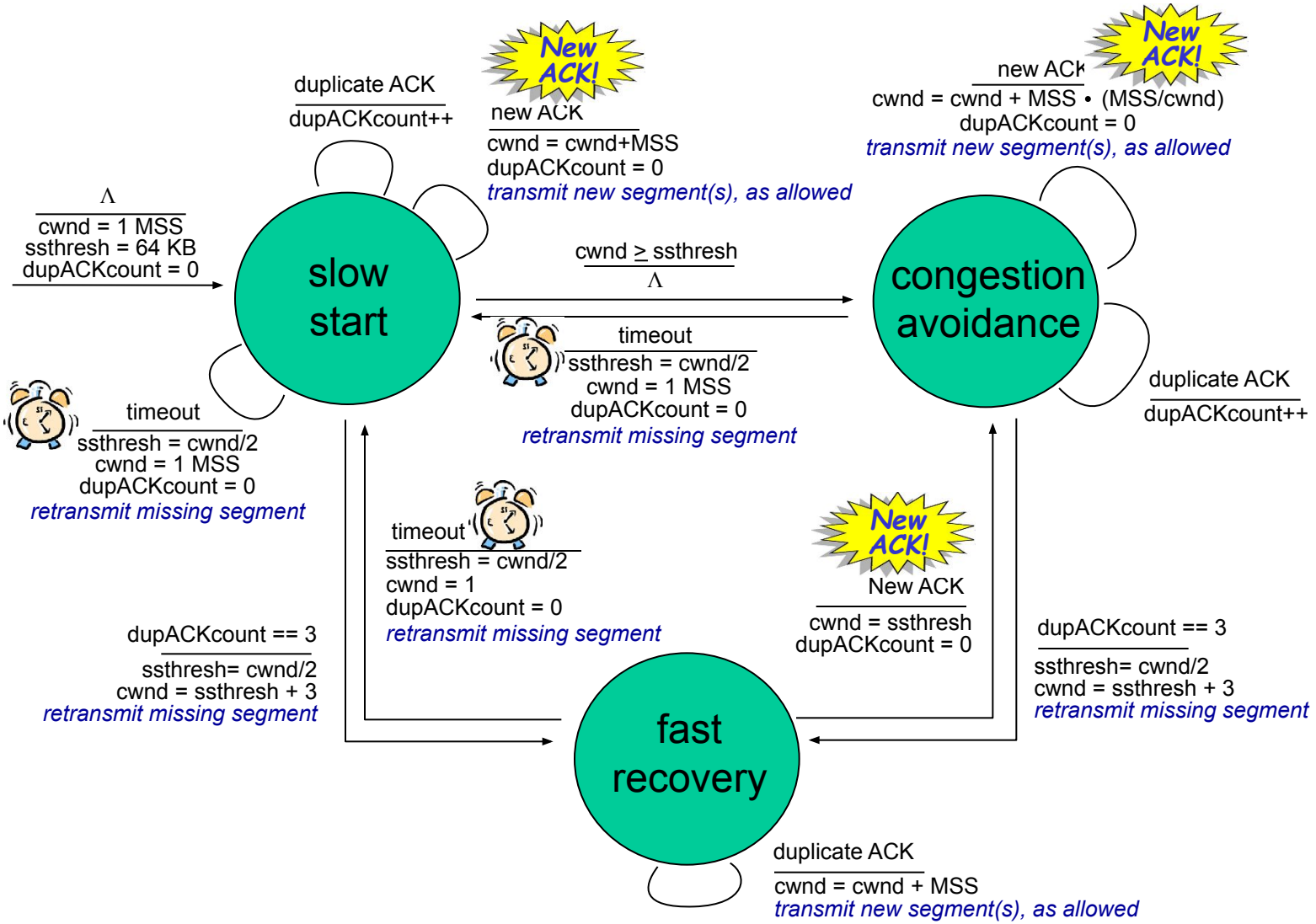
© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Summary: TCP Congestion Control

- When cwnd is below **`Threshold`**, sender in slow-start phase, window grows exponentially.
- When cwnd is above **`Threshold`**, sender is in congestion-avoidance phase, window grows linearly.
- When a triple duplicate ACK occurs, **`Threshold`** set to cwnd/2 and cwnd set to **`Threshold`**.
- When timeout occurs, **`Threshold`** set to cwnd/2 and cwnd is set to 1 MSS.

© J.F. Kurose, K.W. Ross. All Rights Reserved

Summary: TCP Congestion Control

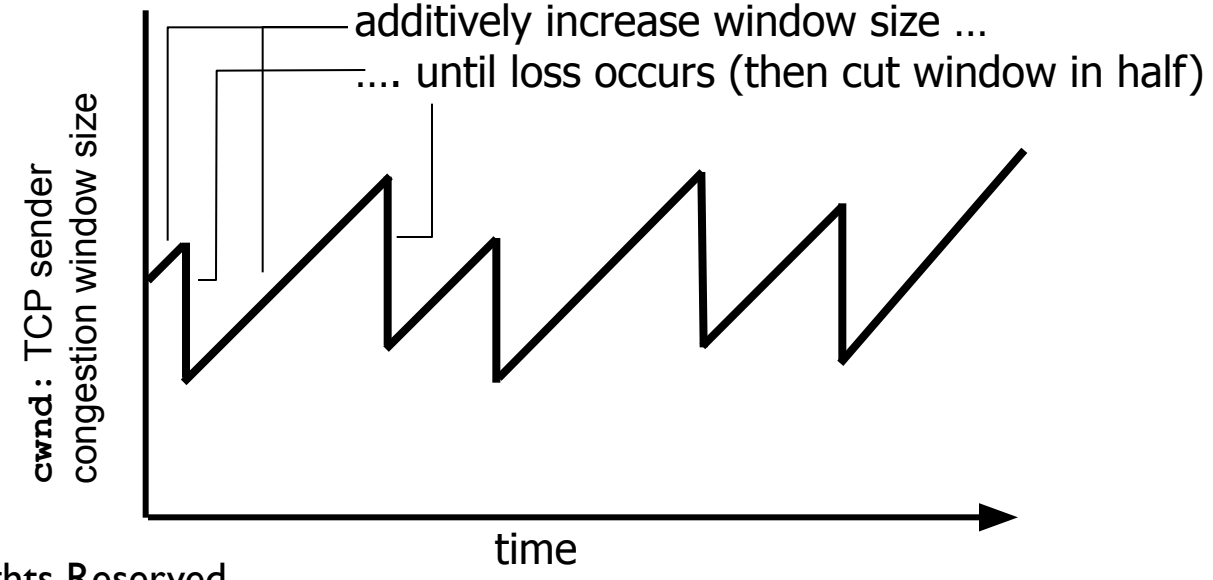


© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP congestion control: Retrospective

- *Ignore* initial slow start period when connection begins
- *Assume* Losses are indicated by 3ACKs rather than timeouts
- *approach:* sender increases transmission rate (window size), probing for usable bandwidth, until loss occurs
  - *additive increase:* increase **`cwnd`** by 1 MSS every RTT until loss detected
  - *multiplicative decrease:* cut **`cwnd`** in half after loss

AIMD saw tooth behavior: probing for bandwidth



© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

TCP Vegas

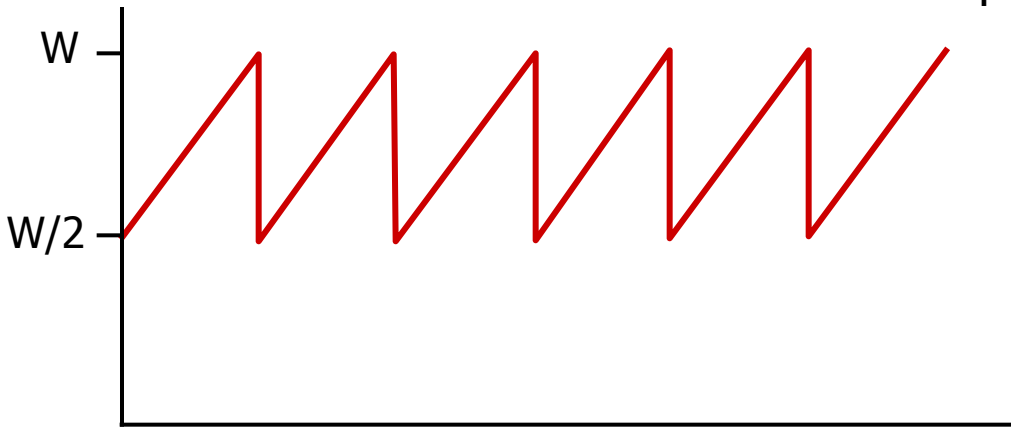
- *Avoid congestion* while maintaining *good throughput*
- Basic idea
  1. detect congestion *before* packet loss occurs
  2. lower the rate linearly when this imminent packet loss is detected
- Imminent packet loss is *predicted* by observing the RTT
  - The longer the RTT of the packets, the greater the congestion in the routers

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP throughput

- avg. TCP thruput as function of cwnd & RTT?
  - ignore slow start after timeout, assume always data to send
- W: window size (measured in bytes) when loss event occur
  - When window is w bytes and round trip time is RTT secs, *throughput is w/RTT* – TCP probes for additional B/W
  - Just after loss, window drops to W/2, *throughput to W/2RTT*.
  - avg. window size (# in-flight bytes) is ¾ W
  - avg. thruput is 3/4W per RTT

$$\text{avg TCP thruput} = \frac{3}{4}\frac{W}{RTT} \text{ bytes/sec}$$



- Saw-tooth behavior of TCP
- Steady state behavior of TCP

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Futures: TCP over "long, fat pipes"

- TCP Traffic
  - SMTP, FTP, Telnet ➜ HTTP ➜ P2P ➜ Grid & Cloud Computing
- High speed TCP connections
- example: TCP connection with 1500 byte segments, 100ms RTT, want 10 Gbps throughput, throughput=W/RTT
- requires avg window size W = 83,333 in-flight segments
- throughput in terms of segment loss probability, L (Loss Rate) [Mathis 1997]:

$$\text{TCP throughput} = \frac{1.22 \times \text{MSS}}{\text{RTT}\sqrt{L}}$$

➜ to achieve 10 Gbps throughput, need a loss rate of $L = 2\times10^{-10}$ *– a very small loss rate!*

one loss event for every 5,000,000,000 segments

- new versions of TCP for high-speed needed!

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Fairness

fairness goal: if *K TCP connections,* each with a different end-to-end path, share same *bottleneck link of bandwidth R bps*, a congestion control mechanism is said to be fair if each connection should have *average rate of R/K*

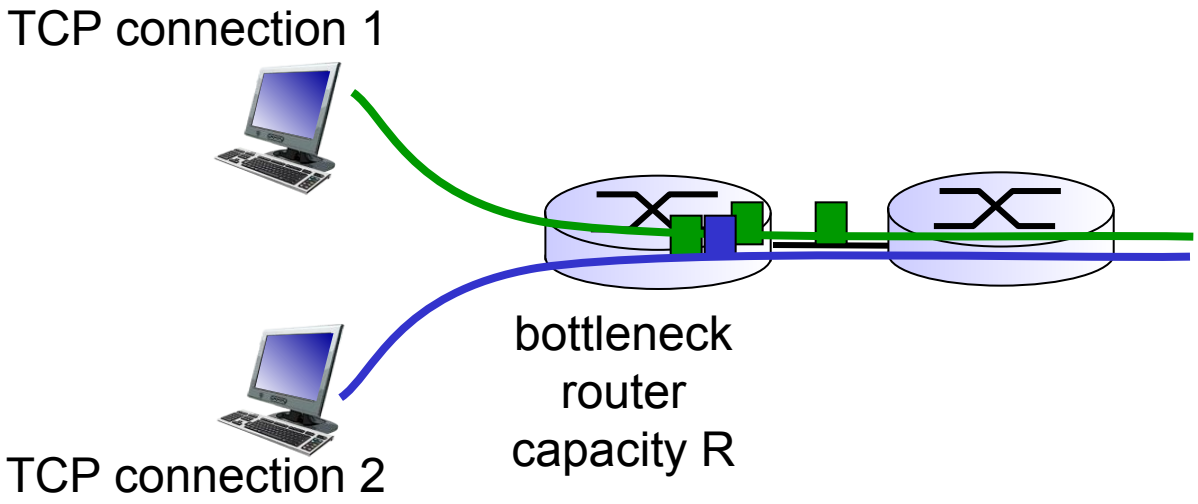
- Bottleneck Link
  - for each connection, all the other links along the connection's path are not congested and have abundant transmission capacity as compared with the transmission capacity of the bottleneck link.
- Assume
  - each connection is transferring a large file
  - there is no UDP traffic passing through the bottleneck link

© J.F. Kurose, K.W. Ross. All Rights Reserved

TCP Fairness

Is TCP's AIMD algorithm fair? Given that different TCP connections may start at different times and may have different window sizes

- TCP congestion control (AIMD) converges to provide an equal share of a bottleneck link's bandwidth among competing TCP connections



- Assume
  - Same MSS & RTT
  - If same cwnd then same throughput
  - Large data to send
  - No other TCP connection
  - No other UDP datagram

- Ignore slow start phase of TCP
- Assume TCP connections are operating in CA mode (AIMD) at all times

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

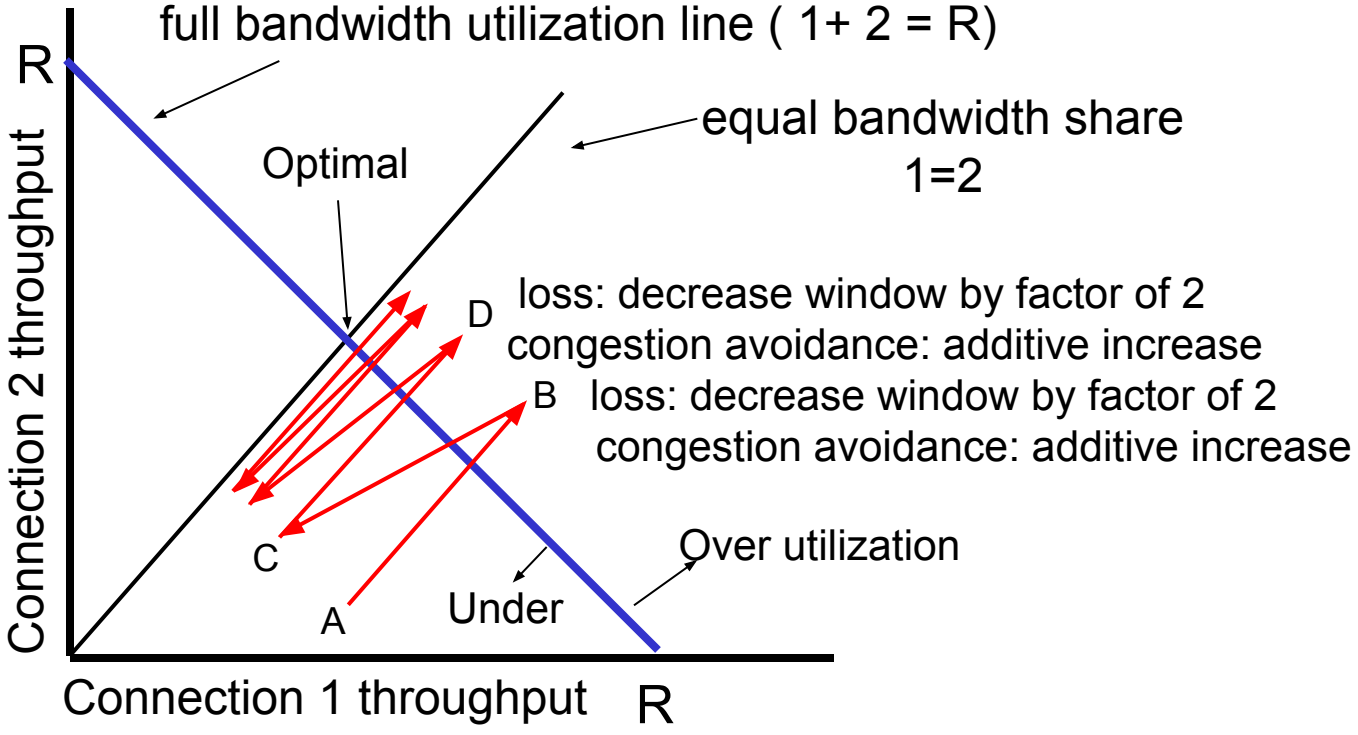
Why is TCP fair?

Fairness: Everyone gets "Fair Share"

Efficiency: Network resources are used well

two competing connections:

- additive increase gives slope of 1, as throughout increases
- multiplicative decrease decreases throughput proportionally



Phase Plot – Fairness and Efficiency

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/

Fairness (more)

Fairness and UDP

- multimedia apps often do not use TCP
  - do not want rate throttled by congestion control
- instead use UDP:
  - send audio/video at constant rate, tolerate packet loss
- Research area: TCP friendly

Fairness and parallel TCP connections

- application can open multiple parallel connections between two hosts
- web browsers do this
- e.g., link of rate R shared with 9 applications with 9 existing connections:
  - new app asks for 1 TCP connection, gets rate R/10
  - new app asks for 11 parallel TCP connections, gets > R/2

© J.F. Kurose, K.W. Ross. All Rights Reserved

Chapter 3: summary

- principles behind transport layer services:
  - multiplexing, demultiplexing
  - reliable data transfer
  - flow control
  - congestion control
- instantiation, implementation in the Internet
  - UDP
  - TCP

next:

- leaving the network "edge" (application, transport layers)
- into the network "core"

© J.F. Kurose, K.W. Ross. All Rights Reserved

END

© J.F. Kurose, K.W. Ross. All Rights Reserved
http://gaia.cs.umass.edu/kurose-ross-ppt-6e/